

PCCP 75 BÀI

FULL VIETNAMESE EXPLAINED SOLUTIONS

JavaScript • Đề bài tiếng Việt • Tư duy • Invariant • Code sạch • Trap • Complexity

Mục tiêu: mở một bài bất kỳ và học được từ đầu mà không cần quay lại đề gốc.

Cách dùng tài liệu

- 1) Đọc “Đề bài tiếng Việt” và tự nói lại output cần trả về.
- 2) Nhìn “Dấu hiệu nhận diện” để đoán pattern trước khi đọc lời giải.
- 3) Đọc “Tư duy từng bước” và invariant/state. Đây là phần cần thuộc hơn code.
- 4) Tự code 10-15 phút rồi mới mở block JavaScript.
- 5) Cuối cùng đọc trap và tự dry-run một test nhỏ.

Lưu ý: phần đề bài được diễn giải đầy đủ bằng tiếng Việt để học, không phải bản dịch máy từng chữ.

1. 개인정보 수집 유효기간

Thời hạn hiệu lực thu thập thông tin cá nhân

★ **BẮT BUỘC** theo roadmap team kia

Chủ đề	Simulation	Pattern	Date normalization / parsing
Priority	A	Complexity	O(n)

A. Đề bài tiếng Việt - đọc để hiểu đề

Hôm nay là một ngày cụ thể. Mỗi loại điều khoản bảo quản thông tin cá nhân có thời hạn hiệu lực tính theo số tháng, và mọi tháng được quy ước có đúng 28 ngày. Mỗi bản ghi gồm ngày thu thập và mã điều khoản áp dụng. Khi đã qua hoặc đúng ngày hết hạn thì bản ghi phải bị hủy. Hãy trả về chỉ số (đánh số từ 1) của tất cả bản ghi đã hết hạn tại ngày hôm nay, theo thứ tự tăng dần.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: Ngày-tháng, cộng kỳ hạn, so sánh hạn
- **Pattern**: Date normalization / parsing
- **Vì sao**: Đổi YYYY.MM.DD -> tổng số ngày theo lịch giả 28 ngày; Map term->months; expire = start + term*28 - 1

C. Tư duy lời giải từng bước

1. Đổi YYYY.MM.DD -> tổng số ngày theo lịch giả 28 ngày.
2. Map term->months.
3. expire = start + term*28 - 1.

Invariant / state phải giữ

Sau khi đổi mọi ngày sang cùng một thang ngày 28-ngày/tháng, so sánh thời gian chỉ còn là so sánh số nguyên; bản ghi hết hạn khi $\text{start} + \text{term} * 28 \leq \text{today}$.

D. JavaScript solution() - block code để học/copy

```
function solution(today, terms, privacies) {
  const toDays = (s) => {
    const [y, m, d] = s.split('.').map(Number);
    return y * 12 * 28 + m * 28 + d;
  };
  const term = new Map(terms.map(x => {
    const [type, months] = x.split(' ');
    return [type, Number(months)];
  }));
  const now = toDays(today);
  const answer = [];
  privacies.forEach((p, i) => {
```

```
const [date, type] = p.split(' ');
const expire = toDays(date) + term.get(type) * 28;
if (expire <= now) answer.push(i + 1);
}
);
return answer;
}
```

E. Đọc code theo cấu trúc

- Map lưu tần suất / ánh xạ / trạng thái để truy cập $O(1)$ trung bình thay vì quét lại dữ liệu.
- Biến/điều kiện quan trọng nhất của bài: Sau khi đổi mọi ngày sang cùng một thang ngày 28-ngày/tháng, so sánh thời gian chỉ còn là so sánh số nguyên; bản ghi hết hạn khi $start + term * 28 \leq today$.

F. Complexity + hidden-test traps

- **Độ phức tạp:** $O(n)$
- **Bẫy dễ sai:** Off-by-one ở ngày hết hạn; parse string
- **Recall 30 giây:** Date normalization / parsing → Đổi YYYY.MM.DD -> tổng số ngày theo lịch giả 28 ngày; Map term->months; $expire = start + term * 28 - 1$

2. 과제 진행하기

Tiến hành bài tập

★ **BẮT BUỘC** theo roadmap team kia

Chủ đề	Simulation	Pattern	Timeline + stack
Priority	A	Complexity	$O(n \log n)$

A. Đề bài tiếng Việt - đọc để hiểu đề

Có nhiều bài tập, mỗi bài có tên, giờ bắt đầu và thời lượng. Khi tới giờ của một bài mới, bài đang làm phải tạm dừng ngay; phần thời gian còn lại của bài bị dừng sẽ được tiếp tục khi có khoảng trống trước bài kế tiếp. Nếu trong một khoảng trống có thể hoàn thành nhiều bài đang tạm dừng thì phải tiếp tục theo thứ tự bài bị dừng gần nhất trước. Hãy trả về thứ tự các bài được hoàn thành.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: Task bắt đầu theo thời gian, task mới ngắt task cũ
- **Pattern**: Timeline + stack
- **Vì sao**: Sort start time; dùng stack lưu [task, remaining]; xử lý gap tới task kế tiếp; tranh thủ hoàn thành task bị pause

C. Tư duy lời giải từng bước

4. Sort start time.
5. dùng stack lưu [task, remaining].
6. xử lý gap tới task kế tiếp.
7. tranh thủ hoàn thành task bị pause.

Invariant / state phải giữ

Stack luôn chứa các bài đã bắt đầu nhưng chưa xong, theo thứ tự thời điểm bị pause; phần tử trên cùng là bài phải được tiếp tục trước.

D. JavaScript solution() - block code để học/copy

```
function solution(plans) {
  const toMin = (t) => {
    const [h, m] = t.split(':').map(Number);
    return h * 60 + m;
  };
  plans = plans.map(([name, start, play]) => [name, toMin(start), Number(play)]);
  .sort((a, b) => a[1] - b[1]);
  const stack = [], answer = [];
  for (let i = 0; i < plans.length; i++) {
    const [name, start, play] = plans[i];
    const nextStart = i + 1 < plans.length ? plans[i+1][1] : Infinity;
```

```

let gap = nextStart - start;
if (play <= gap) {
  answer.push(name);
  gap -= play;
  while (gap > 0 && stack.length) {
    const [paused, remain] = stack.pop();
    if (remain <= gap) {
      answer.push(paused);
      gap -= remain;
    } else {
      stack.push([paused, remain - gap]);
      gap = 0;
    }
  }
} else {
  stack.push([name, play - gap]);
}
}
while (stack.length) answer.push(stack.pop()[0]);
return answer;
}

```

E. Đọc code theo cấu trúc

- Phần sort chuẩn hóa thứ tự dữ liệu trước khi áp dụng greedy/two pointers/binary search hoặc comparator của bài.
- Vòng while là phần điều chỉnh state cho tới khi invariant trở lại đúng; khi học, cần nói được chính xác điều kiện while có nghĩa gì.
- Biến/điều kiện quan trọng nhất của bài: Stack luôn chứa các bài đã bắt đầu nhưng chưa xong, theo thứ tự thời điểm bị pause; phần tử trên cùng là bài phải được tiếp tục trước.

F. Complexity + hidden-test traps

- **Độ phức tạp:** $O(n \log n)$
- **Bẫy dễ sai:** Quên xử lý stack sau task cuối; phút HH:MM
- **Recall 30 giây:** Timeline + stack $\rightarrow$ Sort start time; dùng stack lưu [task, remaining]; xử lý gap tới task kế tiếp; tranh thủ hoàn thành task bị pause

3. 행렬 테두리 회전하기

Xoay viền ma trận

Chủ đề	Simulation	Pattern	Matrix boundary rotation
Priority	B	Complexity	$O(q \cdot (h + w))$

A. Đề bài tiếng Việt - đọc để hiểu đề

Tạo ma trận $rows \times columns$ chứa các số từ 1 tới $rows \times columns$ theo thứ tự từng hàng. Mỗi query chọn một hình chữ nhật bằng hai góc $(x1, y1)$ và $(x2, y2)$, rồi dịch toàn bộ phần tử trên đường viền một ô theo chiều kim đồng hồ. Các query được thực hiện nối tiếp trên trạng thái hiện tại của ma trận. Với mỗi lần xoay, cần ghi lại giá trị nhỏ nhất trong số các phần tử thực sự bị di chuyển và trả về danh sách các giá trị đó.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: Xoay phần viền hình chữ nhật nhiều query
- **Pattern**: Matrix boundary rotation
- **Vì sao**: Giữ temp một góc; đi 4 cạnh đúng thứ tự; cập nhật min

C. Tư duy lời giải từng bước

8. Giữ temp một góc.
9. đi 4 cạnh đúng thứ tự.
10. cập nhật min.

Invariant / state phải giữ

Trong lúc đi 4 cạnh, biến temp luôn giữ giá trị sắp bị ghi đè tiếp theo; nhờ vậy không mất dữ liệu.

D. JavaScript solution() - block code để học/copy

```
function solution(rows, columns, queries) {
    const board = Array.from( {
        length: rows
    }, (_, r) =>
        Array.from( {
            length: columns
        }, (_, c) => r * columns + c + 1));
    const ans = [];
    for (const [x1_, y1_, x2_, y2_] of queries) {
        const x1=x1_-1, y1=y1_-1, x2=x2_-1, y2=y2_-1;
        let prev = board[x1][y1], min = prev;
        for (let c=y1+1; c<=y2; c++) [prev, board[x1][c]]=[board[x1][c], prev], min=Math.min(min, prev);
        for (let r=x1+1; r<=x2; r++) [prev, board[r][y2]]=[board[r][y2], prev], min=Math.min(min, prev);
        for (let c=y2-1; c>=y1; c--) [prev, board[x2][c]]=[board[x2][c], prev], min=Math.min(min, prev);
        for (let r=x2-1; r>=x1; r--) [prev, board[r][y1]]=[board[r][y1], prev], min=Math.min(min, prev);
        ans.push(min);
    }
}
```

```
return ans;  
}
```

E. Đọc code theo cấu trúc

- Đọc code thành ba phần: khởi tạo state → cập nhật state theo từng phần tử/lựa chọn → trả đáp án. Đừng học thuộc từng dòng rồi rạc.
- Biến/điều kiện quan trọng nhất của bài: Trong lúc đi 4 cạnh, biến temp luôn giữ giá trị sắp bị ghi đè tiếp theo; nhờ vậy không mất dữ liệu.

F. Complexity + hidden-test traps

- **Độ phức tạp:** $O(q \cdot (h + w))$
- **Bẫy dễ sai:** Ghi đè mất giá trị; lệch chỉ số 1-based/0-based
- **Recall 30 giây:** Matrix boundary rotation → Giữ temp một góc; đi 4 cạnh đúng thứ tự; cập nhật min

4. 빛의 경로 사이클

Chu kỳ đường đi của tia sáng

Chủ đề	Simulation	Pattern	State graph cycle
Priority	C	Complexity	$O(R \cdot C \cdot 4)$

A. Đề bài tiếng Việt - đọc để hiểu đề

Một lưới gồm các ký tự S, L, R quyết định hướng đi của tia sáng: S đi thẳng, L rẽ trái, R rẽ phải. Khi tia đi ra khỏi một cạnh lưới, nó xuất hiện ở cạnh đối diện tương ứng, nên không gian là dạng quấn vòng. Trạng thái thật sự của tia là cả ô hiện tại và hướng đang nhìn. Hãy tìm tất cả chu kỳ đường đi khác nhau có thể tạo ra, lấy độ dài từng chu kỳ và trả về theo thứ tự tăng dần.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: Grid + hướng đi, tìm các cycle độc lập
- **Pattern**: State graph cycle
- **Vì sao**: State=(r,c,dir), mỗi state chỉ thuộc 1 cycle; mô phỏng tới state đã thăm

C. Tư duy lời giải từng bước

11. State=(r,c,dir), mỗi state chỉ thuộc 1 cycle.
12. mô phỏng tới state đã thăm.

Invariant / state phải giữ

Mỗi state (r,c,dir) chỉ được đánh dấu đúng một lần toàn cục; đi từ một state chưa thăm chắc chắn sẽ quay lại một state trong chính cycle đó.

D. JavaScript solution() - block code để học/copy

```
function solution(grid) {
    const R=grid.length, C=grid[0].length;
    const seen=Array.from( {
        length:R
    }, ()=>Array.from( {
        length:C
    }, ()=>Array(4).fill(false)));
    const dr=[-1, 0, 1, 0], dc=[0, 1, 0, -1];
    const ans=[];
    for(let sr=0; sr<R; sr++) for(let sc=0; sc<C; sc++) for(let sd=0; sd<4; sd++) {
        if(seen[sr][sc][sd]) continue;
        let r=sr, c=sc, d=sd, len=0;
        while(!seen[r][c][d]) {
            seen[r][c][d]=true;
            len++;
            const ch=grid[r][c];
            if(ch==='L') d=(d+3)%4;
```

```

else if(ch==='R') d=(d+1)%4;
r=(r+dr[d]+R)%R;
c=(c+dc[d]+C)%C;
}
ans.push(len);
}
return ans.sort((a, b)=>a-b);
}

```

E. Đọc code theo cấu trúc

- Phần sort chuẩn hóa thứ tự dữ liệu trước khi áp dụng greedy/two pointers/binary search hoặc comparator của bài.
- Vòng while là phần điều chỉnh state cho tới khi invariant trở lại đúng; khi học, cần nói được chính xác điều kiện while có nghĩa gì.
- Biến/điều kiện quan trọng nhất của bài: Mỗi state (r,c,dir) chỉ được đánh dấu đúng một lần toàn cục; đi từ một state chưa thăm chắc chắn sẽ quay lại một state trong chính cycle đó.

F. Complexity + hidden-test traps

- **Độ phức tạp:** $O(R \cdot C \cdot 4)$
- **Bẫy dễ sai:** Wrap modulo âm; mapping L/R hướng
- **Recall 30 giây:** State graph cycle $\rightarrow$ State=(r,c,dir), mỗi state chỉ thuộc 1 cycle; mô phỏng tới state đã thăm

5. 혼자서 하는 틱택토

Tic-tac-toe chơi một mình

Chủ đề	Simulation	Pattern	State validation
Priority	C	Complexity	O(1)

A. Đề bài tiếng Việt - đọc để hiểu đề

Cho trạng thái hiện tại của bàn cờ tic-tac-toe 3x3 với O, X và ô trống. O luôn đi trước, hai bên luân phiên, và ván chơi phải dừng ngay khi một bên đã thắng. Cần quyết định xem bàn cờ đã cho có thể xuất hiện sau một chuỗi nước đi hợp lệ hay không. Phải kiểm tra cả số lượng O/X lẫn trường hợp một hay hai bên đã tạo hàng thắng.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: Kiểm tra board có thể xuất hiện hợp lệ không
- **Pattern**: State validation
- **Vì sao**: Đếm O/X; kiểm tra win rows/cols/diag; áp invariant lượt chơi

C. Tư duy lời giải từng bước

13. Đếm O/X.
14. kiểm tra win rows/cols/diag.
15. áp invariant lượt chơi.

Invariant / state phải giữ

Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: Đếm O/X; kiểm tra win rows/cols/diag; áp invariant lượt chơi

D. JavaScript solution() - block code để học/copy

```
function solution(board) {
  const lines = [
    [0, 1, 2], [3, 4, 5], [6, 7, 8], [0, 3, 6], [1, 4, 7], [2, 5, 8], [0, 4, 8], [2, 4, 6]
  ];
  const s=board.join("");
  const O=[...s].filter(x=>x==='O').length, X=[...s].filter(x=>x==='X').length;
  const win = (ch) => lines.some(line => line.every(i => s[i]===ch));
  const ow=win('O'), xw=win('X');
  if (!(O===X || O===X+1)) return 0;
  if (ow && xw) return 0;
  if (ow && O!==X+1) return 0;
  if (xw && O!==X) return 0;
  return 1;
}
```

E. Đọc code theo cấu trúc

- Đọc code thành ba phần: khởi tạo state → cập nhật state theo từng phần tử/lựa chọn → trả đáp án. Đừng học thuộc từng dòng rồi rạc.

- Biến/điều kiện quan trọng nhất của bài: Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mẫu chốt là: Đếm O/X; kiểm tra win rows/cols/diag; áp invariant lượt chơi

F. Complexity + hidden-test traps

- **Độ phức tạp:** $O(1)$
- **Bẫy dễ sai:** Có cả 2 bên win; count relation sai
- **Recall 30 giây:** State validation → Đếm O/X; kiểm tra win rows/cols/diag; áp invariant lượt chơi

6. 셔틀버스

Xe buýt đưa đón

Chủ đề	Simulation	Pattern	Schedule + greedy
Priority	B	Complexity	$O(k \log k)$

A. Đề bài tiếng Việt - đọc để hiểu đề

Xe shuttle đầu tiên tới lúc 09:00, tổng cộng có n chuyến, cách nhau t phút, mỗi chuyến chở tối đa m người. Danh sách timetable cho biết thời điểm các crew khác tới hàng đợi; ai tới không muộn hơn giờ xe tới có thể lên theo thứ tự thời gian. Con muốn tới trạm muộn nhất có thể nhưng vẫn chắc chắn lên được một chuyến. Hãy trả về thời điểm muộn nhất đó theo HH:MM.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: N chuyến, mỗi chuyến m người, cần giờ muộn nhất để lên
- **Pattern**: Schedule + greedy
- **Vì sao**: Sort crew times; simulate boarding; chuyến cuối: còn chỗ -> giờ bus, đầy -> lastCrew-1

C. Tư duy lời giải từng bước

16. Sort crew times.
17. simulate boarding.
18. chuyến cuối: còn chỗ -> giờ bus, đầy -> lastCrew-1.

Invariant / state phải giữ

Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mẫu chốt là: Sort crew times; simulate boarding; chuyến cuối: còn chỗ -> giờ bus, đầy -> lastCrew-1

D. JavaScript solution() - block code để học/copy

```
function solution(n, t, m, timetable) {
  const toMin=s=> {
    const [h, mm]=s.split(':').map(Number);
    return h*60+mm;
  }
  ;
  const fmt=x=>`${String(Math.floor(x/60)).padStart(2,'0')}:${String(x%60).padStart(2,'0')}`;
  const crew=timetable.map(toMin).sort((a, b)=>a-b);
  let idx=0, lastBoarded=-1;
  for(let i=0; i<n; i++) {
    const bus=540+i*t;
    let count=0;
    while(idx<crew.length && crew[idx]<=bus && count<m) {
      lastBoarded=crew[idx++];
      count++;
    }
    if(i===n-1) {
```

```
    if(count<m) return fmt(bus);  
    return fmt(lastBoarded-1);  
  }  
}  
}
```

E. Đọc code theo cấu trúc

- Phần sort chuẩn hóa thứ tự dữ liệu trước khi áp dụng greedy/two pointers/binary search hoặc comparator của bài.
- Vòng while là phần điều chỉnh state cho tới khi invariant trở lại đúng; khi học, cần nói được chính xác điều kiện while có nghĩa gì.
- Biến/điều kiện quan trọng nhất của bài: Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: Sort crew times; simulate boarding; chuyển cuối: còn chỗ -> giờ bus, đầy -> lastCrew-1

F. Complexity + hidden-test traps

- **Độ phức tạp:** $O(k \log k)$
- **Bẫy dễ sai:** HH:MM conversion; crew đúng giờ vẫn được lên
- **Recall 30 giây:** Schedule + greedy → Sort crew times; simulate boarding; chuyển cuối: còn chỗ -> giờ bus, đầy -> lastCrew-1

7. 문자열 압축

Nén chuỗi

Chủ đề	String & Parsing	Pattern	Chunk simulation
Priority	B	Complexity	$O(n^2)$

A. Đề bài tiếng Việt - đọc để hiểu đề

Cho một chuỗi s . Ta có thể chọn độ dài đơn vị k , chia chuỗi từ trái sang phải thành các đoạn dài k và nén các đoạn liên tiếp giống hệt nhau bằng cách ghi số lần lặp trước đoạn (nếu chỉ xuất hiện một lần thì không ghi số 1). Phần dư ngắn hơn k ở cuối vẫn giữ nguyên. Hãy thử mọi k hợp lý và trả về độ dài nhỏ nhất của chuỗi sau nén.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: Thử độ dài đơn vị nén khác nhau
- **Pattern**: Chunk simulation
- **Vì sao**: For $len=1..n/2$; scan chunk; count repeats; build length

C. Tư duy lời giải từng bước

19. For $len=1..n/2$.
20. scan chunk.
21. count repeats.
22. build length.

Invariant / state phải giữ

Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mẫu chốt là: For $len=1..n/2$; scan chunk; count repeats; build length

D. JavaScript solution() - block code để học/copy

```
function solution(s) {
  let best=s.length;
  for(let len=1; len<=Math.floor(s.length/2); len++) {
    let out="", prev=s.slice(0, len), cnt=1;
    for(let i=len; i<s.length; i+=len) {
      const cur=s.slice(i, i+len);
      if(cur===prev) cnt++;
      else {
        out+=(cnt>1?cnt:"")+prev;
        prev=cur;
        cnt=1;
      }
    }
    out+=(cnt>1?cnt:"")+prev;
    best=Math.min(best, out.length);
  }
  return best;
}
```

```
}
```

E. Đọc code theo cấu trúc

- Đọc code thành ba phần: khởi tạo state → cập nhật state theo từng phần tử/lựa chọn → trả đáp án. Đừng học thuộc từng dòng rồi rạc.
- Biến/điều kiện quan trọng nhất của bài: Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mẫu chốt là: For len=1..n/2; scan chunk; count repeats; build length

F. Complexity + hidden-test traps

- **Độ phức tạp:** $O(n^2)$
- **Bẫy dễ sai:** Chunk cuối ngắn; count=1 không ghi số
- **Recall 30 giây:** Chunk simulation → For len=1..n/2; scan chunk; count repeats; build length

8. 괄호 변환

Biến đổi dấu ngoặc

Chủ đề	String & Parsing	Pattern	Recursive parsing
Priority	C	Complexity	$O(n^2)$ worst

A. Đề bài tiếng Việt - đọc để hiểu đề

Cho một chuỗi ngoặc chỉ gồm '(' và ')' với tổng số hai loại bằng nhau, nhưng chuỗi có thể chưa đúng. Cần chuyển nó thành một chuỗi ngoặc đúng theo quy tắc đệ quy được mô tả: tách tiền tố cân bằng ngắn nhất thành u và phần còn lại v; nếu u đúng thì giữ u và xử lý v, nếu u sai thì bọc kết quả của v bằng ngoặc rồi đảo từng ngoặc bên trong u (bỏ hai đầu). Hãy trả về chuỗi cuối cùng.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: Tách balanced prefix u,v rồi transform
- **Pattern**: Recursive parsing
- **Vì sao**: Find shortest balanced u; nếu correct => u+rec(v); else "("+rec(v)+")+reverse(u inside)

C. Tư duy lời giải từng bước

23. Find shortest balanced u.
24. nếu correct => u+rec(v).
25. else "("+rec(v)+")+reverse(u inside).

Invariant / state phải giữ

Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: Find shortest balanced u; nếu correct => u+rec(v); else "("+rec(v)+")+reverse(u inside)

D. JavaScript solution() - block code để học/copy

```
function solution(p) {
  const correct = (s) => {
    let bal=0;
    for(const ch of s) {
      bal += ch==='('?1:-1;
      if(bal<0) return false;
    }
    return bal===0;
  };
  ;
  const rec = (s) => {
    if(!s) return "";
    let bal=0, i=0;
    do {
      bal += s[i]===('('?1:-1;
      i++;
    }
  }
```

```

while(bal!==0);
const u=s.slice(0, i), v=s.slice(i);
if(correct(u)) return u + rec(v);
let mid="";
for(const ch of u.slice(1, -1)) mid += ch==='('?')': '(';
return '(' + rec(v) + ')' + mid;
}
;
return rec(p);
}

```

E. Đọc code theo cấu trúc

- Vòng while là phần điều chỉnh state cho tới khi invariant trở lại đúng; khi học, cần nói được chính xác điều kiện while có nghĩa gì.
- Biến/điều kiện quan trọng nhất của bài: Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mẫu chốt là: Find shortest balanced u; nếu correct => u+rec(v); else "("+rec(v)+")"+reverse(u inside)

F. Complexity + hidden-test traps

- **Độ phức tạp:** $O(n^2)$ worst
- **Bẫy dễ sai:** Nhầm balanced vs correct
- **Recall 30 giây:** Recursive parsing → Find shortest balanced u; nếu correct => u+rec(v); else "("+rec(v)+")"+reverse(u inside)

9. 수식 최대화

Tối đa hóa biểu thức

Chủ đề	String & Parsing	Pattern	Parsing + permutation
Priority	B	Complexity	O(6n)

A. Đề bài tiếng Việt - đọc để hiểu đề

Cho một biểu thức chỉ gồm số không âm và ba toán tử +, -, *. Ta được phép tự chọn thứ tự ưu tiên giữa các loại toán tử, nhưng tất cả toán tử cùng loại có cùng ưu tiên. Với mỗi thứ tự ưu tiên có thể, tính giá trị biểu thức theo thứ tự đó và lấy trị tuyệt đối. Hãy trả về trị tuyệt đối lớn nhất có thể đạt được.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: Thay thứ tự ưu tiên toán tử để max abs
- **Pattern**: Parsing + permutation
- **Vì sao**: Parse nums/operators; thử mọi permutation operator; reduce list theo precedence

C. Tư duy lời giải từng bước

26. Parse nums/operators.
27. thử mọi permutation operator.
28. reduce list theo precedence.

Invariant / state phải giữ

Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: Parse nums/operators; thử mọi permutation operator; reduce list theo precedence

D. JavaScript solution() - block code để học/copy

```
function solution(expression) {
  const nums=expression.split(/[-+*]/).map(Number);
  const ops=expression.match(/[-+*]/g) ?? [];
  const perms=[['+', '-', '*'], ['+', '*', '-'], ['-', '+', '*'], ['-', '*', '+'], ['*', '+', '-'],
    ['*', '-', '+']];
  const calc=(a, b, op)=>op==='+'?a+b:op==='-'?a-b:a*b;
  let best=0;
  for(const order of perms) {
    let ns=[...nums], os=[...ops];
    for(const target of order) {
      const nn=[ns[0]], no=[];
      for(let i=0; i<os.length; i++) {
        if(os[i]===target) nn[nn.length-1]=calc(nn[nn.length-1], ns[i+1], target);
        else {
          no.push(os[i]);
          nn.push(ns[i+1]);
        }
      }
    }
    ns=nn;
  }
  return Math.abs(ns[0]);
}
```

```
    os=no;
  }
  best=Math.max(best, Math.abs(ns[0]));
}
return best;
}
```

E. Đọc code theo cấu trúc

- Đọc code thành ba phần: khởi tạo state → cập nhật state theo từng phần tử/lựa chọn → trả đáp án. Đừng học thuộc từng dòng rồi rạc.
- Biến/điều kiện quan trọng nhất của bài: Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: Parse nums/operators; thử mọi permutation operator; reduce list theo precedence

F. Complexity + hidden-test traps

- **Độ phức tạp:** $O(6n)$
- **Bẫy dễ sai:** Số âm sau phép tính; mutate phải copy
- **Recall 30 giây:** Parsing + permutation → Parse nums/operators; thử mọi permutation operator; reduce list theo precedence

10. 폰켓몬

Pokémon

Chủ đề	Hash	Pattern	Set cardinality
Priority	B	Complexity	O(n)

A. Đề bài tiếng Việt - đọc để hiểu đề

Có một mảng `nums` biểu diễn các Pokémon, mỗi số là một loài. Bạn buộc phải chọn đúng `nums.length/2` con và muốn số loài khác nhau trong phần đã chọn lớn nhất có thể. Hãy trả về số loài tối đa có thể có trong nhóm được chọn.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: Chọn $n/2$ con, tối đa số loại khác nhau
- **Pattern**: Set cardinality
- **Vì sao**: `answer=min(uniqueCount,n/2)`

C. Tư duy lời giải từng bước

29. `answer=min(uniqueCount,n/2)`.

30. Duy trì state/invariant bên dưới và cập nhật đáp án khi điều kiện của bài được thỏa..

Invariant / state phải giữ

Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là:
`answer=min(uniqueCount,n/2)`

D. JavaScript solution() - block code để học/copy

```
function solution(nums) {  
  return Math.min(new Set(nums).size, nums.length / 2);  
}
```

E. Đọc code theo cấu trúc

- Set dùng để kiểm tra uniqueness/visited hoặc loại trùng mà không cần vòng lặp lồng nhau.
- Biến/điều kiện quan trọng nhất của bài: Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý.
Với bài này, mấu chốt là: `answer=min(uniqueCount,n/2)`

F. Complexity + hidden-test traps

- **Độ phức tạp**: $O(n)$
- **Bẫy dễ sai**: Overthink
- **Recall 30 giây**: Set cardinality $\rightarrow$ `answer=min(uniqueCount,n/2)`

11. 의상

Trang phục

Chủ đề	Hash	Pattern	Frequency product
Priority	A	Complexity	O(n)

A. Đề bài tiếng Việt - đọc để hiểu đề

Mỗi món đồ được cho bởi [tên, loại]. Khi phối đồ, với mỗi loại có thể không chọn gì hoặc chọn đúng một món, nhưng tổng thể phải mặc ít nhất một món. Hai cách phối khác nhau nếu khác ít nhất một món. Hãy tính tổng số cách phối đồ hợp lệ.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: Mỗi loại chọn 0 hoặc 1 món, ít nhất 1 món
- **Pattern**: Frequency product
- **Vì sao**: Map type->count; product(count+1)-1

C. Tư duy lời giải từng bước

31. Map type->count.
32. product(count+1)-1.

Invariant / state phải giữ

Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: Map type->count; product(count+1)-1

D. JavaScript solution() - block code để học/copy

```
function solution(clothes) {  
  const cnt=new Map();  
  for(const [, type] of clothes) cnt.set(type, (cnt.get(type)??0)+1);  
  let ans=1;  
  for(const v of cnt.values()) ans*=v+1;  
  return ans-1;  
}
```

E. Đọc code theo cấu trúc

- Map lưu tần suất / ánh xạ / trạng thái để truy cập O(1) trung bình thay vì quét lại dữ liệu.
- Biến/điều kiện quan trọng nhất của bài: Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: Map type->count; product(count+1)-1

F. Complexity + hidden-test traps

- **Độ phức tạp**: O(n)
- **Bẫy dễ sai**: Quên lựa chọn không mặc ở từng loại
- **Recall 30 giây**: Frequency product → Map type->count; product(count+1)-1

12. 베스트앨범

Album hay nhất

Chủ đề	Hash	Pattern	Group + multi-key sort
Priority	B	Complexity	$O(n \log n)$

A. Đề bài tiếng Việt - đọc để hiểu đề

Mỗi bài hát có thể loại genres[i] và lượt nghe plays[i]. Một album hay nhất sắp các thể loại theo tổng lượt nghe giảm dần; trong mỗi thể loại chỉ chọn tối đa hai bài có lượt nghe cao nhất, và nếu bằng lượt nghe thì ưu tiên bài có chỉ số nhỏ hơn. Hãy trả về danh sách chỉ số bài hát theo đúng thứ tự album.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: Theo genre tổng plays, mỗi genre lấy top 2 bài
- **Pattern**: Group + multi-key sort
- **Vì sao**: Map genre total + song list; sort genre desc total; song desc plays, asc index

C. Tư duy lời giải từng bước

33. Map genre total + song list.
34. sort genre desc total.
35. song desc plays, asc index.

Invariant / state phải giữ

Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: Map genre total + song list; sort genre desc total; song desc plays, asc index

D. JavaScript solution() - block code để học/copy

```
function solution(genres, plays) {
  const total=new Map(), songs=new Map();
  for(let i=0; i<genres.length; i++) {
    const g=genres[i];
    total.set(g, (total.get(g)?0)+plays[i]);
    if(!songs.has(g)) songs.set(g, []);
    songs.get(g).push([plays[i], i]);
  }
  const order=[...total.keys()].sort((a, b)=>total.get(b)-total.get(a));
  const ans=[];
  for(const g of order) {
    songs.get(g).sort((a, b)=>b[0]-a[0] || a[1]-b[1]);
    ans.push(...songs.get(g).slice(0, 2).map(x=>x[1]));
  }
  return ans;
}
```

E. Đọc code theo cấu trúc

- Phần sort chuẩn hóa thứ tự dữ liệu trước khi áp dụng greedy/two pointers/binary search hoặc comparator của bài.
- Map lưu tần suất / ánh xạ / trạng thái để truy cập $O(1)$ trung bình thay vì quét lại dữ liệu.
- Biến/điều kiện quan trọng nhất của bài: Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý.
Với bài này, mấu chốt là: Map genre total + song list; sort genre desc total; song desc plays, asc index

F. Complexity + hidden-test traps

- **Độ phức tạp:** $O(n \log n)$
- **Bẫy dễ sai:** Comparator tie-break index
- **Recall 30 giây:** Group + multi-key sort → Map genre total + song list; sort genre desc total; song desc plays, asc index

13. K 번째수

Số thứ K

Chủ đề	Sorting	Pattern	Slice + sort
Priority	B	Complexity	$O(q \cdot m \log m)$

A. Đề bài tiếng Việt - đọc để hiểu đề

Với mỗi command $[i, j, k]$, lấy đoạn array từ vị trí i tới j (đánh số từ 1), sắp tăng dần rồi lấy phần tử thứ k trong đoạn đã sắp. Các command độc lập trên mảng gốc. Hãy trả về kết quả của tất cả command.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: Query $[i, j, k]$ trên array
- **Pattern**: Slice + sort
- **Vì sao**: $\text{slice}(i-1, j).sort((a, b) \Rightarrow a - b)[k-1]$

C. Tư duy lời giải từng bước

36. $\text{slice}(i-1, j).sort((a, b) \Rightarrow a - b)[k-1]$.

37. Duy trì state/invariant bên dưới và cập nhật đáp án khi điều kiện của bài được thỏa..

Invariant / state phải giữ

Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là:

$\text{slice}(i-1, j).sort((a, b) \Rightarrow a - b)[k-1]$

D. JavaScript solution() - block code để học/copy

```
function solution(array, commands) {  
  return commands.map(([i, j, k]) => array.slice(i-1, j).sort((a, b) => a-b)[k-1]);  
}
```

E. Đọc code theo cấu trúc

- Phần sort chuẩn hóa thứ tự dữ liệu trước khi áp dụng greedy/two pointers/binary search hoặc comparator của bài.
- Biến/điều kiện quan trọng nhất của bài: Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: $\text{slice}(i-1, j).sort((a, b) \Rightarrow a - b)[k-1]$

F. Complexity + hidden-test traps

- **Độ phức tạp**: $O(q \cdot m \log m)$
- **Bẫy dễ sai**: JS sort mặc định lexicographic
- **Recall 30 giây**: Slice + sort $\rightarrow \text{slice}(i-1, j).sort((a, b) \Rightarrow a - b)[k-1]$

14. 가장 큰 수

Số lớn nhất

Chủ đề	Sorting	Pattern	Custom comparator
Priority	B	Complexity	$O(n \log n * \text{digits})$

A. Đề bài tiếng Việt - đọc để hiểu đề

Cho một dãy số nguyên không âm. Hãy sắp xếp chúng theo một thứ tự sao cho khi nối dạng chuỗi lại ta nhận được số lớn nhất có thể. So sánh hai số a, b phải dựa vào hai cách ghép $a+b$ và $b+a$, không dựa trên giá trị riêng lẻ. Nếu kết quả toàn số 0 thì chỉ trả về '0'.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: Ghép các số để chuỗi lớn nhất
- **Pattern**: Custom comparator
- **Vì sao**: `map(String).sort((a,b)=>(b+a).localeCompare(a+b)); join`; xử lý all zero

C. Tư duy lời giải từng bước

38. `map(String).sort((a,b)=>(b+a).localeCompare(a+b)).`

39. `join`.

40. xử lý all zero.

Invariant / state phải giữ

Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mẫu chốt là:
`map(String).sort((a,b)=>(b+a).localeCompare(a+b)); join`; xử lý all zero

D. JavaScript solution() - block code để học/copy

```
function solution(numbers) {
  const s=numbers.map(String).sort((a, b)=>(b+a).localeCompare(a+b)).join("");
  return s[0]=== '0' ? '0' : s;
}
```

E. Đọc code theo cấu trúc

- Phần sort chuẩn hóa thứ tự dữ liệu trước khi áp dụng greedy/two pointers/binary search hoặc comparator của bài.
- Biến/điều kiện quan trọng nhất của bài: Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mẫu chốt là: `map(String).sort((a,b)=>(b+a).localeCompare(a+b)); join`; xử lý all zero

F. Complexity + hidden-test traps

- **Độ phức tạp**: $O(n \log n * \text{digits})$
- **Bẫy dễ sai**: Comparator đảo chiều; "000" -> "0"
- **Recall 30 giây**: Custom comparator → `map(String).sort((a,b)=>(b+a).localeCompare(a+b)); join`; xử lý all zero

15. H-Index

H-Index

Chủ đề	Sorting	Pattern	Sort + threshold
Priority	B	Complexity	$O(n \log n)$

A. Đề bài tiếng Việt - đọc để hiểu đề

$\text{citations}[i]$ là số lần bài báo thứ i được trích dẫn. H-index là số h lớn nhất sao cho có ít nhất h bài báo được trích dẫn ít nhất h lần. Hãy tính H-index từ mảng citations . Sau khi sắp xếp giảm dần, vị trí i chính là số lượng bài đang xét nên có thể kiểm tra trực tiếp điều kiện $\text{citations}[i] \geq i+1$.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: Tìm h lớn nhất sao có ít nhất h bài $\geq h$
- **Pattern**: Sort + threshold
- **Vì sao**: Sort desc; tăng h khi $\text{citations}[i] \geq i+1$

C. Tư duy lời giải từng bước

41. Sort desc.

42. tăng h khi $\text{citations}[i] \geq i+1$.

Invariant / state phải giữ

Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: Sort desc; tăng h khi $\text{citations}[i] \geq i+1$

D. JavaScript solution() - block code để học/copy

```
function solution(citations) {
  citations.sort((a, b) => b-a);
  let h=0;
  while(h<citations.length && citations[h]>=h+1) h++;
  return h;
}
```

E. Đọc code theo cấu trúc

- Phần sort chuẩn hóa thứ tự dữ liệu trước khi áp dụng greedy/two pointers/binary search hoặc comparator của bài.
- Vòng while là phần điều chỉnh state cho tới khi invariant trở lại đúng; khi học, cần nói được chính xác điều kiện while có nghĩa gì.
- Biến/điều kiện quan trọng nhất của bài: Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: Sort desc; tăng h khi $\text{citations}[i] \geq i+1$

F. Complexity + hidden-test traps

- **Độ phức tạp**: $O(n \log n)$
- **Bẫy dễ sai**: Nhầm điều kiện và output citation

- **Recall 30 giây:** Sort + threshold $\rightarrow$ Sort desc; tăng h khi citations[i] $\geq$ i+1

16. 크레인 인형뽑기 게임

Máy gấp thú bông

Chủ đề	Stack/Queue/Heap	Pattern	Simulation + stack
Priority	B	Complexity	$O(m \cdot R)$

A. Đề bài tiếng Việt - đọc để hiểu đề

Một bàn máy gấp được biểu diễn bằng ma trận; mỗi cột chứa các con thú xếp từ trên xuống và 0 là ô trống. Mỗi move chọn một cột, máy gấp con thú trên cùng còn lại trong cột đó và thả vào giỏ. Nếu hai con cùng loại nằm kề nhau ở đỉnh giỏ thì chúng biến mất và tính là 2 con bị loại. Hãy trả về tổng số thú biến mất sau toàn bộ moves.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: Lấy phần tử đầu tiên khác 0 mỗi cột, loại cặp giống nhau
- **Pattern**: Simulation + stack
- **Vì sao**: For move: scan rows; push/pop stack; count +=2 khi trùng top

C. Tư duy lời giải từng bước

43. For move: scan rows.
44. push/pop stack.
45. count +=2 khi trùng top.

Invariant / state phải giữ

Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: For move: scan rows; push/pop stack; count +=2 khi trùng top

D. JavaScript solution() - block code để học/copy

```
function solution(board, moves) {
  const stack=[];
  let removed=0;
  for(const m of moves) {
    const c=m-1;
    for(let r=0; r<board.length; r++) {
      if(board[r][c]===0) continue;
      const x=board[r][c];
      board[r][c]=0;
      if(stack.length && stack.at(-1)===x) {
        stack.pop();
        removed+=2;
      } else stack.push(x);
      break;
    }
  }
  return removed;
}
```

E. Đọc code theo cấu trúc

- Đọc code thành ba phần: khởi tạo state → cập nhật state theo từng phần tử/lựa chọn → trả đáp án. Đừng học thuộc từng dòng rồi rạc.
- Biến/điều kiện quan trọng nhất của bài: Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mẫu chốt là: For move: scan rows; push/pop stack; count +=2 khi trùng top

F. Complexity + hidden-test traps

- **Độ phức tạp:** $O(m \cdot R)$
- **Bẫy dễ sai:** Index cột 1-based; bỏ qua 0
- **Recall 30 giây:** Simulation + stack → For move: scan rows; push/pop stack; count +=2 khi trùng top

17. 올바른 괄호

Dấu ngoặc hợp lệ

Chủ đề	Stack/Queue/Heap	Pattern	Stack / counter
Priority	B	Complexity	O(n)

A. Đề bài tiếng Việt - đọc để hiểu đề

Cho chuỗi chỉ gồm '(' và ')'. Một chuỗi đúng là chuỗi có thể ghép cặp ngoặc mở-đóng đúng thứ tự, không bao giờ có ngoặc đóng khi chưa có ngoặc mở tương ứng và cuối cùng không còn ngoặc mở dư. Hãy trả về true/false xem chuỗi có hợp lệ hay không.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: Chuỗi ngoặc có đóng mở đúng thứ tự
- **Pattern**: Stack / counter
- **Vì sao**: Counter +1/-1; nếu âm return false; cuối phải 0

C. Tư duy lời giải từng bước

46. Counter +1/-1.
47. nếu âm return false.
48. cuối phải 0.

Invariant / state phải giữ

balance là số ngoặc mở chưa được ghép; balance không bao giờ được âm và phải bằng 0 ở cuối.

D. JavaScript solution() - block code để học/copy

```
function solution(s) {
  let bal=0;
  for(const ch of s) {
    bal += ch==='('?1:-1;
    if(bal<0) return false;
  }
  return bal===0;
}
```

E. Đọc code theo cấu trúc

- Đọc code thành ba phần: khởi tạo state → cập nhật state theo từng phần tử/lựa chọn → trả đáp án. Đừng học thuộc từng dòng rồi rạc.
- Biến/điều kiện quan trọng nhất của bài: balance là số ngoặc mở chưa được ghép; balance không bao giờ được âm và phải bằng 0 ở cuối.

F. Complexity + hidden-test traps

- **Độ phức tạp**: O(n)
- **Bẫy dễ sai**: Chỉ check tổng cuối là chưa đủ

- **Recall 30 giây:** Stack / counter $\rightarrow$ Counter $+1/-1$; nếu âm return false; cuối phải 0

18. 짝지어 제거하기

Loại bỏ từng cặp

Chủ đề	Stack/Queue/Heap	Pattern	Adjacent cancellation stack
Priority	B	Complexity	O(n)

A. Đề bài tiếng Việt - đọc để hiểu đề

Cho chuỗi chữ cái. Có thể lặp lại thao tác xóa hai ký tự giống nhau đứng cạnh nhau; sau khi xóa, hai phần còn lại nối lại và có thể tạo cặp mới. Hãy xác định có thể xóa hết chuỗi hay không. Kết quả là 1 nếu xóa hết được, ngược lại 0.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: Xóa cặp ký tự giống nhau liên tiếp tới ổn định
- **Pattern**: Adjacent cancellation stack
- **Vì sao**: Nếu `top==ch` -> pop, else push

C. Tư duy lời giải từng bước

49. Nếu `top==ch` -> pop, else push.

50. Duy trì state/invariant bên dưới và cập nhật đáp án khi điều kiện của bài được thỏa..

Invariant / state phải giữ

Stack luôn là dạng chuỗi còn lại sau khi đã xóa hết mọi cặp giống nhau có thể xóa trong prefix đã đọc.

D. JavaScript solution() - block code để học/copy

```
function solution(s) {
  const st=[];
  for(const ch of s) {
    if(st.length && st.at(-1)===ch) st.pop();
    else st.push(ch);
  }
  return st.length===0 ? 1 : 0;
}
```

E. Đọc code theo cấu trúc

- Đọc code thành ba phần: khởi tạo state → cập nhật state theo từng phần tử/lựa chọn → trả đáp án. Đừng học thuộc từng dòng rồi rạc.
- Biến/điều kiện quan trọng nhất của bài: Stack luôn là dạng chuỗi còn lại sau khi đã xóa hết mọi cặp giống nhau có thể xóa trong prefix đã đọc.

F. Complexity + hidden-test traps

- **Độ phức tạp**: O(n)
- **Bẫy dễ sai**: Dùng replace lặp gây O(n^2)

- **Recall 30 giây:** Adjacent cancellation stack → Nếu $top == ch \rightarrow pop$, else push

19. 다리를 지나는 트럭

Xe tải qua cầu

Chủ đề	Stack/Queue/Heap	Pattern	Queue simulation
Priority	B	Complexity	O(n)

A. Đề bài tiếng Việt - đọc để hiểu đề

Một cây cầu có độ dài `bridge_length`, chịu được tổng trọng lượng tối đa `weight`. Mỗi giây, xe đang trên cầu tiến thêm một ô; xe mới chỉ được vào nếu tổng trọng lượng sau khi thêm không vượt giới hạn. Mỗi truck phải ở trên cầu đúng `bridge_length` giây mới ra khỏi cầu. Hãy tính thời gian để toàn bộ xe đi qua.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: Cầu length, weight limit, truck theo thứ tự
- **Pattern**: Queue simulation
- **Vì sao**: Queue [weight, exitTime] hoặc queue fixed length; mỗi tick pop xe rồi, thử push xe mới

C. Tư duy lời giải từng bước

51. Queue [weight, exitTime] hoặc queue fixed length.
52. mỗi tick pop xe rồi, thử push xe mới.

Invariant / state phải giữ

Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: Queue [weight, exitTime] hoặc queue fixed length; mỗi tick pop xe rồi, thử push xe mới

D. JavaScript solution() - block code để học/copy

```
function solution(bridge_length, weight, truck_weights) {
  const q=[];
  let head=0, time=0, sum=0, idx=0;
  while(idx<truck_weights.length || head<q.length) {
    time++;
    while(head<q.length && q[head][1]===time) {
      sum-=q[head][0];
      head++;
    }
    if(idx<truck_weights.length && sum+truck_weights[idx]<=weight) {
      const w=truck_weights[idx++];
      sum+=w;
      q.push([w, time+bridge_length]);
    }
  }
  return time;
}
```

E. Đọc code theo cấu trúc

- Vòng while là phần điều chỉnh state cho tới khi invariant trở lại đúng; khi học, cần nói được chính xác điều kiện while có nghĩa gì.
- Biến/điều kiện quan trọng nhất của bài: Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mẫu chốt là: Queue [weight, exitTime] hoặc queue fixed length; mỗi tick pop xe rồi, thử push xe mới

F. Complexity + hidden-test traps

- **Độ phức tạp:** $O(n)$
- **Bẫy dễ sai:** shift() JS có thể chậm; thời điểm vào/ra
- **Recall 30 giây:** Queue simulation → Queue [weight, exitTime] hoặc queue fixed length; mỗi tick pop xe rồi, thử push xe mới

20. 기능개발

Phát triển chức năng

Chủ đề	Stack/Queue/Heap	Pattern	Queue/grouping
Priority	A	Complexity	O(n)

A. Đề bài tiếng Việt - đọc để hiểu đề

Mỗi chức năng có tiến độ hiện tại `progresses[i]` và tốc độ phát triển mỗi ngày `speeds[i]`. Một chức năng chỉ được deploy khi hoàn thành, nhưng thứ tự deploy phải giữ nguyên: chức năng sau dù hoàn thành sớm vẫn phải chờ chức năng trước. Mỗi lần deploy có thể phát hành liên tiếp nhiều chức năng đã hoàn thành. Hãy trả về số lượng chức năng trong từng đợt deploy.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: Tính ngày hoàn thành, deploy theo thứ tự
- **Pattern**: Queue/grouping
- **Vì sao**: `days=ceil((100-progress)/speed)`; group liên tiếp nếu `day<=currentReleaseDay`

C. Tư duy lời giải từng bước

53. `days=ceil((100-progress)/speed)`.

54. group liên tiếp nếu `day<=currentReleaseDay`.

Invariant / state phải giữ

`releaseDay` là ngày hoàn thành của chức năng đầu tiên trong batch hiện tại; các chức năng sau có `day <= releaseDay` thuộc cùng batch.

D. JavaScript solution() - block code để học/copy

```
function solution(progresses, speeds) {
  const days=progresses.map((p, i)=>Math.ceil((100-p)/speeds[i]));
  const ans=[];
  let release=days[0], count=1;
  for(let i=1; i<days.length; i++) {
    if(days[i]<=release) count++;
    else {
      ans.push(count);
      release=days[i];
      count=1;
    }
  }
  ans.push(count);
  return ans;
}
```

E. Đọc code theo cấu trúc

- Đọc code thành ba phần: khởi tạo state → cập nhật state theo từng phần tử/lựa chọn → trả đáp án. Đừng học thuộc từng dòng rồi rạc.
- Biến/điều kiện quan trọng nhất của bài: `releaseDay` là ngày hoàn thành của chức năng đầu tiên trong batch hiện tại; các chức năng sau có `day <= releaseDay` thuộc cùng batch.

F. Complexity + hidden-test traps

- **Độ phức tạp:** $O(n)$
- **Bẫy dễ sai:** So sánh với `max release day`, không phải phần tử trước
- **Recall 30 giây:** Queue/grouping → `days=ceil((100-progress)/speed)`; group liên tiếp nếu `day <= currentReleaseDay`

21. 프로세스

Tiến trình

Chủ đề	Stack/Queue/Heap	Pattern	Queue + priority
Priority	B	Complexity	$O(n^2)$ simple

A. Đề bài tiếng Việt - đọc để hiểu đề

Có một hàng đợi các process với độ ưu tiên khác nhau. Luôn lấy process đầu hàng đợi ra; nếu phía sau còn process có priority cao hơn thì đưa process vừa lấy xuống cuối, nếu không thì thực thi nó. Cho location ban đầu của process mục tiêu, hãy trả về nó được thực thi ở lượt thứ mấy.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: Process có priority cao hơn chờ thì current quay lại queue
- **Pattern**: Queue + priority
- **Vì sao**: Queue indices; nếu còn priority lớn hơn thì push lại, else execute

C. Tư duy lời giải từng bước

55. Queue indices.

56. nếu còn priority lớn hơn thì push lại, else execute.

Invariant / state phải giữ

Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: Queue indices; nếu còn priority lớn hơn thì push lại, else execute

D. JavaScript solution() - block code để học/copy

```
function solution(priorities, location) {
  const q=priorities.map((p, i)=>[p, i]);
  let head=0, order=0;
  while(head<q.length) {
    const [p, i]=q[head++];
    let higher=false;
    for(let j=head; j<q.length; j++) if(q[j][0]>p) {
      higher=true;
      break;
    }
    if(higher) q.push([p, i]);
    else {
      order++;
      if(i===location) return order;
    }
  }
}
```

E. Đọc code theo cấu trúc

- Vòng while là phần điều chỉnh state cho tới khi invariant trở lại đúng; khi học, cần nói được chính xác điều kiện while có nghĩa gì.
- Biến/điều kiện quan trọng nhất của bài: Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: Queue indices; nếu còn priority lớn hơn thì push lại, else execute

F. Complexity + hidden-test traps

- **Độ phức tạp:** $O(n^2)$ simple
- **Bẫy dễ sai:** Dùng shift nhiều; track target index
- **Recall 30 giây:** Queue + priority → Queue indices; nếu còn priority lớn hơn thì push lại, else execute

22. 디펜스 게임

Game phòng thủ

Chủ đề	Stack/Queue/Heap	Pattern	Min-heap / greedy undo
Priority	B	Complexity	$O(n \log k)$

A. Đề bài tiếng Việt - đọc để hiểu đề

Có k lính và n lượt địch enemies[i]. Bình thường phải dùng số lính bằng số địch ở lượt đó; có n lần dùng 'quyền bất tử' để bỏ qua toàn bộ chi phí của một lượt. Mục tiêu là sống qua nhiều lượt nhất. Hãy tìm số round tối đa có thể vượt qua bằng cách dùng quyền bất tử một cách tối ưu.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: Dùng k lần vô địch để bỏ qua wave lớn nhất đã gặp
- **Pattern**: Min-heap / greedy undo
- **Vì sao**: Trừ soldiers mỗi wave, push enemy to max-heap; hoặc min-heap giữ k wave lớn nhất được skip

C. Tư duy lời giải từng bước

57. Trừ soldiers mỗi wave, push enemy to max-heap.

58. hoặc min-heap giữ k wave lớn nhất được skip.

Invariant / state phải giữ

Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: Trừ soldiers mỗi wave, push enemy to max-heap; hoặc min-heap giữ k wave lớn nhất được skip

D. JavaScript solution() - block code để học/copy

```
function solution(n, k, enemy) {
  class MinHeap {
    constructor() {
      this.a=[];
    }
    push(x) {
      let a=this.a;
      a.push(x);
      let i=a.length-1;
      while(i) {
        let p=(i-1)>>1;
        if(a[p]<=x)break;
        a[i]=a[p];
        i=p;
      }
      a[i]=x;
    }
    pop() {
      const a=this.a, root=a[0], last=a.pop();
      if(a.length) {
```

```

let i=0;
a[0]=last;
while(1) {
  let l=i*2+1, r=l+1, m=i;
  if(l<a.length&&a[l]<a[m])m=l;
  if(r<a.length&&a[r]<a[m])m=r;
  if(m===i)break;
  [a[i], a[m]]=a[m], a[i];
  i=m;
}
}
return root;
}
get size() {
  return this.a.length;
}
}
const h=new MinHeap();
let spent=0;
for(let i=0; i<enemy.length; i++) {
  h.push(enemy[i]);
  if(h.size>k) spent+=h.pop();
  if(spent>n) return i;
}
return enemy.length;
}

```

E. Đọc code theo cấu trúc

- Vòng while là phần điều chỉnh state cho tới khi invariant trở lại đúng; khi học, cần nói được chính xác điều kiện while có nghĩa gì.
- Biến/điều kiện quan trọng nhất của bài: Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: Trừ soldiers mỗi wave, push enemy to max-heap; hoặc min-heap giữ k wave lớn nhất được skip

F. Complexity + hidden-test traps

- **Độ phức tạp:** $O(n \log k)$
- **Bẫy dễ sai:** Heap direction; khi soldiers < 0 phải hoàn lại đúng wave lớn nhất
- **Recall 30 giây:** Min-heap / greedy undo → Trừ soldiers mỗi wave, push enemy to max-heap; hoặc min-heap giữ k wave lớn nhất được skip

23. 이중우선순위큐

Hàng đợi ưu tiên kép

Chủ đề	Stack/Queue/Heap	Pattern	Dual heap / multiset
Priority	C	Complexity	$O(n \log n)$

A. Đề bài tiếng Việt - đọc để hiểu đề

Thiết kế cấu trúc hỗ trợ I x (chèn x), D 1 (xóa giá trị lớn nhất), D -1 (xóa giá trị nhỏ nhất). Nếu hàng đợi rỗng thì lệnh xóa bị bỏ qua. Sau toàn bộ operations, trả về [max,min] hoặc [0,0] nếu rỗng. Cần xử lý đồng bộ hai phía min/max mà không để phần tử đã xóa tồn tại sai trong heap còn lại.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: Insert, delete min/max
- **Pattern**: Dual heap / multiset
- **Vì sao**: Hai heap + lazy deletion / Map counts; hoặc JS sorted array nếu constraints chịu được

C. Tư duy lời giải từng bước

59. Hai heap + lazy deletion / Map counts.

60. hoặc JS sorted array nếu constraints chịu được.

Invariant / state phải giữ

Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: Hai heap + lazy deletion / Map counts; hoặc JS sorted array nếu constraints chịu được

D. JavaScript solution() - block code để học/copy

```
function solution(operations) {
  class Heap {
    constructor(cmp) {
      this.a=[];
      this.cmp=cmp;
    }
    push(x) {
      const a=this.a;
      a.push(x);
      let i=a.length-1;
      while(i) {
        const p=(i-1)>>1;
        if(!this.cmp(a[i], a[p])) break;
        [a[i], a[p]]=[a[p], a[i]];
        i=p;
      }
    }
    pop() {
      const a=this.a;
      if(!a.length) return;
    }
  }
```

```

const root=a[0], last=a.pop();
if(a.length) {
  a[0]=last;
  let i=0;
  while(1) {
    let m=i, l=i*2+1, r=l+1;
    if(l<a.length&&this.cmp(a[l], a[m]))m=l;
    if(r<a.length&&this.cmp(a[r], a[m]))m=r;
    if(m===i)break;
    [a[i], a[m]]=[a[m], a[i]];
    i=m;
  }
}
return root;
}
peek() {
  return this.a[0];
}
}
const minH=new Heap((a, b)=>a<b), maxH=new Heap((a, b)=>a>b), count=new Map();
let size=0;
const clean=(h)=> {
  while(h.a.length && (count.get(h.peek())??0)===0) h.pop();
}
;
for(const op of operations) {
  const [cmd, s]=op.split(' '), x=Number(s);
  if(cmd==='I') {
    minH.push(x);
    maxH.push(x);
    count.set(x, (count.get(x)?0)+1);
    size++;
  } else if(size) {
    const h=x===1?maxH:minH;
    clean(h);
    const v=h.pop();
    count.set(v, count.get(v)-1);
    size--;
    if(size===0) count.clear();
  }
}
if(!size) return [0, 0];
clean(minH);
clean(maxH);
return [maxH.peek(), minH.peek()];
}

```

E. Đọc code theo cấu trúc

- Map lưu tần suất / ánh xạ / trạng thái để truy cập $O(1)$ trung bình thay vì quét lại dữ liệu.
- Vòng while là phần điều chỉnh state cho tới khi invariant trở lại đúng; khi học, cần nói được chính xác điều kiện while có nghĩa gì.
- Biến/điều kiện quan trọng nhất của bài: Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: Hai heap + lazy deletion / Map counts; hoặc JS sorted array nếu constraints chịu được

F. Complexity + hidden-test traps

- **Độ phức tạp:** $O(n \log n)$
- **Bẫy dễ sai:** Heap stale elements; đồng bộ counts
- **Recall 30 giây:** Dual heap / multiset → Hai heap + lazy deletion / Map counts; hoặc JS sorted array nếu constraints chịu được

24. 모의고사

Thi thử

Chủ đề	Brute Force & Math	Pattern	Pattern cycling
Priority	B	Complexity	O(n)

A. Đề bài tiếng Việt - đọc để hiểu đề

Ba người làm bài trắc nghiệm theo ba chu kỳ đáp án cố định khác nhau. Cho answers là đáp án đúng của từng câu, hãy tính số câu đúng của mỗi người bằng cách so với chu kỳ tương ứng. Trả về số thứ tự người có điểm cao nhất; nếu hòa thì trả về tất cả theo thứ tự tăng dần.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: So đáp án với 3 pattern lặp
- **Pattern**: Pattern cycling
- **Vì sao**: score[i%pattern.length], lấy max, return indices

C. Tư duy lời giải từng bước

61. score[i%pattern.length], lấy max, return indices.

62. Duy trì state/invariant bên dưới và cập nhật đáp án khi điều kiện của bài được thỏa..

Invariant / state phải giữ

Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: score[i%pattern.length], lấy max, return indices

D. JavaScript solution() - block code để học/copy

```
function solution(answers) {
  const patterns=[[1, 2, 3, 4, 5], [2, 1, 2, 3, 2, 4, 2, 5], [3, 3, 1, 1, 2, 2, 4, 4, 5, 5]];
  const score=[0, 0, 0];
  answers.forEach((a, i)=>patterns.forEach((p, j)=> {
    if(a===p[i%p.length]) score[j]++;
  }
));
  const mx=Math.max(...score);
  return score.map((v, i)=>v===mx?i+1:null).filter(Boolean);
}
```

E. Đọc code theo cấu trúc

- Đọc code thành ba phần: khởi tạo state → cập nhật state theo từng phần tử/lựa chọn → trả đáp án. Đừng học thuộc từng dòng rồi rạc.
- Biến/điều kiện quan trọng nhất của bài: Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: score[i%pattern.length], lấy max, return indices

F. Complexity + hidden-test traps

- **Độ phức tạp:** $O(n)$
- **Bẫy dễ sai:** Modulo index
- **Recall 30 giây:** Pattern cycling → `score[i%pattern.length]`, lấy max, return indices

25. 카펫

Tấm thảm

Chủ đề	Brute Force & Math	Pattern	Factor search
Priority	B	Complexity	$O(\sqrt{n})$

A. Đề bài tiếng Việt - đọc để hiểu đề

Một tấm thảm có brown ô viền màu nâu bao quanh yellow ô màu vàng ở bên trong. Tấm thảm là hình chữ nhật, kích thước nguyên và chiều rộng không nhỏ hơn chiều cao. Hãy tìm [width,height] sao cho tổng ô đúng bằng brown+yellow và phần bên trong có đúng yellow ô.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: brown/yellow suy ra width,height
- **Pattern**: Factor search
- **Vì sao**: $\text{total} = b + y$; thử h từ $1.. \sqrt{\text{total}}$, $w = \text{total}/h$; $(w-2)(h-2) == y$

C. Tư duy lời giải từng bước

63. $\text{total} = b + y$.

64. thử h từ $1.. \sqrt{\text{total}}$, $w = \text{total}/h$.

65. $(w-2)(h-2) == y$.

Invariant / state phải giữ

Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: $\text{total} = b + y$; thử h từ $1.. \sqrt{\text{total}}$, $w = \text{total}/h$; $(w-2)(h-2) == y$

D. JavaScript solution() - block code để học/copy

```
function solution(brown, yellow) {
  const total = brown + yellow;
  for (let h = 3; h <= Math.sqrt(total); h++) if (total % h === 0) {
    const w = total / h;
    if ((w - 2) * (h - 2) === yellow) return [w, h];
  }
}
```

E. Đọc code theo cấu trúc

- Đọc code thành ba phần: khởi tạo state → cập nhật state theo từng phần tử/lựa chọn → trả đáp án. Đừng học thuộc từng dòng rồi rạc.
- Biến/điều kiện quan trọng nhất của bài: Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: $\text{total} = b + y$; thử h từ $1.. \sqrt{\text{total}}$, $w = \text{total}/h$; $(w-2)(h-2) == y$

F. Complexity + hidden-test traps

- **Độ phức tạp**: $O(\sqrt{n})$
- **Bẫy dễ sai**: $w \geq h$; border formula

- **Recall 30 giây:** Factor search $\rightarrow$ total=b+y; thử h từ 1...sqrt(total), w=total/h; (w-2)(h-2)=y

26. 점 찍기

Chăm điểm

Chủ đề	Brute Force & Math	Pattern	Math counting
Priority	B	Complexity	$O(d/k)$

A. Đề bài tiếng Việt - đọc để hiểu đề

Trên hệ trục tọa độ góc phần tư thứ nhất, xét các điểm có tọa độ là bội của k và nằm trong hoặc trên đường tròn bán kính d tâm tại gốc. Hãy đếm số cặp (x,y) thỏa $x \geq 0, y \geq 0, x$ và y chia hết cho $k, x^2 + y^2 \leq d^2$.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: Đếm lattice points theo bước k trong quarter circle
- **Pattern**: Math counting
- **Vì sao**: For $x=0;k;\dots \leq d: yMax=floor(sqrt(d^2-x^2)/k)*k; add yMax/k+1$

C. Tư duy lời giải từng bước

66. For $x=0$.

67. k .

68. $\dots \leq d: yMax=floor(sqrt(d^2-x^2)/k)*k$.

69. add $yMax/k+1$.

Invariant / state phải giữ

Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mẫu chốt là: For $x=0;k;\dots \leq d: yMax=floor(sqrt(d^2-x^2)/k)*k; add yMax/k+1$

D. JavaScript solution() - block code để học/copy

```
function solution(k, d) {
  let ans=0;
  for(let x=0; x<=d; x+=k) {
    const y=Math.floor(Math.sqrt(d*d-x*x)/k);
    ans += y+1;
  }
  return ans;
}
```

E. Đọc code theo cấu trúc

- Đọc code thành ba phần: khởi tạo state → cập nhật state theo từng phần tử/lựa chọn → trả đáp án. Đừng học thuộc từng dòng rồi rạc.
- Biến/điều kiện quan trọng nhất của bài: Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mẫu chốt là: For $x=0;k;\dots \leq d: yMax=floor(sqrt(d^2-x^2)/k)*k; add yMax/k+1$

F. Complexity + hidden-test traps

- **Độ phức tạp**: $O(d/k)$

- **Bẫy dễ sai:** Overflow d^2 ; floating precision vừa đủ
- **Recall 30 giây:** Math counting → For $x=0;k;\dots \leq d$: $yMax = \text{floor}(\sqrt{d^2 - x^2}/k) * k$; add $yMax/k + 1$

27. 광물 캐기

Khai thác khoáng sản

★ **BẮT BUỘC** theo roadmap team kia

Chủ đề	Greedy	Pattern	Greedy grouping
Priority	A	Complexity	$O(n \log n)$

A. Đề bài tiếng Việt - đọc để hiểu đề

Có ba loại cuốc với mức mệt khác nhau khi đào diamond, iron, stone. Mỗi cuốc đào được tối đa 5 khoáng vật rồi hỏng; khoáng vật phải được đào theo đúng thứ tự trong minerals và có thể dừng khi hết cuốc hoặc hết khoáng. Hãy chọn thứ tự sử dụng các loại cuốc để tổng mệt mỗi nhỏ nhất.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: Pick có giới hạn, mỗi pick đào tối đa 5 khoáng theo thứ tự
- **Pattern**: Greedy grouping
- **Vì sao**: Chỉ xét số mineral đào được = picks*5; group 5; score group theo độ khó; sort desc; gán pick tốt nhất

C. Tư duy lời giải từng bước

70. Chỉ xét số mineral đào được = picks*5.

71. group 5.

72. score group theo độ khó.

73. sort desc.

74. gán pick tốt nhất.

Invariant / state phải giữ

Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: Chỉ xét số mineral đào được = picks*5; group 5; score group theo độ khó; sort desc; gán pick tốt nhất

D. JavaScript solution() - block code để học/copy

```
function solution(picks, minerals) {
  const can=Math.min(minerals.length, (picks[0]+picks[1]+picks[2])*5);
  const groups=[];
  const cost= {
    diamond:[1, 5, 25], iron:[1, 1, 5], stone:[1, 1, 1]
  }
  ;
  for(let i=0; i<can; i+=5) {
    let d=0, ir=0, st=0;
    for(const m of minerals.slice(i, i+5)) {
      d+=cost[m][0];
      ir+=cost[m][1];
      st+=cost[m][2];
    }
  }
}
```

```

groups.push([st, ir, dl]);
// hardness first
}
groups.sort((a, b)=>b[0]-a[0] || b[1]-a[1] || b[2]-a[2]);
let ans=0, gi=0;
for(let pick=0; pick<3; pick++) while(picks[pick]-- > 0 &&
gi<groups.length) ans+=groups[gi++][pick===0?2:pick===1?1:0];
return ans;
}

```

E. Đọc code theo cấu trúc

- Phần sort chuẩn hóa thứ tự dữ liệu trước khi áp dụng greedy/two pointers/binary search hoặc comparator của bài.
- Vòng while là phần điều chỉnh state cho tới khi invariant trở lại đúng; khi học, cần nói được chính xác điều kiện while có nghĩa gì.
- Biến/điều kiện quan trọng nhất của bài: Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: Chỉ xét số mineral đào được = picks*5; group 5; score group theo độ khó; sort desc; gán pick tốt nhất

F. Complexity + hidden-test traps

- **Độ phức tạp:** $O(n \log n)$
- **Bẫy dễ sai:** Sort cả group ngoài range; nhóm cuối; matrix fatigue
- **Recall 30 giây:** Greedy grouping → Chỉ xét số mineral đào được = picks*5; group 5; score group theo độ khó; sort desc; gán pick tốt nhất

28. 구명보트

Thuyền cứu sinh

Chủ đề	Greedy	Pattern	Two pointers
Priority	A	Complexity	$O(n \log n)$

A. Đề bài tiếng Việt - đọc để hiểu đề

Mỗi thuyền chở tối đa 2 người và tổng cân nặng không vượt limit. Mỗi người phải được đưa đi và ta muốn số thuyền ít nhất. Sau khi sắp cân nặng, luôn cho người nặng nhất lên; nếu người nhẹ nhất còn có thể đi cùng thì ghép hai người, nếu không người nặng nhất đi một mình.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: Mỗi thuyền tối đa 2 người, limit
- **Pattern**: Two pointers
- **Vì sao**: Sort asc; left lightest/right heaviest; luôn lấy right, nếu $left + right \leq limit$ thì $left++$

C. Tư duy lời giải từng bước

75. Sort asc.

76. left lightest/right heaviest.

77. luôn lấy right, nếu $left + right \leq limit$ thì $left++$.

Invariant / state phải giữ

right là người nặng nhất chưa xử lý. Người này chắc chắn phải dùng một thuyền; quyết định duy nhất là có ghép được left hay không.

D. JavaScript solution() - block code để học/copy

```
function solution(people, limit) {
  people.sort((a, b) => a - b);
  let l = 0, r = people.length - 1, boats = 0;
  while(l <= r) {
    if(l === r) {
      boats++;
      break;
    }
    if(people[l] + people[r] <= limit) l++;
    r--;
    boats++;
  }
  return boats;
}
```

E. Đọc code theo cấu trúc

- Phân sort chuẩn hóa thứ tự dữ liệu trước khi áp dụng greedy/two pointers/binary search hoặc comparator của bài.

- Vòng while là phần điều chỉnh state cho tới khi invariant trở lại đúng; khi học, cần nói được chính xác điều kiện while có nghĩa gì.
- Biến/điều kiện quan trọng nhất của bài: right là người nặng nhất chưa xử lý. Người này chắc chắn phải dùng một thuyền; quyết định duy nhất là có ghép được left hay không.

F. Complexity + hidden-test traps

- **Độ phức tạp:** $O(n \log n)$
- **Bẫy dễ sai:** Khi không ghép được vẫn phải right-- và boats++
- **Recall 30 giây:** Two pointers → Sort asc; left lightest/right heaviest; luôn lấy right, nếu left+right<=limit thì left++

29. 단속카메라

Camera tốc độ

Chủ đề	Greedy	Pattern	Interval greedy
Priority	B	Complexity	$O(n \log n)$

A. Đề bài tiếng Việt - đọc để hiểu đề

Mỗi xe có đoạn [entry,exit] trên trục đường. Một camera đặt tại vị trí x sẽ chụp được xe nếu x nằm trong đoạn di chuyển của xe đó. Hãy đặt ít camera nhất để mọi xe đều bị chụp ít nhất một lần. Bài toán tương đương chọn ít điểm nhất đâm xuyên tất cả các interval.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: Đặt ít camera nhất cắt mọi interval
- **Pattern**: Interval greedy
- **Vì sao**: Sort routes theo end asc; nếu start > camera thì đặt camera=end

C. Tư duy lời giải từng bước

78. Sort routes theo end asc.

79. nếu start > camera thì đặt camera=end.

Invariant / state phải giữ

Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: Sort routes theo end asc; nếu start > camera thì đặt camera=end

D. JavaScript solution() - block code để học/copy

```
function solution(routes) {
  routes.sort((a, b) => a[1] - b[1]);
  let cam = -Infinity, ans = 0;
  for(const [s, e] of routes) if(s > cam) {
    cam = e;
    ans++;
  }
  return ans;
}
```

E. Đọc code theo cấu trúc

- Phần sort chuẩn hóa thứ tự dữ liệu trước khi áp dụng greedy/two pointers/binary search hoặc comparator của bài.
- Biến/điều kiện quan trọng nhất của bài: Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: Sort routes theo end asc; nếu start > camera thì đặt camera=end

F. Complexity + hidden-test traps

- **Độ phức tạp**: $O(n \log n)$

- **Bẫy dễ sai:** Inclusive boundary: $\text{start} \leq \text{camera}$ được cover
- **Recall 30 giây:** Interval greedy $\rightarrow$ Sort routes theo end asc; nếu $\text{start} > \text{camera}$ thì đặt $\text{camera} = \text{end}$

30. 최고의 집합

Tập hợp tốt nhất

Chủ đề	Greedy	Pattern	Equal distribution
Priority	B	Complexity	O(n)

A. Đề bài tiếng Việt - đọc để hiểu đề

Cần tạo một mảng s số nguyên dương có tổng bằng n sao cho tích của các phần tử lớn nhất. Nếu không thể tạo vì $n < s$ thì trả về $[-1]$. Khi có thể, tích lớn nhất đạt được khi các số càng cân bằng càng tốt; hãy trả về mảng theo thứ tự tăng dần.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: n số dương tổng s , product max
- **Pattern**: Equal distribution
- **Vì sao**: Nếu $s < n \rightarrow [-1]$; chia đều: $\text{base} = \text{floor}(s/n)$, remainder; output nondecreasing

C. Tư duy lời giải từng bước

80. Nếu $s < n \rightarrow [-1]$.

81. chia đều: $\text{base} = \text{floor}(s/n)$, remainder.

82. output nondecreasing.

Invariant / state phải giữ

Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: Nếu $s < n \rightarrow [-1]$; chia đều: $\text{base} = \text{floor}(s/n)$, remainder; output nondecreasing

D. JavaScript solution() - block code để học/copy

```
function solution(n, s) {
  if(s < n) return [-1];
  const base = Math.floor(s/n), rem = s % n;
  return [...Array(n - rem).fill(base), ...Array(rem).fill(base + 1)];
}
```

E. Đọc code theo cấu trúc

- Đọc code thành ba phần: khởi tạo state $\rightarrow$ cập nhật state theo từng phần tử/lựa chọn $\rightarrow$ trả đáp án. Đừng học thuộc từng dòng rồi rạc.
- Biến/điều kiện quan trọng nhất của bài: Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: Nếu $s < n \rightarrow [-1]$; chia đều: $\text{base} = \text{floor}(s/n)$, remainder; output nondecreasing

F. Complexity + hidden-test traps

- **Độ phức tạp**: $O(n)$
- **Bẫy dễ sai**: Dùng số chẵn quá 1 làm product giảm

- **Recall 30 giây:** Equal distribution → Nếu $s < n \rightarrow [-1]$; chia đều: $\text{base} = \text{floor}(s/n)$, remainder; output nondecreasing

31. 두 큐 합 같게 만들기

Làm tổng hai queue bằng nhau

Chủ đề	Prefix Sum/Sliding Window	Pattern	Two queues / two pointers
Priority	A	Complexity	O(n)

A. Đề bài tiếng Việt - đọc để hiểu đề

Có hai queue chứa số dương. Mỗi thao tác lấy phần tử đầu của một queue và đẩy vào cuối queue kia. Hãy tìm số thao tác nhỏ nhất để tổng hai queue bằng nhau, hoặc -1 nếu không thể. Vì tổng toàn bộ cố định, mục tiêu mỗi queue phải đạt đúng một nửa tổng.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: Chuyển đầu queue qua lại để tổng bằng nhau
- **Pattern**: Two queues / two pointers
- **Vì sao**: $\text{target} = (\text{sum1} + \text{sum2}) / 2$; nếu $\text{sum1} < \text{target}$ lấy từ q2, $> \text{target}$ lấy từ q1; head indices

C. Tư duy lời giải từng bước

83. $\text{target} = (\text{sum1} + \text{sum2}) / 2$.

84. nếu $\text{sum1} < \text{target}$ lấy từ q2, $> \text{target}$ lấy từ q1.

85. head indices.

Invariant / state phải giữ

Hai con trỏ mô phỏng đúng thứ tự dequeue/enqueue trên dãy ghép; tổng q1 được cập nhật O(1) sau mỗi move.

D. JavaScript solution() - block code để học/copy

```
function solution(queue1, queue2) {
  const total = [...queue1, ...queue2].reduce((a, b) => a + b, 0);
  if (total % 2) return -1;
  const target = total / 2, a = [...queue1, ...queue2, ...queue1, ...queue2];
  let l = 0, r = queue1.length, sum = queue1.reduce((x, y) => x + y, 0), moves = 0;
  const limit = queue1.length * 4;
  while (moves <= limit) {
    if (sum === target) return moves;
    if (sum < target) sum += a[r++];
    else sum -= a[l++];
    moves++;
  }
  return -1;
}
```

E. Đọc code theo cấu trúc

- Vòng while là phần điều chỉnh state cho tới khi invariant trở lại đúng; khi học, cần nói được chính xác điều kiện while có nghĩa gì.

- BFS dùng queue; tránh `Array.shift()` trên dữ liệu lớn, ưu tiên head index để dequeue $O(1)$.
- Biến/điều kiện quan trọng nhất của bài: Hai con trỏ mô phỏng đúng thứ tự dequeue/enqueue trên dãy ghép; tổng $q1$ được cập nhật $O(1)$ sau mỗi move.

F. Complexity + hidden-test traps

- **Độ phức tạp:** $O(n)$
- **Bẫy dễ sai:** Tổng lẻ; vòng lặp vô hạn $\rightarrow$ bound $\sim 4n$
- **Recall 30 giây:** Two queues / two pointers $\rightarrow$ $\text{target} = (\text{sum1} + \text{sum2}) / 2$; nếu $\text{sum1} < \text{target}$ lấy từ $q2$, $> \text{target}$ lấy $q1$; head indices

32. 연속된 부분 수열의 합

Tổng dãy con liên tiếp

Chủ đề	Prefix Sum/Sliding Window	Pattern	Sliding window positive
Priority	A	Complexity	O(n)

A. Đề bài tiếng Việt - đọc để hiểu đề

Cho sequence không giảm và số k. Hãy tìm một đoạn liên tiếp có tổng đúng k; nếu có nhiều đoạn, ưu tiên đoạn ngắn nhất, nếu vẫn hòa thì ưu tiên chỉ số bắt đầu nhỏ hơn. Trả về [left,right]. Tính chất số dương/không giảm cho phép dùng two pointers/sliding window.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: Tìm subarray sum=k, ngắn nhất, tie earliest
- **Pattern**: Sliding window positive
- **Vì sao**: right mở rộng sum; while sum>k shrink; khi ==k update length

C. Tư duy lời giải từng bước

86. right mở rộng sum.
87. while sum>k shrink.
88. khi ==k update length.

Invariant / state phải giữ

Window [left,right] là đoạn đang xét; khi sum > k, tăng left vì mọi số không âm nên mở rộng thêm không thể làm tổng giảm.

D. JavaScript solution() - block code để học/copy

```
function solution(sequence, k) {
  let l=0, sum=0, best=[0, sequence.length-1];
  for(let r=0; r<sequence.length; r++) {
    sum+=sequence[r];
    while(sum>k && l<=r) sum-=sequence[l++];
    if(sum==k && r-l<best[1]-best[0]) best=[l, r];
  }
  return best;
}
```

E. Đọc code theo cấu trúc

- Vòng while là phần điều chỉnh state cho tới khi invariant trở lại đúng; khi học, cần nói được chính xác điều kiện while có nghĩa gì.
- Biến/điều kiện quan trọng nhất của bài: Window [left,right] là đoạn đang xét; khi sum > k, tăng left vì mọi số không âm nên mở rộng thêm không thể làm tổng giảm.

F. Complexity + hidden-test traps

- **Độ phức tạp:** $O(n)$
- **Bẫy dễ sai:** Chỉ dùng được vì số dương
- **Recall 30 giây:** Sliding window positive $\rightarrow$ right mở rộng sum; while sum > k shrink; khi == k update length

33. 파괴되지 않은 건물

Tòa nhà không bị phá hủy

★ BẮT BUỘC theo roadmap team kia

Chủ đề	Prefix Sum/Sliding Window	Pattern	2D difference / prefix
Priority	A	Complexity	O(NM+K)

A. Đề bài tiếng Việt - đọc để hiểu đề

Một bảng biểu diễn độ bền của các tòa nhà. Mỗi skill tác động cộng hoặc trừ một giá trị lên toàn bộ hình chữ nhật $[r1, c1]..[r2, c2]$. Có thể có rất nhiều skill nên cập nhật từng ô sẽ quá chậm. Sau khi áp dụng tất cả skill, hãy đếm số ô có độ bền > 0 . Cần dùng mảng hiệu 2D rồi prefix sum để gộp toàn bộ tác động.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: Nhiều rectangle range updates, cuối đếm > 0
- **Pattern**: 2D difference / prefix
- **Vì sao**: Diff 2D: 4 corner updates; prefix ngang rồi dọc; cộng board

C. Tư duy lời giải từng bước

89. Diff 2D: 4 corner updates.

90. prefix ngang rồi dọc.

91. cộng board.

Invariant / state phải giữ

Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: Diff 2D: 4 corner updates; prefix ngang rồi dọc; cộng board

D. JavaScript solution() - block code để học/copy

```
function solution(board, skill) {
  const R=board.length, C=board[0].length;
  const diff=Array.from( {
    length:R+1
  }, ()=>Array(C+1).fill(0));
  for(const [type, r1, c1, r2, c2, degree] of skill) {
    const v=type===1?-degree:degree;
    diff[r1][c1]+=v;
    diff[r1][c2+1]-=v;
    diff[r2+1][c1]-=v;
    diff[r2+1][c2+1]+=v;
  }
  for(let r=0; r<=R; r++) for(let c=1; c<=C; c++) diff[r][c]+=diff[r][c-1];
  for(let c=0; c<=C; c++) for(let r=1; r<=R; r++) diff[r][c]+=diff[r-1][c];
```



```
let ans=0;
for(let r=0; r<R; r++) for(let c=0; c<C; c++) if(board[r][c]+diff[r][c]>0) ans++;
return ans;
}
```

E. Đọc code theo cấu trúc

- Đọc code thành ba phần: khởi tạo state → cập nhật state theo từng phần tử/lựa chọn → trả đáp án. Đừng học thuộc từng dòng rồi rạc.
- Biến/điều kiện quan trọng nhất của bài: Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: Diff 2D: 4 corner updates; prefix ngang rồi dọc; cộng board

F. Complexity + hidden-test traps

- **Độ phức tạp:** $O(NM+K)$
- **Bẫy dễ sai:** Dấu ở 4 góc; size $n+1, m+1$; skill type attack/heal
- **Recall 30 giây:** 2D difference / prefix → Diff 2D: 4 corner updates; prefix ngang rồi dọc; cộng board

34. 보석 쇼핑

Mua sắm đá quý

Chủ đề	Prefix Sum/Sliding Window	Pattern	Variable sliding window + Map
Priority	A	Complexity	O(n)

A. Đề bài tiếng Việt - đọc để hiểu đề

Một dãy cửa hàng bán các loại đá quý; bạn muốn mua một đoạn liên tiếp ngắn nhất chứa đủ mọi loại đá quý xuất hiện trong toàn bộ dãy. Nếu có nhiều đoạn cùng ngắn nhất thì chọn đoạn bắt đầu sớm hơn. Trả về vị trí bắt đầu/kết thúc theo chỉ số từ 1. Đây là sliding window với Map tần suất.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: Đoạn ngắn nhất chứa đủ mọi loại
- **Pattern**: Variable sliding window + Map
- **Vì sao**: Need=Set size; right add; while map.size==need update answer rồi remove left

C. Tư duy lời giải từng bước

92. Need=Set size.

93. right add.

94. while map.size==need update answer rồi remove left.

Invariant / state phải giữ

Window luôn chứa số loại được theo dõi bởi Map; khi đã đủ mọi loại, dịch left tới khi không thể co thêm mà vẫn đủ.

D. JavaScript solution() - block code để học/copy

```
function solution(gems) {
  const need=new Set(gems).size, cnt=new Map();
  let l=0, best=[0, gems.length-1];
  for(let r=0; r<gems.length; r++) {
    cnt.set(gems[r], (cnt.get(gems[r])??0)+1);
    while(cnt.size==need) {
      if(r-l<best[1]-best[0]) best=[l, r];
      const x=gems[l++], n=cnt.get(x)-1;
      if(n===0) cnt.delete(x);
      else cnt.set(x, n);
    }
  }
  return [best[0]+1, best[1]+1];
}
```

E. Đọc code theo cấu trúc

- Map lưu tần suất / ánh xạ / trạng thái để truy cập O(1) trung bình thay vì quét lại dữ liệu.

- Set dùng để kiểm tra uniqueness/visited hoặc loại trùng mà không cần vòng lặp lồng nhau.
- Vòng while là phần điều chỉnh state cho tới khi invariant trở lại đúng; khi học, cần nói được chính xác điều kiện while có nghĩa gì.
- Biến/điều kiện quan trọng nhất của bài: Window luôn chứa số loại được theo dõi bởi Map; khi đã đủ mọi loại, dịch left tới khi không thể co thêm mà vẫn đủ.

F. Complexity + hidden-test traps

- **Độ phức tạp:** $O(n)$
- **Bẫy dễ sai:** Count về 0 phải delete key; tie earliest
- **Recall 30 giây:** Variable sliding window + Map $\rightarrow$ Need=Set size; right add; while map.size==need update answer rồi remove left

35. 퍼즐 게임 챌린지

Thử thách game giải đố

★ **BẮT BUỘC** theo roadmap team kia

Chủ đề	Binary Search	Pattern	Binary search on answer
Priority	A	Complexity	$O(n \log \text{maxDiff})$

A. Đề bài tiếng Việt - đọc để hiểu đề

Có nhiều level puzzle, mỗi puzzle có độ khó và thời gian xử lý. Nếu skill của người chơi thấp hơn difficulty thì phải mắc lỗi nhiều lần và tốn thêm thời gian phụ; nếu skill đủ cao thì chỉ tốn thời gian cơ bản. Cho tổng thời gian limit, hãy tìm skill nhỏ nhất để hoàn thành toàn bộ puzzle trong limit. Predicate 'skill x có hoàn thành kịp không?' là đơn điệu nên binary search trên đáp án.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: Tìm skill level nhỏ nhất để $\text{total time} \leq \text{limit}$
- **Pattern**: Binary search on answer
- **Vì sao**: feasible(level) tính total; level tăng => time không tăng; lower_bound first true

C. Tư duy lời giải từng bước

95. feasible(level) tính total.

96. level tăng => time không tăng.

97. lower_bound first true.

Invariant / state phải giữ

Nếu skill x hoàn thành kịp, mọi skill lớn hơn x cũng hoàn thành kịp; đó là tính đơn điệu để binary search.

D. JavaScript solution() - block code để học/copy

```
function solution(diffs, times, limit) {
  const ok=(level)=> {
    let total=0;
    for(let i=0; i<diffs.length; i++) {
      if(diffs[i]<=level) total+=times[i];
      else total+=(diffs[i]-level)*(times[i]+(i?times[i-1]:0))+times[i];
      if(total>limit) return false;
    }
    return true;
  };
  let lo=0, hi=Math.max(...diffs);
  while(lo+1<hi) {
    const mid=Math.floor((lo+hi)/2);
    if(ok(mid)) hi=mid;
    else lo=mid;
  }
}
```

```
}  
return ok(0)?0:hi;  
}
```

E. Đọc code theo cấu trúc

- Vòng while là phần điều chỉnh state cho tới khi invariant trở lại đúng; khi học, cần nói được chính xác điều kiện while có nghĩa gì.
- Biến/điều kiện quan trọng nhất của bài: Nếu skill x hoàn thành kịp, mọi skill lớn hơn x cũng hoàn thành kịp; đó là tính đơn điệu để binary search.

F. Complexity + hidden-test traps

- **Độ phức tạp:** $O(n \log \text{maxDiff})$
- **Bẫy dễ sai:** times[i-1] ở i=0; overflow Number vẫn safe trong JS nhưng early break tốt
- **Recall 30 giây:** Binary search on answer → feasible(level) tính total; level tăng => time không tăng; lower_bound first true

36. 순위 검색

Tìm kiếm xếp hạng

Chủ đề	Binary Search	Pattern	Precompute buckets + lower_bound
Priority	B	Complexity	$O(N*16 + Q \log N)$

A. Đề bài tiếng Việt - đọc để hiểu đề

Mỗi ứng viên có 4 thuộc tính phân loại và một điểm số. Có nhiều query trong đó mỗi thuộc tính có thể là giá trị cụ thể hoặc '-' nghĩa là bất kỳ, kèm điều kiện điểm $\geq x$. Với mỗi query, hãy đếm số ứng viên thỏa. Cần tiền xử lý các tổ hợp key và lưu danh sách điểm đã sort để mỗi query dùng lower_bound.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: Query 4 thuộc tính có wildcard + score threshold
- **Pattern**: Precompute buckets + lower_bound
- **Vì sao**: Mỗi applicant sinh 16 key wildcard; mỗi key sort scores; query lower_bound $\geq$ score

C. Tư duy lời giải từng bước

98. Mỗi applicant sinh 16 key wildcard.

99. mỗi key sort scores.

100. query lower_bound $\geq$ score.

Invariant / state phải giữ

Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: Mỗi applicant sinh 16 key wildcard; mỗi key sort scores; query lower_bound $\geq$ score

D. JavaScript solution() - block code để học/copy

```
function solution(info, query) {
  const map=new Map();
  for(const line of info) {
    const [a, b, c, d, s]=line.split(' '), score=Number(s), attrs=[a, b, c, d];
    for(let mask=0; mask<16; mask++) {
      const key=attrs.map((x, i)=>(mask>>i)&1?'-':x).join(' ');
      if(!map.has(key)) map.set(key, []);
      map.get(key).push(score);
    }
  }
  for(const arr of map.values()) arr.sort((a, b)=>a-b);
  const lower=(arr, x)=> {
    let l=0, r=arr.length;
    while(l<r) {
      const m=(l+r)>>1;
      if(arr[m]>=x)r=m;
      else l=m+1;
    }
  }
```

```

    }
    return l;
  }
  ;
  return query.map(q=> {
    const parts=q.replace(/ and /g, ' ').split(' '), score=Number(parts.pop()), key=parts.join(' ');
    const arr=map.get(key)??[]; return arr.length-lower(arr, score);
  }
  );
}

```

E. Đọc code theo cấu trúc

- Phần sort chuẩn hóa thứ tự dữ liệu trước khi áp dụng greedy/two pointers/binary search hoặc comparator của bài.
- Map lưu tần suất / ánh xạ / trạng thái để truy cập $O(1)$ trung bình thay vì quét lại dữ liệu.
- Vòng while là phần điều chỉnh state cho tới khi invariant trở lại đúng; khi học, cần nói được chính xác điều kiện while có nghĩa gì.
- Biến/điều kiện quan trọng nhất của bài: Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: Mỗi applicant sinh 16 key wildcard; mỗi key sort scores; query lower_bound $\geq$ score

F. Complexity + hidden-test traps

- **Độ phức tạp:** $O(N*16 + Q \log N)$
- **Bẫy dễ sai:** String key consistency; binary search boundary
- **Recall 30 giây:** Precompute buckets + lower_bound $\rightarrow$ Mỗi applicant sinh 16 key wildcard; mỗi key sort scores; query lower_bound $\geq$ score

37. Nhập quốc gia

Kiểm tra nhập cảnh

★ **BẮT BUỘC** theo roadmap team kia

Chủ đề	Binary Search	Pattern	Binary search on answer
Priority	A	Complexity	$O(m \log \text{answer})$

A. Đề bài tiếng Việt - đọc để hiểu đề

Có n người cần qua kiểm tra nhập cảnh và nhiều quầy, quầy i cần $\text{times}[i]$ phút cho một người. Các quầy làm việc song song. Hãy tìm thời gian nhỏ nhất để xử lý ít nhất n người. Với một thời gian T , số người tối đa xử lý được là $\text{sum}(\text{floor}(T/\text{times}[i]))$, tạo predicate đơn điệu cho binary search.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: Tìm thời gian nhỏ nhất để xử lý $\geq n$ người
- **Pattern**: Binary search on answer
- **Vì sao**: $\text{feasible}(t) = \text{sum floor}(t/\text{time}[i]) \geq n$; search $[1, \text{max} * \text{time} * n]$

C. Tư duy lời giải từng bước

101. $\text{feasible}(t) = \text{sum floor}(t/\text{time}[i]) \geq n$.

102. search $[1, \text{max} * \text{time} * n]$.

Invariant / state phải giữ

Nếu thời gian T xử lý đủ n người, mọi thời gian $> T$ cũng đủ; kiểm tra bằng tổng $\text{floor}(T/\text{time}[i])$.

D. JavaScript solution() - block code để học/copy

```
function solution(n, times) {
  let lo=0, hi=Math.max(...times)*n;
  while(lo+1<hi) {
    const mid=Math.floor((lo+hi)/2);
    let done=0;
    for(const t of times) {
      done+=Math.floor(mid/t);
      if(done>=n) break;
    }
    if(done>=n) hi=mid;
    else lo=mid;
  }
  return hi;
}
```

E. Đọc code theo cấu trúc

- Vòng while là phần điều chỉnh state cho tới khi invariant trở lại đúng; khi học, cần nói được chính xác điều kiện while có nghĩa gì.

- Biến/điều kiện quan trọng nhất của bài: Nếu thời gian T xử lý đủ n người, mọi thời gian $>T$ cũng đủ; kiểm tra bằng tổng $\text{floor}(T/\text{time}[i])$.

F. Complexity + hidden-test traps

- **Độ phức tạp:** $O(m \log \text{answer})$
- **Bẫy dễ sai:** Upper bound lớn; overflow ở ngôn ngữ int
- **Recall 30 giây:** Binary search on answer $\rightarrow \text{feasible}(t) = \sum \text{floor}(t/\text{time}[i]) \geq n$; search $[1, \text{max} * \text{time} * n]$

38. 징검다리 건너기

Bảng qua đá

★ BẮT BUỘC theo roadmap team kia

Chủ đề	Binary Search	Pattern	Binary search on answer
Priority	A	Complexity	$O(n \log \max \text{Stone})$

A. Đề bài tiếng Việt - đọc để hiểu đề

Có dãy đá, `stones[i]` là số người tối đa có thể giẫm lên viên đá đó trước khi nó trở nên không dùng được. Một người có thể nhảy qua tối đa $k-1$ viên liên tiếp, tức không được có k viên liên tiếp đều 'hổng' đối với mức người đang thử. Hãy tìm số người tối đa có thể qua sông. Có thể binary search số người và kiểm tra chuỗi k viên không chịu nổi mức đó.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: Max số người có thể qua trước khi k đá liên tiếp cạn
- **Pattern**: Binary search on answer
- **Vì sao**: `feasible(x)`: có xuất hiện k stones với `stone-x < 0` / `stone < x`? xác định convention rồi search max true

C. Tư duy lời giải từng bước

103. `feasible(x)`: có xuất hiện k stones với `stone-x < 0` / `stone < x`? xác định convention rồi search max true.
104. Duy trì state/invariant bên dưới và cập nhật đáp án khi điều kiện của bài được thỏa..

Invariant / state phải giữ

Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: `feasible(x)`: có xuất hiện k stones với `stone-x < 0` / `stone < x`? xác định convention rồi search max true

D. JavaScript solution() - block code để học/copy

```
function solution(stones, k) {
  const ok=(people)=> {
    let zero=0;
    for(const s of stones) {
      if(s<people) {
        if(++zero>=k) return false;
      } else zero=0;
    }
    return true;
  }
  ;
  let lo=0, hi=200000001;
  while(lo+1<hi) {
    const mid=Math.floor((lo+hi)/2);
    if(ok(mid)) lo=mid;
    else hi=mid;
  }
}
```

```
}  
return lo;  
}
```

E. Đọc code theo cấu trúc

- Vòng while là phần điều chỉnh state cho tới khi invariant trở lại đúng; khi học, cần nói được chính xác điều kiện while có nghĩa gì.
- Biến/điều kiện quan trọng nhất của bài: Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: `feasible(x)`: có xuất hiện `k` stones với `stone-x < 0` / `stone < x`? xác định convention rồi search max true

F. Complexity + hidden-test traps

- **Độ phức tạp:** $O(n \log \text{maxStone})$
- **Bẫy dễ sai:** Off-by-one cực dễ; predicate người thứ `x` có qua được hay không
- **Recall 30 giây:** Binary search on answer $\rightarrow$ `feasible(x)`: có xuất hiện `k` stones với `stone-x < 0` / `stone < x`? xác định convention rồi search max true

39. 피로도

Độ mệt mỏi

★ **BẮT BUỘC** theo roadmap team kia

Chủ đề	DFS/Backtracking	Pattern	Permutation DFS
Priority	A	Complexity	$O(n!)$ $n \leq 8$

A. Đề bài tiếng Việt - đọc để hiểu đề

Người chơi bắt đầu với độ mệt k . Mỗi dungeon có $[\text{minimum}, \text{cost}]$: chỉ vào được khi mệt hiện tại $\geq \text{minimum}$, và sau khi đi sẽ giảm cost. Mỗi dungeon chỉ đi tối đa một lần, thử tự chọn. Hãy tìm số dungeon tối đa có thể khám phá. Vì số dungeon nhỏ, thử mọi thứ tự bằng DFS/backtracking.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: Tối đa dungeon đi được với fatigue k
- **Pattern**: Permutation DFS
- **Vì sao**: `visited[]`; `DFS(currentK, count)`; thử mọi dungeon đủ min; `mark` -> `recurse` -> `unmark`

C. Tư duy lời giải từng bước

105. `visited[]`.
106. `DFS(currentK, count)`.
107. thử mọi dungeon đủ min.
108. `mark` -> `recurse` -> `unmark`.

Invariant / state phải giữ

`visited` đánh dấu dungeon đã đi trong nhánh DFS; fatigue là lượng mệt còn lại đúng sau chuỗi lựa chọn của nhánh đó.

D. JavaScript solution() - block code để học/copy

```
function solution(k, dungeons) {
  let best=0;
  const used=Array(dungeons.length).fill(false);
  function dfs(fatigue, count) {
    best=Math.max(best, count);
    for(let i=0; i<dungeons.length; i++) {
      if(used[i] || fatigue<dungeons[i][0]) continue;
      used[i]=true;
      dfs(fatigue-dungeons[i][1], count+1);
      used[i]=false;
    }
  }
  dfs(k, 0);
  return best;
}
```

E. Đọc code theo cấu trúc

- DFS tương ứng với một chuỗi lựa chọn; trước khi gọi sâu hơn phải cập nhật state, và sau khi quay về phải backtrack nếu state dùng chung.
- Biến/điều kiện quan trọng nhất của bài: visited đánh dấu dungeon đã đi trong nhánh DFS; fatigue là lượng mệt còn lại đúng sau chuỗi lựa chọn của nhánh đó.

F. Complexity + hidden-test traps

- **Độ phức tạp:** $O(n!)$ $n \leq 8$
- **Bẫy dễ sai:** Quên backtrack visited; minRequired khác consumed
- **Recall 30 giây:** Permutation DFS $\rightarrow$ visited[]; DFS(currentK,count); thử mọi dungeon đủ min; mark $\rightarrow$ recurse $\rightarrow$ unmark

40. N-Queen

N quân hậu

Chủ đề	DFS/Backtracking	Pattern	Backtracking constraints
Priority	B	Complexity	$O(N!)$ pruned

A. Đề bài tiếng Việt - đọc để hiểu đề

Đặt N quân hậu lên bàn $N \times N$ sao cho không hai quân nào cùng hàng, cùng cột hoặc cùng đường chéo. Vì ta có thể đặt đúng một hậu ở mỗi hàng, DFS theo hàng và thử từng cột còn hợp lệ. Hãy đếm tổng số cách đặt hợp lệ.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: Đặt N queen không cùng cột/chéo
- **Pattern**: Backtracking constraints
- **Vì sao**: row-by-row; usedCol, diag1=r-c, diag2=r+c

C. Tư duy lời giải từng bước

109. row-by-row.

110. usedCol, diag1=r-c, diag2=r+c.

Invariant / state phải giữ

Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: row-by-row; usedCol, diag1=r-c, diag2=r+c

D. JavaScript solution() - block code để học/copy

```
function solution(n) {
  let ans=0;
  const cols=new Set(), d1=new Set(), d2=new Set();
  function dfs(r) {
    if(r===n) {
      ans++;
      return;
    }
    for(let c=0; c<n; c++) {
      if(cols.has(c) || d1.has(r-c) || d2.has(r+c)) continue;
      cols.add(c);
      d1.add(r-c);
      d2.add(r+c);
      dfs(r+1);
      cols.delete(c);
      d1.delete(r-c);
      d2.delete(r+c);
    }
  }
  dfs(0);
  return ans;
}
```

```
}
```

E. Đọc code theo cấu trúc

- Set dùng để kiểm tra uniqueness/visited hoặc loại trùng mà không cần vòng lặp lồng nhau.
- DFS tương ứng với một chuỗi lựa chọn; trước khi gọi sâu hơn phải cập nhật state, và sau khi quay về phải backtrack nếu state dùng chung.
- Biến/điều kiện quan trọng nhất của bài: Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mẫu chốt là: row-by-row; usedCol, diag1=r-c, diag2=r+c

F. Complexity + hidden-test traps

- **Độ phức tạp:** $O(N!)$ pruned
- **Bẫy dễ sai:** Copy board không cần thiết; diagonal keys
- **Recall 30 giây:** Backtracking constraints → row-by-row; usedCol, diag1=r-c, diag2=r+c

41. 후보키

Khóa ứng viên

Chủ đề	DFS/Backtracking	Pattern	Combination + set
Priority	C	Complexity	$O(2^C * R)$

A. Đề bài tiếng Việt - đọc để hiểu đề

Một relation là bảng dữ liệu. Một tập cột là candidate key nếu (1) giá trị kết hợp của các cột đó phân biệt được mọi hàng - tính duy nhất, và (2) không thể bỏ bớt cột nào mà vẫn duy nhất - tính tối thiểu. Hãy đếm số candidate key. Có thể duyệt các subset cột, kiểm tra uniqueness bằng Set và minimality với các key đã chọn.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: Tìm tập cột unique và minimal
- **Pattern**: Combination + set
- **Vì sao**: Enumerate bitmasks/combinations; uniqueness bằng Set row signatures; minimal: không chứa candidate cũ

C. Tư duy lời giải từng bước

111. Enumerate bitmasks/combinations.
112. uniqueness bằng Set row signatures.
113. minimal: không chứa candidate cũ.

Invariant / state phải giữ

Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mẫu chốt là: Enumerate bitmasks/combinations; uniqueness bằng Set row signatures; minimal: không chứa candidate cũ

D. JavaScript solution() - block code để học/copy

```
function solution(relation) {
  const R=relation.length, C=relation[0].length, keys=[];
  for(let mask=1; mask<(1<<C); mask++) {
    if(keys.some(k=>(k&mask)===k)) continue;
    const set=new Set();
    for(const row of relation) {
      const key=row.filter((_, i)=>mask&(1<<i)).join("\u0001");
      set.add(key);
    }
    if(set.size===R) keys.push(mask);
  }
  return keys.length;
}
```

E. Đọc code theo cấu trúc

- Set dùng để kiểm tra uniqueness/visited hoặc loại trùng mà không cần vòng lặp lồng nhau.

- Biến/điều kiện quan trọng nhất của bài: Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: Enumerate bitmasks/combinations; uniqueness bằng Set row signatures; minimal: không chứa candidate cũ

F. Complexity + hidden-test traps

- **Độ phức tạp:** $O(2^C * R)$
- **Bẫy dễ sai:** Minimality kiểm tra subset bitmask
- **Recall 30 giây:** Combination + set → Enumerate bitmasks/combinations; uniqueness bằng Set row signatures; minimal: không chứa candidate cũ

42. 메뉴 리뉴얼

Làm mới menu

Chủ đề	DFS/Backtracking	Pattern	Combination counting
Priority	B	Complexity	$O(\sum C(m,k))$

A. Đề bài tiếng Việt - đọc để hiểu đề

Dựa trên lịch sử gọi món, mỗi order là chuỗi các món đã gọi cùng nhau. Với từng độ dài course, cần tìm các combo món xuất hiện trong ít nhất 2 order và có tần suất cao nhất ở độ dài đó. Thứ tự món trong combo không quan trọng. Hãy sinh tổ hợp từ từng order đã sort, đếm bằng Map rồi lấy các combo đạt max cho từng course.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: Tổ hợp món phổ biến theo từng course length
- **Pattern**: Combination counting
- **Vì sao**: Sort chars mỗi order; generate combinations size k; Map count; lấy $\max \geq 2$

C. Tư duy lời giải từng bước

114. Sort chars mỗi order.
115. generate combinations size k.
116. Map count.
117. lấy $\max \geq 2$.

Invariant / state phải giữ

Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: Sort chars mỗi order; generate combinations size k; Map count; lấy $\max \geq 2$

D. JavaScript solution() - block code để học/copy

```
function solution(orders, course) {
  const ans=[];
  for(const len of course) {
    const cnt=new Map();
    const comb=(arr, start, pick)=> {
      if(pick.length===len) {
        const k=pick.join("");
        cnt.set(k, (cnt.get(k)?0)+1);
        return;
      }
      for(let i=start; i<arr.length; i++) comb(arr, i+1, [...pick, arr[i]]);
    }
    ;
    for(const o of orders) comb([...o].sort(), 0, []);
    const max=Math.max(0, ...cnt.values());
    if(max>=2) for(const [k, v] of cnt) if(v===max) ans.push(k);
  }
}
```

```
return ans.sort();  
}
```

E. Đọc code theo cấu trúc

- Phần sort chuẩn hóa thứ tự dữ liệu trước khi áp dụng greedy/two pointers/binary search hoặc comparator của bài.
- Map lưu tần suất / ánh xạ / trạng thái để truy cập $O(1)$ trung bình thay vì quét lại dữ liệu.
- Biến/điều kiện quan trọng nhất của bài: Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: Sort chars mỗi order; generate combinations size k; Map count; lấy $\max \geq 2$

F. Complexity + hidden-test traps

- **Độ phức tạp:** $O(\sum C(m,k))$
- **Bẫy dễ sai:** Không tính duplicate order combination sai
- **Recall 30 giây:** Combination counting $\rightarrow$ Sort chars mỗi order; generate combinations size k; Map count; lấy $\max \geq 2$

43. 모음사전

Từ điển nguyên âm

Chủ đề	DFS/Backtracking	Pattern	DFS enumeration / positional weights
Priority	B	Complexity	$O(5^5)$

A. Đề bài tiếng Việt - đọc để hiểu đề

Một 'từ điển nguyên âm' chứa mọi chuỗi dài từ 1 đến 5 chỉ dùng A,E,I,O,U, được sắp theo thứ tự từ điển theo quy tắc sinh tự nhiên. Cho word, hãy trả về vị trí của nó tính từ 1. Có thể DFS sinh toàn bộ từ theo đúng thứ tự hoặc tính trực tiếp bằng trọng số vị trí.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: Thứ tự từ tạo từ A,E,I,O,U length<=5
- **Pattern**: DFS enumeration / positional weights
- **Vì sao**: DFS theo lexicographic và counter; hoặc weights [781,156,31,6,1]

C. Tư duy lời giải từng bước

118. DFS theo lexicographic và counter.

119. hoặc weights [781,156,31,6,1].

Invariant / state phải giữ

Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: DFS theo lexicographic và counter; hoặc weights [781,156,31,6,1]

D. JavaScript solution() - block code để học/copy

```
function solution(word) {
  const w=[781, 156, 31, 6, 1], idx= {
    A:0, E:1, I:2, O:3, U:4
  }
  ;
  let ans=0;
  for(let i=0; i<word.length; i++) ans += idx[word[i]]*w[i]+1;
  return ans;
}
```

E. Đọc code theo cấu trúc

- Đọc code thành ba phần: khởi tạo state → cập nhật state theo từng phần tử/lựa chọn → trả đáp án. Đừng học thuộc từng dòng rồi rạc.
- Biến/điều kiện quan trọng nhất của bài: Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: DFS theo lexicographic và counter; hoặc weights [781,156,31,6,1]

F. Complexity + hidden-test traps

- **Độ phức tạp:** $O(5^5)$
- **Bẫy dễ sai:** Đếm cả prefix word
- **Recall 30 giây:** DFS enumeration / positional weights → DFS theo lexicographic và counter; hoặc weights [781,156,31,6,1]

44. 양궁대회

Cuộc thi bắn cung

Chủ đề	DFS/Backtracking	Pattern	Backtracking allocation
Priority	B	Complexity	$O(2^{11})$

A. Đề bài tiếng Việt - đọc để hiểu đề

Hai người thi bắn cung với n mũi tên. Apeach đã có phân bố info; Ryan quyết định phân bố n mũi vào 11 vùng điểm 10..0. Ở mỗi vùng, ai có nhiều mũi hơn nhận toàn bộ điểm vùng, hòa thì Apeach thắng vùng. Ryan muốn chênh lệch điểm lớn nhất; nếu nhiều cách cùng chênh lệch, ưu tiên cách có nhiều mũi hơn ở vùng điểm thấp hơn. Trả về phân bố tốt nhất hoặc [-1] nếu không thể thắng.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: Phân n mũi tên vào 11 score để max chênh
- **Pattern**: Backtracking allocation
- **Vì sao**: DFS score 10->0, mỗi ô chọn thắng apeach bằng $a[i]+1$ hoặc bỏ; leftover dồn score0; tie low-score more

C. Tư duy lời giải từng bước

120. DFS score 10->0, mỗi ô chọn thắng apeach bằng $a[i]+1$ hoặc bỏ.
121. leftover dồn score0.
122. tie low-score more.

Invariant / state phải giữ

Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: DFS score 10->0, mỗi ô chọn thắng apeach bằng $a[i]+1$ hoặc bỏ; leftover dồn score0; tie low-score more

D. JavaScript solution() - block code để học/copy

```
function solution(n, info) {
  let bestDiff=0, best=null;
  const cur=Array(11).fill(0);
  const better=(a, b)=> {
    for(let i=10; i>=0; i--) if(a[i]!==b[i]) return a[i]>b[i];
    return false;
  }
  ;
  function dfs(i, remain) {
    if(i===10) {
      cur[10]=remain;
      score();
      cur[10]=0;
      return;
    }
    const need=info[i]+1;
    if(need<=remain) {
```

```

    cur[i]=need;
    dfs(i+1, remain-need);
    cur[i]=0;
  }
  dfs(i+1, remain);
}
function score() {
  let r=0, a=0;
  for(let i=0; i<11; i++) {
    if(cur[i]===0&&info[i]===0) continue;
    if(cur[i]>info[i]) r+=10-i;
    else a+=10-i;
  }
  const diff=r-a;
  if(diff>0 && (diff>bestDiff || (diff===bestDiff && (!best || better(cur, best))))) {
    bestDiff=diff;
    best=[...cur];
  }
}
dfs(0, n);
return best?[-1];
}

```

E. Đọc code theo cấu trúc

- DFS tương ứng với một chuỗi lựa chọn; trước khi gọi sâu hơn phải cập nhật state, và sau khi quay về phải backtrack nếu state dùng chung.
- Biến/điều kiện quan trọng nhất của bài: Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: DFS score 10->0, mỗi ô chọn thẳng apeach bằng $a[i]+1$ hoặc bỏ; leftover dồn score0; tie low-score more

F. Complexity + hidden-test traps

- **Độ phức tạp:** $O(2^{11})$
- **Bẫy dễ sai:** Tie-break compare từ index10 ngược
- **Recall 30 giây:** Backtracking allocation → DFS score 10->0, mỗi ô chọn thẳng apeach bằng $a[i]+1$ hoặc bỏ; leftover dồn score0; tie low-score more

45. 외벽 점검

Kiểm tra tường ngoài

★ **BẮT BUỘC** theo roadmap team kia

Chủ đề	DFS/Backtracking	Pattern	Circular + permutation
Priority	A	Complexity	$O(W * D! * W)$

A. Đề bài tiếng Việt - đọc để hiểu đề

Tường ngoài hình tròn có n vị trí, một số vị trí weak cần kiểm tra. Mỗi người bạn có thể kiểm tra liên tục một đoạn dài $dist[i]$, mỗi người dùng tối đa một lần. Hãy tìm số bạn ít nhất để phủ hết các điểm yếu hoặc -1 nếu không thể. Kỹ thuật thường dùng: nhân đôi dãy weak để phá vòng tròn, thử mọi điểm bắt đầu và permutation của $dist$.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: Chọn bạn theo $dist$ và điểm bắt đầu để cover weak circular
- **Pattern**: Circular + permutation
- **Vì sao**: Duplicate weak + n ; thử mỗi start; permutations $dist$; greedy cover bằng từng friend

C. Tư duy lời giải từng bước

123. Duplicate weak + n .
124. thử mỗi start.
125. permutations $dist$.
126. greedy cover bằng từng friend.

Invariant / state phải giữ

Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: Duplicate weak + n ; thử mỗi start; permutations $dist$; greedy cover bằng từng friend

D. JavaScript solution() - block code để học/copy

```
function solution(n, weak, dist) {
    const W=weak.length, ext=[...weak, ...weak.map(x=>x+n)], used=Array(dist.length).fill(false);
    let ans=Infinity;
    function perm(path) {
        if(path.length===dist.length) {
            for(let start=0; start<W; start++) {
                let cnt=1, cover=ext[start]+path[0], ok=true;
                for(let i=start; i<start+W; i++) if(ext[i]>cover) {
                    cnt++;
                    if(cnt>path.length) {
                        ok=false;
                        break;
                    }
                }
                cover=ext[i]+path[cnt-1];
            }
        }
    }
}
```



```

    if(ok) ans=Math.min(ans, cnt);
  }
  return;
}
for(let i=0; i<dist.length; i++) if(!used[i]) {
  used[i]=true;
  path.push(dist[i]);
  perm(path);
  path.pop();
  used[i]=false;
}
}
perm([]);
return ans===Infinity?-1:ans;
}

```

E. Đọc code theo cấu trúc

- Đọc code thành ba phần: khởi tạo state → cập nhật state theo từng phần tử/lựa chọn → trả đáp án. Đừng học thuộc từng dòng rồi rạc.
- Biến/điều kiện quan trọng nhất của bài: Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: Duplicate weak + n; thử mỗi start; permutations dist; greedy cover bằng từng friend

F. Complexity + hidden-test traps

- **Độ phức tạp:** $O(W * D! * W)$
- **Bẫy dễ sai:** Circular indexing; số friend min
- **Recall 30 giây:** Circular + permutation → Duplicate weak + n; thử mỗi start; permutations dist; greedy cover bằng từng friend

46. 양과 늑대

Cừu và sói

★ BẮT BUỘC theo roadmap team kia

Chủ đề	DFS/Backtracking	Pattern	State-space DFS
Priority	A	Complexity	Exponential, $n \leq 17$

A. Đề bài tiếng Việt - đọc để hiểu đề

Một cây chứa các node là cừu hoặc sói. Bắt đầu từ node 0, khi đi tới node mới sẽ thu con vật tại đó; luôn phải duy trì số cừu đã thu > số sói, nếu không bị sói ăn hết. Từ các node đã thăm có thể chọn bất kỳ node con đang 'mở khóa' để đi tiếp, không nhất thiết theo một đường duy nhất. Hãy tìm số cừu tối đa có thể thu.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: Di chuyển trong tree nhưng các node khả dụng tích lũy; sheep>wolf luôn
- **Pattern**: State-space DFS
- **Vì sao**: DFS(sheep,wolf,availableNodes); chọn 1 available, thêm children, recurse

C. Tư duy lời giải từng bước

127. DFS(sheep,wolf,availableNodes).
128. chọn 1 available, thêm children, recurse.

Invariant / state phải giữ

Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: DFS(sheep,wolf,availableNodes); chọn 1 available, thêm children, recurse

D. JavaScript solution() - block code để học/copy

```
function solution(info, edges) {
  const child=Array.from( {
    length:info.length
  }, ()=>[]);
  for(const [p, c] of edges) child[p].push(c);
  let best=0;
  function dfs(sheep, wolf, available) {
    best=Math.max(best, sheep);
    for(let i=0; i<available.length; i++) {
      const node=available[i], ns=sheep+(info[node]===0), nw=wolf+(info[node]===1);
      if(nw>=ns) continue;
      const next=available.slice(0, i).concat(available.slice(i+1), child[node]);
      dfs(ns, nw, next);
    }
  }
  dfs(1, 0, [...child[0]]);
}
```

```
return best;  
}
```

E. Đọc code theo cấu trúc

- DFS tương ứng với một chuỗi lựa chọn; trước khi gọi sâu hơn phải cập nhật state, và sau khi quay về phải backtrack nếu state dùng chung.
- Biến/điều kiện quan trọng nhất của bài: Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: DFS(sheep,wolf,availableNodes); chọn 1 available, thêm children, recurse

F. Complexity + hidden-test traps

- **Độ phức tạp:** Exponential, $n \leq 17$
- **Bẫy dễ sai:** Không phải DFS tree thường; visited path state khác nhau
- **Recall 30 giây:** State-space DFS $\rightarrow$ DFS(sheep,wolf,availableNodes); chọn 1 available, thêm children, recurse

47. 여행경로

Lộ trình du lịch

Chủ đề	DFS/Backtracking	Pattern	Euler-like backtracking lexical
Priority	B	Complexity	O(E! worst)

A. Đề bài tiếng Việt - đọc để hiểu đề

Cho danh sách vé [from,to], phải dùng tất cả vé đúng một lần, bắt đầu từ ICN. Nếu có nhiều hành trình hợp lệ, chọn chuỗi sân bay nhỏ nhất theo thứ tự từ điển. Hãy trả về toàn bộ lộ trình. Có thể sort adjacency rồi DFS/backtracking hoặc dùng Euler path (Hierholzer) tùy cách triển khai.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: Dùng hết ticket từ ICN, lexical smallest route
- **Pattern**: Euler-like backtracking lexical
- **Vì sao**: Sort tickets lexical; DFS with used[]; stop khi route length=tickets+1

C. Tư duy lời giải từng bước

129. Sort tickets lexical.
130. DFS with used[].
131. stop khi route length=tickets+1.

Invariant / state phải giữ

Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mẫu chốt là: Sort tickets lexical; DFS with used[]; stop khi route length=tickets+1

D. JavaScript solution() - block code để học/copy

```
function solution(tickets) {
  tickets.sort((a, b) => a[1].localeCompare(b[1]));
  const used = Array(tickets.length).fill(false), path = ['ICN'];
  function dfs(cur, count) {
    if(count === tickets.length) return true;
    for(let i = 0; i < tickets.length; i++) {
      if(!used[i] && tickets[i][0] === cur) {
        used[i] = true;
        path.push(tickets[i][1]);
        if(dfs(tickets[i][1], count+1)) return true;
        path.pop();
        used[i] = false;
      }
    }
    return false;
  }
  dfs('ICN', 0);
}
```

```
return path;
}
```

E. Đọc code theo cấu trúc

- Phần sort chuẩn hóa thứ tự dữ liệu trước khi áp dụng greedy/two pointers/binary search hoặc comparator của bài.
- DFS tương ứng với một chuỗi lựa chọn; trước khi gọi sâu hơn phải cập nhật state, và sau khi quay về phải backtrack nếu state dùng chung.
- Biến/điều kiện quan trọng nhất của bài: Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: Sort tickets lexical; DFS with used[]; stop khi route length=tickets+1

F. Complexity + hidden-test traps

- **Độ phức tạp:** $O(E!)$ worst)
- **Bẫy dễ sai:** Duplicate tickets cần index/used riêng
- **Recall 30 giây:** Euler-like backtracking lexical → Sort tickets lexical; DFS with used[]; stop khi route length=tickets+1

48. 불량 사용자

Người dùng xấu

Chủ đề	DFS/Backtracking	Pattern	Matching + set of sets
Priority	B	Complexity	Exponential small

A. Đề bài tiếng Việt - đọc để hiểu đề

Có danh sách `user_id` và các `pattern banned_id`; trong `pattern`, '*' khớp với đúng một ký tự bất kỳ. Cần chọn một user khác nhau cho mỗi `banned pattern` sao cho tất cả khớp; thứ tự gán không làm hai tập kết quả khác nhau nếu tập user giống nhau. Hãy đếm số tập user duy nhất có thể tạo. DFS gán từng `pattern` và lưu tập kết quả dưới dạng canonical key.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: Mỗi `banned pattern` match 1 user, đếm tập user unique
- **Pattern**: Matching + set of sets
- **Vì sao**: Precompute candidates; DFS per banned id; used users; canonical sorted set key

C. Tư duy lời giải từng bước

132. Precompute candidates.
133. DFS per banned id.
134. used users.
135. canonical sorted set key.

Invariant / state phải giữ

Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: Precompute candidates; DFS per banned id; used users; canonical sorted set key

D. JavaScript solution() - block code để học/copy

```
function solution(user_id, banned_id) {
  const match=(u, b)=>u.length===b.length&&[...u].every((ch, i)=>b[i]=== '*' | | b[i]===ch);
  const sets=new Set(), used=Array(user_id.length).fill(false);
  function dfs(i, pick) {
    if(i===banned_id.length) {
      sets.add([...pick].sort().join(' '));
      return;
    }
    for(let j=0; j<user_id.length; j++) if(!used[j]&&match(user_id[j], banned_id[i])) {
      used[j]=true;
      pick.push(user_id[j]);
      dfs(i+1, pick);
      pick.pop();
      used[j]=false;
    }
  }
}
```

```
dfs(0, []);  
return sets.size;  
}
```

E. Đọc code theo cấu trúc

- Phần sort chuẩn hóa thứ tự dữ liệu trước khi áp dụng greedy/two pointers/binary search hoặc comparator của bài.
- Set dùng để kiểm tra uniqueness/visited hoặc loại trùng mà không cần vòng lặp lồng nhau.
- DFS tương ứng với một chuỗi lựa chọn; trước khi gọi sâu hơn phải cập nhật state, và sau khi quay về phải backtrack nếu state dùng chung.
- Biến/điều kiện quan trọng nhất của bài: Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: Precompute candidates; DFS per banned id; used users; canonical sorted set key

F. Complexity + hidden-test traps

- **Độ phức tạp:** Exponential small
- **Bẫy dễ sai:** Đối thứ tự banned có thể tạo cùng tập -> dedupe final
- **Recall 30 giây:** Matching + set of sets → Precompute candidates; DFS per banned id; used users; canonical sorted set key

49. 게임 맵 최단거리

Khoảng cách ngắn nhất trên bản đồ game

★ **BẮT BUỘC** theo roadmap team kia

Chủ đề	BFS/Graph Traversal	Pattern	Grid BFS shortest path
Priority	A	Complexity	O(RC)

A. Đề bài tiếng Việt - đọc để hiểu đề

Bản đồ game là lưới 0/1, 1 đi được, 0 là tường. Xuất phát ở góc trên trái và cần tới góc dưới phải, mỗi bước đi 4 hướng. Hãy trả về số ô/độ dài đường đi ngắn nhất; nếu không tới được thì -1. Vì mỗi bước có cùng cost, BFS đảm bảo lần đầu tới đích là tối ưu.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: Unweighted grid, đi 4 hướng, tìm shortest
- **Pattern**: Grid BFS shortest path
- **Vì sao**: Queue + head index; visited/dist; first reach target shortest

C. Tư duy lời giải từng bước

136. Queue + head index.
137. visited/dist.
138. first reach target shortest.

Invariant / state phải giữ

Khi một ô được lấy khỏi queue lần đầu, distance của nó là ngắn nhất vì BFS mở rộng theo từng lớp khoảng cách.

D. JavaScript solution() - block code để học/copy

```
function solution(maps) {
  const R=maps.length, C=maps[0].length, q=[[0, 0]], dist=Array.from( {
    length:R
  }, ()=>Array(C).fill(-1));
  let head=0;
  dist[0][0]=1;
  const d=[[1, 0], [-1, 0], [0, 1], [0, -1]];
  while(head<q.length) {
    const [r, c]=q[head++];
    if(r===R-1&& c===C-1)return dist[r][c];
    for(const [dr, dc] of d) {
      const nr=r+dr, nc=c+dc;
      if(nr<0 | nr>=R | nc<0 | nc>=C | maps[nr][nc]===0 | dist[nr][nc]!==-1)continue;
      dist[nr][nc]=dist[r][c]+1;
      q.push([nr, nc]);
    }
  }
}
```



```
}  
return -1;  
}
```

E. Đọc code theo cấu trúc

- Vòng while là phần điều chỉnh state cho tới khi invariant trở lại đúng; khi học, cần nói được chính xác điều kiện while có nghĩa gì.
- BFS dùng queue; tránh Array.shift() trên dữ liệu lớn, ưu tiên head index để dequeue $O(1)$.
- Biến/điều kiện quan trọng nhất của bài: Khi một ô được lấy khỏi queue lần đầu, distance của nó là ngắn nhất vì BFS mở rộng theo từng lớp khoảng cách.

F. Complexity + hidden-test traps

- **Độ phức tạp:** $O(RC)$
- **Bẫy dễ sai:** Mark visited khi enqueue, không phải dequeue
- **Recall 30 giây:** Grid BFS shortest path $\rightarrow$ Queue + head index; visited/dist; first reach target shortest

50. 리코쳅 로봇

Robot Ricochet

★ **BẮT BUỘC** theo roadmap team kia

Chủ đề	BFS/Graph Traversal	Pattern	BFS with slide transition
Priority	A	Complexity	$O(RC \cdot \max(R, C))$

A. Đề bài tiếng Việt - đọc để hiểu đề

Một robot trượt trên lưới theo 4 hướng. Khi chọn hướng, nó tiếp tục trượt cho tới ngay trước tường hoặc biên mới dừng, không thể dừng giữa đường. Cho vị trí bắt đầu R và đích G, hãy tìm số lần trượt ít nhất để tới đích, hoặc -1 nếu không thể. Mỗi vị trí dừng là một node; BFS trên các vị trí dừng.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: Mỗi move trượt tới trước tường/biên, tìm min moves
- **Pattern**: BFS with slide transition
- **Vì sao**: BFS states cells; neighbor = slide(dir) until blocked; enqueue landing cell

C. Tư duy lời giải từng bước

139. BFS states cells.
140. neighbor = slide(dir) until blocked.
141. enqueue landing cell.

Invariant / state phải giữ

Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: BFS states cells; neighbor = slide(dir) until blocked; enqueue landing cell

D. JavaScript solution() - block code để học/copy

```
function solution(board) {
  const R=board.length, C=board[0].length, d=[[1, 0], [-1, 0], [0, 1], [0, -1]];
  let sr, sc;
  for(let r=0; r<R; r++)for(let c=0; c<C; c++)if(board[r][c]=== 'R')[sr, sc]=[r, c];
  const dist=Array.from( {
    length:R
  }, ()=>Array(C).fill(-1)), q=[[sr, sc]];
  let head=0;
  dist[sr][sc]=0;
  while(head<q.length) {
    const [r, c]=q[head++];
    if(board[r][c]=== 'G')return dist[r][c];
    for(const [dr, dc] of d) {
      let nr=r, nc=c;
      while(1) {
```

```

const tr=nr+dr, tc=nc+dc;
if(tr<0 || tr>=R || tc<0 || tc>=C || board[tr][tc]==='D')break;
nr=tr;
nc=tc;
}
if(dist[nr][nc]===-1) {
  dist[nr][nc]=dist[r][c]+1;
  q.push([nr, nc]);
}
}
}
return -1;
}

```

E. Đọc code theo cấu trúc

- Vòng while là phần điều chỉnh state cho tới khi invariant trở lại đúng; khi học, cần nói được chính xác điều kiện while có nghĩa gì.
- BFS dùng queue; tránh Array.shift() trên dữ liệu lớn, ưu tiên head index để dequeue $O(1)$.
- Biến/điều kiện quan trọng nhất của bài: Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: BFS states cells; neighbor = slide(dir) until blocked; enqueue landing cell

F. Complexity + hidden-test traps

- **Độ phức tạp:** $O(RC \cdot \max(R, C))$
- **Bẫy dễ sai:** Không dừng ở giữa; tránh self-loop
- **Recall 30 giây:** BFS with slide transition $\rightarrow$ BFS states cells; neighbor = slide(dir) until blocked; enqueue landing cell

51. 미로 탈출

Thoát mê cung

Chủ đề	BFS/Graph Traversal	Pattern	Two-stage BFS
Priority	B	Complexity	O(RC)

A. Đề bài tiếng Việt - đọc để hiểu đề

Mê cung chứa S (start), L (lever), E (exit), X (tường). Bạn bắt buộc phải đi từ S tới L để gạt cần trước rồi mới tới E. Mỗi bước 4 hướng và cost 1. Hãy trả về tổng khoảng cách ngắn nhất S->L + L->E hoặc -1 nếu một chặng không tồn tại.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: Start -> Lever -> Exit, walls
- **Pattern**: Two-stage BFS
- **Vì sao**: BFS S->L rồi L->E, cộng dist; thiếu đoạn nào -1

C. Tư duy lời giải từng bước

142. BFS S->L rồi L->E, cộng dist.

143. thiếu đoạn nào -1.

Invariant / state phải giữ

Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: BFS S->L rồi L->E, cộng dist; thiếu đoạn nào -1

D. JavaScript solution() - block code để học/copy

```
function solution(maps) {
  const R=maps.length, C=maps[0].length, d=[[1, 0], [-1, 0], [0, 1], [0, -1]];
  let S, L, E;
  for(let r=0; r<R; r++)for(let c=0; c<C; c++) {
    if(maps[r][c]=== 'S')S=[r, c];
    if(maps[r][c]=== 'L')L=[r, c];
    if(maps[r][c]=== 'E')E=[r, c];
  }
  const bfs=(st, goal)=> {
    const q=[st], dist=Array.from( {
      length:R
    }, ()=>Array(C).fill(-1));
    let h=0;
    dist[st[0]][st[1]]=0;
    while(h<q.length) {
      const [r, c]=q[h++];
      if(r===goal[0]&&c===goal[1])return dist[r][c];
      for(const [dr, dc] of d) {
        const nr=r+dr, nc=c+dc;
```

```

    if(nr<0 | nr>=R | nc<0 | nc>=C | maps[nr][nc]==='X' | dist[nr][nc]!==-1)continue;
    dist[nr][nc]=dist[r][c]+1;
    q.push([nr, nc]);
  }
}
return -1;
}
;
const a=bfs(S, L);
if(a<0)return -1;
const b=bfs(L, E);
return b<0?-1:a+b;
}

```

E. Đọc code theo cấu trúc

- Vòng while là phần điều chỉnh state cho tới khi invariant trở lại đúng; khi học, cần nói được chính xác điều kiện while có nghĩa gì.
- BFS dùng queue; tránh Array.shift() trên dữ liệu lớn, ưu tiên head index để dequeue $O(1)$.
- Biến/điều kiện quan trọng nhất của bài: Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: BFS S->L rồi L->E, cộng dist; thiếu đoạn nào -1

F. Complexity + hidden-test traps

- **Độ phức tạp:** $O(RC)$
- **Bẫy dễ sai:** Không thể đi thẳng E nếu chưa lever
- **Recall 30 giây:** Two-stage BFS → BFS S->L rồi L->E, cộng dist; thiếu đoạn nào -1

52. 거리두기 확인하기

Kiểm tra giãn cách

Chủ đề	BFS/Graph Traversal	Pattern	Local BFS / Manhattan
Priority	B	Complexity	$O(5 \times 25)$

A. Đề bài tiếng Việt - đọc để hiểu đề

Mỗi phòng chờ 5x5 có P (người), O (trống), X (vách). Quy tắc giãn cách yêu cầu mọi cặp người có Manhattan distance ≤ 2 phải được ngăn cách phù hợp bởi vách. Với mỗi phòng, trả 1 nếu hợp lệ, 0 nếu vi phạm. Có thể BFS từ từng P trong phạm vi 2 hoặc kiểm tra một số pattern cục bộ.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: Mỗi P không được có P khác trong distance ≤ 2 nếu không bị partition chắn
- **Pattern**: Local BFS / Manhattan
- **Vì sao**: Từ mỗi P BFS depth ≤ 2 qua non-X; thấy P khác $\rightarrow$ invalid

C. Tư duy lời giải từng bước

144. Từ mỗi P BFS depth ≤ 2 qua non-X.

145. thấy P khác $\rightarrow$ invalid.

Invariant / state phải giữ

Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: Từ mỗi P BFS depth ≤ 2 qua non-X; thấy P khác $\rightarrow$ invalid

D. JavaScript solution() - block code để học/copy

```
function solution(places) {
  const d=[[1, 0], [-1, 0], [0, 1], [0, -1]];
  const ok=(g)=> {
    for(let sr=0; sr<5; sr++)for(let sc=0; sc<5; sc++)if(g[sr][sc]==='P') {
      const q=[[sr, sc, 0]], seen=Array.from( {
        length:5
      }, ()=>Array(5).fill(false));
      let h=0;
      seen[sr][sc]=true;
      while(h<q.length) {
        const [r, c, dist]=q[h++];
        if(dist===2)continue;
        for(const [dr, dc] of d) {
          const nr=r+dr, nc=c+dc;
          if(nr<0 || nr>=5 || nc<0 || nc>=5 || seen[nr][nc] || g[nr][nc]==='X')continue;
          if(g[nr][nc]==='P')return false;
          seen[nr][nc]=true;
          q.push([nr, nc, dist+1]);
        }
      }
    }
  }
  return places.every(g=>ok(g));
}
```

```
    }  
  }  
  return true;  
}  
;  
return places.map(p=>ok(p)?1:0);  
}
```

E. Đọc code theo cấu trúc

- Vòng while là phần điều chỉnh state cho tới khi invariant trở lại đúng; khi học, cần nói được chính xác điều kiện while có nghĩa gì.
- BFS dùng queue; tránh Array.shift() trên dữ liệu lớn, ưu tiên head index để dequeue $O(1)$.
- Biến/điều kiện quan trọng nhất của bài: Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: Từ mỗi P BFS depth ≤ 2 qua non-X; thấy P khác $\rightarrow$ invalid

F. Complexity + hidden-test traps

- **Độ phức tạp:** $O(5 \cdot 25)$
- **Bẫy dễ sai:** Diagonal và distance2 cần xét partition đúng
- **Recall 30 giây:** Local BFS / Manhattan $\rightarrow$ Từ mỗi P BFS depth ≤ 2 qua non-X; thấy P khác $\rightarrow$ invalid

53. 무인도 여행

Du lịch đảo hoang

Chủ đề	BFS/Graph Traversal	Pattern	Connected components
Priority	B	Complexity	O(RC)

A. Đề bài tiếng Việt - đọc để hiểu đề

Một bản đồ gồm số và X. Các ô số nối 4 hướng tạo thành từng hòn đảo; giá trị số là số ngày lương thực tại ô. Với mỗi đảo, tính tổng lương thực bằng BFS/DFS. Trả về các tổng tăng dần; nếu không có đảo nào thì [-1].

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: Grid digits + X; sum mỗi island
- **Pattern**: Connected components
- **Vì sao**: For each unvisited non-X BFS/DFS accumulate; sort sums

C. Tư duy lời giải từng bước

146. For each unvisited non-X BFS/DFS accumulate.

147. sort sums.

Invariant / state phải giữ

Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: For each unvisited non-X BFS/DFS accumulate; sort sums

D. JavaScript solution() - block code để học/copy

```
function solution(maps) {
  const R=maps.length, C=maps[0].length, seen=Array.from( {
    length:R
  }, ()=>Array(C).fill(false)), d=[[1, 0], [-1, 0], [0, 1], [0, -1]], ans=[];
  for(let sr=0; sr<R; sr++)for(let sc=0; sc<C; sc++)if(!seen[sr][sc]&&maps[sr][sc]!='X') {
    let sum=0, q=[[sr, sc]], h=0;
    seen[sr][sc]=true;
    while(h<q.length) {
      const [r, c]=q[h++];
      sum+=Number(maps[r][c]);
      for(const [dr, dc] of d) {
        const nr=r+dr, nc=c+dc;
        if(nr<0 | nr>=R | nc<0 | nc>=C | seen[nr][nc] | maps[nr][nc]=='X')continue;
        seen[nr][nc]=true;
        q.push([nr, nc]);
      }
    }
    ans.push(sum);
  }
  return ans.length?ans.sort((a, b)=>a-b):[-1];
}
```



```
}
```

E. Đọc code theo cấu trúc

- Phần sort chuẩn hóa thứ tự dữ liệu trước khi áp dụng greedy/two pointers/binary search hoặc comparator của bài.
- Vòng while là phần điều chỉnh state cho tới khi invariant trở lại đúng; khi học, cần nói được chính xác điều kiện while có nghĩa gì.
- BFS dùng queue; tránh Array.shift() trên dữ liệu lớn, ưu tiên head index để dequeue $O(1)$.
- Biến/điều kiện quan trọng nhất của bài: Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mẫu chốt là: For each unvisited non-X BFS/DFS accumulate; sort sums

F. Complexity + hidden-test traps

- **Độ phức tạp:** $O(RC)$
- **Bẫy dễ sai:** char digit parse; nếu none [-1]
- **Recall 30 giây:** Connected components → For each unvisited non-X BFS/DFS accumulate; sort sums

54. 단어 변환

Biến đổi từ

★ **BẮT BUỘC** theo roadmap team kia

Chủ đề	BFS/Graph Traversal	Pattern	Implicit graph BFS
Priority	A	Complexity	$O(N^2 * L)$

A. Đề bài tiếng Việt - đọc để hiểu đề

Cho begin, target và danh sách words. Mỗi bước có thể đổi đúng một ký tự để chuyển từ từ hiện tại sang một từ trong words, và mỗi bước cost 1. Hãy tìm số bước ít nhất để tới target, hoặc 0 nếu không thể. Xem mỗi từ là node, nối cạnh nếu khác đúng một ký tự; BFS cho shortest path.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: Mỗi bước đổi đúng 1 ký tự, shortest tới target
- **Pattern**: Implicit graph BFS
- **Vì sao**: BFS words; edge if diffCount==1; visited

C. Tư duy lời giải từng bước

148. BFS words.
149. edge if diffCount==1.
150. visited.

Invariant / state phải giữ

Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: BFS words; edge if diffCount==1; visited

D. JavaScript solution() - block code để học/copy

```
function solution(begin, target, words) {
  const idx=words.indexOf(target);
  if(idx<0)return 0;
  const q=[[begin, 0]], seen=new Set([begin]);
  let h=0;
  const one=(a, b)=> {
    let d=0;
    for(let i=0; i<a.length; i++)if(a[i]!=b[i]&&++d>1)return false;
    return d===1;
  }
  ;
  while(h<q.length) {
    const [w, dist]=q[h++];
    if(w===target)return dist;
    for(const x of words)if(!seen.has(x)&&one(w, x)) {
      seen.add(x);
    }
  }
}
```

```
    q.push([x, dist+1]);
  }
}
return 0;
}
```

E. Đọc code theo cấu trúc

- Set dùng để kiểm tra uniqueness/visited hoặc loại trùng mà không cần vòng lặp lồng nhau.
- Vòng while là phần điều chỉnh state cho tới khi invariant trở lại đúng; khi học, cần nói được chính xác điều kiện while có nghĩa gì.
- BFS dùng queue; tránh Array.shift() trên dữ liệu lớn, ưu tiên head index để dequeue $O(1)$.
- Biến/điều kiện quan trọng nhất của bài: Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: BFS words; edge if diffCount==1; visited

F. Complexity + hidden-test traps

- **Độ phức tạp:** $O(N^2 * L)$
- **Bẫy dễ sai:** Nếu target không trong words $\rightarrow 0$
- **Recall 30 giây:** Implicit graph BFS $\rightarrow$ BFS words; edge if diffCount==1; visited

55. 블록 이동하기

Di chuyển khối

★ **BẮT BUỘC** theo roadmap team kia

Chủ đề	BFS/Graph Traversal	Pattern	BFS composite state
Priority	A	Complexity	$O(N^2)$

A. Đề bài tiếng Việt - đọc để hiểu đề

Robot là một khối domino 1×2 nằm trên lưới có tường. Nó có thể tịnh tiến 4 hướng nếu cả hai ô mới trống, hoặc xoay 90 độ quanh một đầu nếu các ô cần thiết không bị chặn. Bắt đầu ở hai ô đầu hàng trên và cần để ít nhất một phân robot tới ô đích. Hãy tìm số move tối thiểu bằng BFS trên state gồm hai tọa độ đã chuẩn hóa.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: Robot 2 ô, move/rotate, shortest
- **Pattern**: BFS composite state
- **Vì sao**: Canonical state = sorted 2 cells; generate 4 translations + rotations khi 2 ô bên cạnh trống

C. Tư duy lời giải từng bước

151. Canonical state = sorted 2 cells.
152. generate 4 translations + rotations khi 2 ô bên cạnh trống.

Invariant / state phải giữ

Visited phải gắn với cả hai ô của robot sau khi chuẩn hóa thứ tự; cùng hai ô là cùng một state dù sinh ra theo cách khác.

D. JavaScript solution() - block code để học/copy

```
function solution(board) {
  const n=board.length, dirs=[[1, 0], [-1, 0], [0, 1], [0, -1]];
  const norm=(a, b)=> {
    const p=[a, b].sort((x, y)=>x[0]-y[0] || x[1]-y[1]);
    return p;
  };
  ;
  const key=(a, b)=> {
    const p=norm(a, b);
    return `${p[0]}|${p[1]}`;
  };
  ;
  const q=[[[0, 0], [0, 1], 0]], seen=new Set([key([0, 0], [0, 1])]);
  let h=0;
  while(h<q.length) {
    const [a, b, dist]=q[h++];
    if((a[0]===n-1&&a[1]===n-1) || (b[0]===n-1&&b[1]===n-1))return dist;
```

```

const next=[];
for(const [dr, dc] of dirs) {
  const na=[a[0]+dr, a[1]+dc], nb=[b[0]+dr, b[1]+dc];
  if(na[0]>=0&&na[0]<n&&na[1]>=0&&na[1]<n&&nb[0]>=0&&nb[0]<n&&nb[1]>=0&&nb[1]<n&&board[na[0]]
[na[1]]===0&&board[nb[0]][nb[1]]===0)next.push([na,
  nb]);
}
if(a[0]===b[0]) {
  for(const dr of [-1, 1]) if(a[0]+dr>=0&&a[0]+dr<n&&board[a[0]+dr][a[1]]===0&&board[b[0]+dr][b[1]]===0) {
    next.push([a, [a[0]+dr, a[1]]]);
    next.push([b, [b[0]+dr, b[1]]]);
  }
} else {
  for(const dc of [-1, 1]) if(a[1]+dc>=0&&a[1]+dc<n&&board[a[0]][a[1]+dc]===0&&board[b[0]][b[1]+dc]===0) {
    next.push([a, [a[0], a[1]+dc]]);
    next.push([b, [b[0], b[1]+dc]]);
  }
}
for(const [x, y] of next) {
  const k=key(x, y);
  if(!seen.has(k)) {
    seen.add(k);
    q.push([x, y, dist+1]);
  }
}
}
}

```

E. Đọc code theo cấu trúc

- Phần sort chuẩn hóa thứ tự dữ liệu trước khi áp dụng greedy/two pointers/binary search hoặc comparator của bài.
- Set dùng để kiểm tra uniqueness/visited hoặc loại trùng mà không cần vòng lặp lồng nhau.
- Vòng while là phần điều chỉnh state cho tới khi invariant trở lại đúng; khi học, cần nói được chính xác điều kiện while có nghĩa gì.
- BFS dùng queue; tránh Array.shift() trên dữ liệu lớn, ưu tiên head index để dequeue $O(1)$.
- Biến/điều kiện quan trọng nhất của bài: Visited phải gắn với cả hai ô của robot sau khi chuẩn hóa thứ tự; cùng hai ô là cùng một state dù sinh ra theo cách khác.

F. Complexity + hidden-test traps

- **Độ phức tạp:** $O(N^2)$
- **Bẫy dễ sai:** Dedupe orientation/state; padding board giúp boundary
- **Recall 30 giây:** BFS composite state $\rightarrow$ Canonical state = sorted 2 cells; generate 4 translations + rotations khi 2 ô bên cạnh trống

56. 경주로 건설

Xây đường đua

★ **BẮT BUỘC** theo roadmap team kia

Chủ đề	BFS/Graph Traversal	Pattern	Shortest path with direction state
Priority	A	Complexity	$O(N^2 \cdot 4)$

A. Đề bài tiếng Việt - đọc để hiểu đề

Trên lưới, cần xây đường đua từ góc trên trái tới góc dưới phải. Đi thẳng thêm một ô tốn chi phí cơ bản, còn đổi hướng tạo góc cua tốn thêm chi phí lớn. Vì chi phí tới cùng một ô phụ thuộc hướng đi vào, state phải gồm (r, c, dir) , không thể chỉ lưu dist theo ô. Hãy tìm tổng chi phí nhỏ nhất bằng Dijkstra hoặc BFS có relaxation theo cost.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: Cost 100 thẳng, +500 corner
- **Pattern**: Shortest path with direction state
- **Vì sao**: State= (r, c, dir) ; dist 3D; relax next cost; queue/Dijkstra-like

C. Tư duy lời giải từng bước

- 153. State= (r, c, dir) .
- 154. dist 3D.
- 155. relax next cost.
- 156. queue/Dijkstra-like.

Invariant / state phải giữ

$dist[r][c][dir]$ là chi phí nhỏ nhất đã biết để tới ô (r, c) với hướng đi cuối là dir ; không được gộp các dir .

D. JavaScript solution() - block code để học/copy

```
function solution(board) {
  const n=board.length, dirs=[[1, 0], [-1, 0], [0, 1], [0, -1]], INF=1e15;
  const dist=Array.from( {
    length:n
  }, ()=>Array.from( {
    length:n
  }, ()=>Array(4).fill(INF))), q=[];
  for(let d=0; d<4; d++) dist[0][0][d]=0;
  q.push([0, 0, -1, 0]);
  let head=0, ans=INF;
  while(head<q.length) {
    const [r, c, pd, cost]=q[head++];
    if(cost>ans)continue;
```

```

if(r===n-1&&c===n-1) {
  ans=Math.min(ans, cost);
  continue;
}
for(let nd=0; nd<4; nd++) {
  const nr=r+dirs[nd][0], nc=c+dirs[nd][1];
  if(nr<0 | nr>=n | nc<0 | nc>=n | board[nr][nc])continue;
  const ncst=cost+(pd===-1 | pd===nd?100:600);
  if(ncst<dist[nr][nc][nd]) {
    dist[nr][nc][nd]=ncst;
    q.push([nr, nc, nd, ncst]);
  }
}
}
return ans;
}

```

E. Đọc code theo cấu trúc

- Vòng while là phần điều chỉnh state cho tới khi invariant trở lại đúng; khi học, cần nói được chính xác điều kiện while có nghĩa gì.
- BFS dùng queue; tránh Array.shift() trên dữ liệu lớn, ưu tiên head index để dequeue $O(1)$.
- Biến/điều kiện quan trọng nhất của bài: $dist[r][c][dir]$ là chi phí nhỏ nhất đã biết để tới ô (r,c) với hướng đi cuối là dir ; không được gộp các dir .

F. Complexity + hidden-test traps

- **Độ phức tạp:** $O(N^2 \cdot 4)$
- **Bẫy dễ sai:** Visited chỉ theo cell là sai vì hướng ảnh hưởng tương lai
- **Recall 30 giây:** Shortest path with direction state $\rightarrow$ State= (r,c,dir) ; dist 3D; relax next cost; queue/Dijkstra-like

57. 퍼즐 조각 채우기

Lắp mảnh ghép puzzle

★ **BẮT BUỘC** theo roadmap team kia

Chủ đề	BFS/Graph Traversal	Pattern	Components + normalization + rotation
Priority	A	Complexity	$O(N^2 * \text{shape rotations})$

A. Đề bài tiếng Việt - đọc để hiểu đề

game_board có các ô trống tạo thành các vùng puzzle cần lắp; table có các mảnh ghép tạo thành các component. Có thể xoay mảnh 0/90/180/270 độ nhưng không lật, mỗi mảnh dùng một lần và phải khớp chính xác vùng trống. Hãy tách component bằng BFS, chuẩn hóa tọa độ, sinh các rotation canonical rồi ghép để tối đa số ô được lắp.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: Match shapes từ table vào holes game_board
- **Pattern**: Components + normalization + rotation
- **Vì sao**: Extract components; normalize coords to origin; generate 4 rotations normalized; multiset match by shape

C. Tư duy lời giải từng bước

157. Extract components.
158. normalize coords to origin.
159. generate 4 rotations normalized.
160. multiset match by shape.

Invariant / state phải giữ

Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: Extract components; normalize coords to origin; generate 4 rotations normalized; multiset match by shape

D. JavaScript solution() - block code để học/copy

```
function solution(game_board, table) {
  const n=table.length, d=[[1, 0], [-1, 0], [0, 1], [0, -1]];
  const extract=(g, target)=> {
    const seen=Array.from( {
      length:n
    }, ()=>Array(n).fill(false)), comps=[];
    for(let sr=0; sr<n; sr++)for(let sc=0; sc<n; sc++)if(!seen[sr][sc]&&g[sr][sc]==target) {
      const q=[[sr, sc]], pts=[], h=0;
      seen[sr][sc]=true;
      while(h<q.length) {
        const [r, c]=q[h++];
```



```

pts.push([r, c]);
for(const [dr, dc] of d) {
  const nr=r+dr, nc=c+dc;
  if(nr<0 || nr>=n || nc<0 || nc>=n || seen[nr][nc] || g[nr][nc]!==target)continue;
  seen[nr][nc]=true;
  q.push([nr, nc]);
}
}
comps.push(pts);
}
return comps;
}
;
const norm=(pts)=> {
  const mr=Math.min(...pts.map(p=>p[0])), mc=Math.min(...pts.map(p=>p[1]));
  return pts.map([r, c])=>[r-mr, c-mc]).sort((a, b)=>a[0]-b[0] || a[1]-b[1]);
}
;
const rot=pts=>norm(pts.map([r, c])=>[c, -r]));
const str=pts=>norm(pts).map(p=>p.join(',')).join(';');
const holes=extract(game_board, 0), pieces=extract(table, 1), used=Array(pieces.length).fill(false);
let ans=0;
for(const h of holes) {
  const hs=str(h);
  outer:for(let i=0; i<pieces.length; i++)if(!used[i]&&pieces[i].length===h.length) {
    let p=pieces[i];
    for(let k=0; k<4; k++) {
      if(str(p)===hs) {
        used[i]=true;
        ans+=h.length;
        break outer;
      }
      p=rot(p);
    }
  }
}
return ans;
}

```

E. Đọc code theo cấu trúc

- Phần sort chuẩn hóa thứ tự dữ liệu trước khi áp dụng greedy/two pointers/binary search hoặc comparator của bài.
- Vòng while là phần điều chỉnh state cho tới khi invariant trở lại đúng; khi học, cần nói được chính xác điều kiện while có nghĩa gì.
- BFS dùng queue; tránh Array.shift() trên dữ liệu lớn, ưu tiên head index để dequeue $O(1)$.
- Biến/điều kiện quan trọng nhất của bài: Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: Extract components; normalize coords to origin; generate 4 rotations normalized; multiset match by shape

F. Complexity + hidden-test traps

- **Độ phức tạp:** $O(N^2 * \text{shape rotations})$
- **Bẫy dễ sai:** Sort coords canonical; mỗi piece dùng 1 lần

- **Recall 30 giây:** Components + normalization + rotation → Extract components; normalize coords to origin; generate 4 rotations normalized; multiset match by shape

58. 네트워크

Network

Chủ đề	BFS/Graph Traversal	Pattern	Connected components graph
Priority	B	Complexity	$O(n^2)$

A. Đề bài tiếng Việt - đọc để hiểu đề

Cho ma trận computers biểu diễn kết nối trực tiếp giữa các máy. Hai máy thuộc cùng network nếu có đường nối trực tiếp hoặc gián tiếp. Hãy đếm số thành phần liên thông. Chỉ cần DFS/BFS từ mỗi máy chưa thăm và tăng bộ đếm mỗi lần bắt đầu một traversal mới.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: Đếm số component trong adjacency matrix
- **Pattern**: Connected components graph
- **Vì sao**: For i unvisited DFS/BFS, count++

C. Tư duy lời giải từng bước

161. For i unvisited DFS/BFS, count++.

162. Duy trì state/invariant bên dưới và cập nhật đáp án khi điều kiện của bài được thỏa..

Invariant / state phải giữ

Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: For i unvisited DFS/BFS, count++

D. JavaScript solution() - block code để học/copy

```
function solution(n, computers) {
  const seen=Array(n).fill(false);
  let ans=0;
  function dfs(u) {
    seen[u]=true;
    for(let v=0; v<n; v++)if(computers[u][v]&&!seen[v])dfs(v);
  }
  for(let i=0; i<n; i++)if(!seen[i]) {
    ans++;
    dfs(i);
  }
  return ans;
}
```

E. Đọc code theo cấu trúc

- DFS tương ứng với một chuỗi lựa chọn; trước khi gọi sâu hơn phải cập nhật state, và sau khi quay về phải backtrack nếu state dùng chung.

- Biến/điều kiện quan trọng nhất của bài: Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mẫu chốt là: For i unvisited DFS/BFS, count++

F. Complexity + hidden-test traps

- **Độ phức tạp:** $O(n^2)$
- **Bẫy dễ sai:** Self edge irrelevant
- **Recall 30 giây:** Connected components graph → For i unvisited DFS/BFS, count++

59. 표 병합

Gộp bảng

Chủ đề	Tree/Union-Find	Pattern	Union-find + value semantics
Priority	C	Complexity	$O(\text{commands} \times 2500)$

A. Đề bài tiếng Việt - đọc để hiểu đề

Có bảng 50x50, hỗ trợ UPDATE một ô, UPDATE toàn bộ ô có value1 thành value2, MERGE hai ô để chúng thuộc cùng nhóm và chia sẻ một giá trị, UNMERGE một ô để tách toàn bộ nhóm nhưng chỉ ô được chỉ định giữ giá trị cũ, và PRINT. Hãy mô phỏng chính xác quan hệ nhóm và giá trị. Union-Find hoặc quản lý group id đều được, nhưng UNMERGE là phần khó nhất vì phải tách toàn bộ thành phần.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: UPDATE/MERGE/UNMERGE/PRINT spreadsheet cells
- **Pattern**: Union-find + value semantics
- **Vì sao**: DSU parent for groups + value on root; MERGE roots; UNMERGE tách mọi member giữ value ở selected cell

C. Tư duy lời giải từng bước

163. DSU parent for groups + value on root.
164. MERGE roots.
165. UNMERGE tách mọi member giữ value ở selected cell.

Invariant / state phải giữ

Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: DSU parent for groups + value on root; MERGE roots; UNMERGE tách mọi member giữ value ở selected cell

D. JavaScript solution() - block code để học/copy

```
function solution(commands) {
  const N=2500, parent=Array.from( {
    length:N
  }, (_, i)=>i), value=Array(N).fill('EMPTY');
  const id=(r, c)=>(r-1)*50+(c-1);
  const find=x=>parent[x]===x?(parent[x]=find(parent[x]));
  const ans=[];
  for(const cmd of commands) {
    const p=cmd.split(' ');
    if(p[0]=== 'UPDATE'&&p.length===4) {
      value[find(id(+p[1], +p[2]))]=p[3];
    } else if(p[0]=== 'UPDATE') {
      for(let i=0; i<N; i++)if(find(i)===i&&value[i]===p[1])value[i]=p[2];
    } else if(p[0]=== 'MERGE') {
```

```

let a=find(id(+p[1], +p[2])), b=find(id(+p[3], +p[4]));
if(a!==b) {
  const v=value[a]!=='EMPTY'?value[a]:value[b];
  parent[b]=a;
  value[a]=v;
  value[b]='EMPTY';
}
} else if(p[0]=='UNMERGE') {
  const cell=id(+p[1], +p[2]), root=find(cell), keep=value[root], members=[];
  for(let i=0; i<N; i++)if(find(i)===root)members.push(i);
  for(const x of members) {
    parent[x]=x;
    value[x]='EMPTY';
  }
  value[cell]=keep;
} else ans.push(value[find(id(+p[1], +p[2]))]);
}
return ans;
}

```

E. Đọc code theo cấu trúc

- Đọc code thành ba phần: khởi tạo state → cập nhật state theo từng phần tử/lựa chọn → trả đáp án. Đừng học thuộc từng dòng rồi rạc.
- Biến/điều kiện quan trọng nhất của bài: Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: DSU parent for groups + value on root; MERGE roots; UNMERGE tách mọi member giữ value ở selected cell

F. Complexity + hidden-test traps

- **Độ phức tạp:** $O(\text{commands} * 2500)$
- **Bẫy dễ sai:** UNMERGE cần scan all 2500 cells; merge values rule
- **Recall 30 giây:** Union-find + value semantics → DSU parent for groups + value on root; MERGE roots; UNMERGE tách mọi member giữ value ở selected cell

60.길 찾기 게임

Game tìm đường

Chủ đề	Tree/Union-Find	Pattern	Build binary tree
Priority	C	Complexity	$O(n^2)$ simple

A. Đề bài tiếng Việt - đọc để hiểu đề

Cho các node với tọa độ (x,y) . Cây nhị phân được xây sao cho node có y cao hơn nằm trên, và trong mỗi subtree, x nhỏ hơn root đi trái, x lớn hơn đi phải. Cần dựng cây theo quy tắc đó rồi trả về thứ tự preorder và postorder của các node theo số thứ tự gốc.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: Node coordinates, root high y ; x determines left/right; return preorder/postorder
- **Pattern**: Build binary tree
- **Vì sao**: Sort y desc, x asc; BST insert by x ; traverse

C. Tư duy lời giải từng bước

166. Sort y desc, x asc.
167. BST insert by x .
168. traverse.

Invariant / state phải giữ

Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: Sort y desc, x asc; BST insert by x ; traverse

D. JavaScript solution() - block code để học/copy

```
function solution(nodeinfo) {
  const nodes=nodeinfo.map(([x, y], i)=>({
    x, y, id:i+1, left:null, right:null
  })).sort((a, b)=>b.y-a.y | a.x-b.x);
  const root=nodes[0];
  for(let i=1; i<nodes.length; i++) {
    let cur=root;
    while(1) {
      if(nodes[i].x<cur.x) {
        if(!cur.left) {
          cur.left=nodes[i];
          break;
        }
        cur=cur.left;
      } else {
        if(!cur.right) {
          cur.right=nodes[i];
          break;
        }
        cur=cur.right;
      }
    }
  }
}
```

```

    }
    cur=cur.right;
  }
}
}
const pre=[], post=[];
const dfs=n=> {
  if(!n)return;
  pre.push(n.id);
  dfs(n.left);
  dfs(n.right);
  post.push(n.id);
}
;
dfs(root);
return [pre, post];
}

```

E. Đọc code theo cấu trúc

- Phần sort chuẩn hóa thứ tự dữ liệu trước khi áp dụng greedy/two pointers/binary search hoặc comparator của bài.
- Vòng while là phần điều chỉnh state cho tới khi invariant trở lại đúng; khi học, cần nói được chính xác điều kiện while có nghĩa gì.
- DFS tương ứng với một chuỗi lựa chọn; trước khi gọi sâu hơn phải cập nhật state, và sau khi quay về phải backtrack nếu state dùng chung.
- Biến/điều kiện quan trọng nhất của bài: Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: Sort y desc,x asc; BST insert by x; traverse

F. Complexity + hidden-test traps

- **Độ phức tạp:** $O(n^2)$ simple
- **Bẫy dễ sai:** Recursion depth
- **Recall 30 giây:** Build binary tree → Sort y desc,x asc; BST insert by x; traverse

61. 다단계 칫솔 판매

Bán bàn chải đa cấp

Chủ đề	Tree/Union-Find	Pattern	Parent chain propagation
Priority	B	Complexity	O(total chain)

A. Đề bài tiếng Việt - đọc để hiểu đề

Một mô hình bán hàng đa cấp có quan hệ giới thiệu parent. Khi một người bán tạo lợi nhuận, 10% (lấy phần nguyên) được chuyển lên người giới thiệu, phần còn lại người bán giữ; quá trình tiếp tục lên trên cho tới center hoặc khi phần chuyển bằng 0. Với nhiều sale, hãy cộng dồn lợi nhuận cuối của từng người theo thứ tự enroll.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: 10% commission đẩy lên parent tới root hoặc amount 0
- **Pattern**: Parent chain propagation
- **Vì sao**: Map name->index,parent; mỗi sale while amount>0: give=floor(amount*0.1), keep=amount-give

C. Tư duy lời giải từng bước

169. Map name->index,parent.

170. mỗi sale while amount>0: give=floor(amount*0.1), keep=amount-give.

Invariant / state phải giữ

Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: Map name->index,parent; mỗi sale while amount>0: give=floor(amount*0.1), keep=amount-give

D. JavaScript solution() - block code để học/copy

```
function solution(enroll, referral, seller, amount) {
  const idx=new Map(enroll.map((x, i)=>[x, i])), parent=referral.map(x=>x==='-'?-1:idx.get(x)),
  ans=Array(enroll.length).fill(0);
  for(let i=0; i<seller.length; i++) {
    let cur=idx.get(seller[i]), money=amount[i]*100;
    while(cur!=-1 && money>0) {
      const up=Math.floor(money*0.1);
      ans[cur]+=money-up;
      money=up;
      cur=parent[cur];
    }
  }
  return ans;
}
```

E. Đọc code theo cấu trúc

- Map lưu tần suất / ánh xạ / trạng thái để truy cập O(1) trung bình thay vì quét lại dữ liệu.
- Vòng while là phần điều chỉnh state cho tới khi invariant trở lại đúng; khi học, cần nói được chính xác điều kiện while có nghĩa gì.

- Biến/điều kiện quan trọng nhất của bài: Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mẫu chốt là: Map name->index,parent; mỗi sale while amount>0: give=floor(amount*0.1), keep=amount-give

F. Complexity + hidden-test traps

- **Độ phức tạp:** $O(\text{total chain})$
- **Bẫy dễ sai:** Không dùng floating; parent "-"
- **Recall 30 giây:** Parent chain propagation → Map name->index,parent; mỗi sale while amount>0: give=floor(amount*0.1), keep=amount-give

62. 배달

Giao hàng

Chủ đề	Graph/Shortest Path	Pattern	Dijkstra
Priority	B	Complexity	$O((V+E)\log V)$

A. Đề bài tiếng Việt - đọc để hiểu đề

Có N làng và các road hai chiều có trọng số thời gian. Từ làng 1, thức ăn chỉ giao được tới làng có shortest distance $\leq K$. Hãy đếm số làng nhận được giao hàng. Dùng Dijkstra trên adjacency list; nếu N nhỏ cũng có thể dùng Floyd-Warshall.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: Từ 1 tới làng, đếm $\text{dist} \leq K$
- **Pattern**: Dijkstra
- **Vì sao**: Adj list weighted undirected; min-heap Dijkstra

C. Tư duy lời giải từng bước

171. Adj list weighted undirected.
172. min-heap Dijkstra.

Invariant / state phải giữ

Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: Adj list weighted undirected; min-heap Dijkstra

D. JavaScript solution() - block code để học/copy

```
function solution(N, road, K) {
  const adj=Array.from( {
    length:N+1
  }, ()=>[]);
  for(const [a, b, w] of road) {
    adj[a].push([b, w]);
    adj[b].push([a, w]);
  }
  const dist=Array(N+1).fill(Infinity);
  dist[1]=0;
  const q=[[0, 1]];
  while(q.length) {
    q.sort((a, b)=>b[0]-a[0]);
    const [d, u]=q.pop();
    if(d!==dist[u])continue;
    for(const [v, w] of adj[u])if(d+w<dist[v]) {
      dist[v]=d+w;
      q.push([dist[v], v]);
    }
  }
}
```

```
}  
return dist.slice(1).filter(x=>x<=K).length;  
}
```

E. Đọc code theo cấu trúc

- Phần sort chuẩn hóa thứ tự dữ liệu trước khi áp dụng greedy/two pointers/binary search hoặc comparator của bài.
- Vòng while là phần điều chỉnh state cho tới khi invariant trở lại đúng; khi học, cần nói được chính xác điều kiện while có nghĩa gì.
- Biến/điều kiện quan trọng nhất của bài: Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: Adj list weighted undirected; min-heap Dijkstra

F. Complexity + hidden-test traps

- **Độ phức tạp:** $O((V+E)\log V)$
- **Bẫy dễ sai:** Nhiều edge cùng cặp; heap implementation JS
- **Recall 30 giây:** Dijkstra → Adj list weighted undirected; min-heap Dijkstra

63. 전력망을 둘로 나누기

Chia lưới điện thành hai

Chủ đề	Graph/Shortest Path	Pattern	Tree cut + DFS
Priority	B	Complexity	$O(n^2)$

A. Đề bài tiếng Việt - đọc để hiểu đề

Mạng điện là một cây n node có $n-1$ dây. Cắt đúng một dây sẽ chia cây thành hai thành phần. Hãy chọn dây cắt sao cho chênh lệch số node hai phía nhỏ nhất. Có thể thử cắt từng edge rồi DFS/BFS đếm một component, hoặc tính subtree sizes một lần.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: Bỏ 1 edge, minimize abs sizes
- **Pattern**: Tree cut + DFS
- **Vì sao**: For each edge removed, DFS count one side; $\text{diff} = \text{abs}(n - 2 * \text{count})$

C. Tư duy lời giải từng bước

173. For each edge removed, DFS count one side.

174. $\text{diff} = \text{abs}(n - 2 * \text{count})$.

Invariant / state phải giữ

Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: For each edge removed, DFS count one side; $\text{diff} = \text{abs}(n - 2 * \text{count})$

D. JavaScript solution() - block code để học/copy

```
function solution(n, wires) {
  let best=n;
  for(let skip=0; skip<wires.length; skip++) {
    const adj=Array.from( {
      length:n+1
    }, ()=>[]);
    wires.forEach(([a, b], i)=> {
      if(i!==skip) {
        adj[a].push(b); adj[b].push(a);
      }
    });
    const q=[1], seen=new Set([1]);
    let h=0;
    while(h<q.length) {
      const u=q[h++];
      for(const v of adj[u])if(!seen.has(v)) {
        seen.add(v);
        q.push(v);
      }
    }
  }
}
```

```
    }  
  }  
  best=Math.min(best, Math.abs(n-2*seen.size));  
}  
return best;  
}
```

E. Đọc code theo cấu trúc

- Set dùng để kiểm tra uniqueness/visited hoặc loại trùng mà không cần vòng lặp lồng nhau.
- Vòng while là phần điều chỉnh state cho tới khi invariant trở lại đúng; khi học, cần nói được chính xác điều kiện while có nghĩa gì.
- Biến/điều kiện quan trọng nhất của bài: Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: For each edge removed, DFS count one side; $\text{diff} = \text{abs}(n - 2 * \text{count})$

F. Complexity + hidden-test traps

- **Độ phức tạp:** $O(n^2)$
- **Bẫy dễ sai:** $O(n^2)$ đủ vì n nhỏ
- **Recall 30 giây:** Tree cut + DFS $\rightarrow$ For each edge removed, DFS count one side; $\text{diff} = \text{abs}(n - 2 * \text{count})$

64. Hạng taxi 요금

Phí taxi đi chung

★ BẮT BUỘC theo roadmap team kia

Chủ đề	Graph/Shortest Path	Pattern	All-pairs shortest path
Priority	A	Complexity	$O(n^3)$

A. Đề bài tiếng Việt - đọc để hiểu đề

Có graph taxi với phí cạnh không âm. Hai người cùng xuất phát ở s , cần tới a và b ; họ có thể đi chung một đoạn rồi tách, hoặc tách ngay. Hãy tìm tổng phí nhỏ nhất. Nếu chọn điểm tách k , chi phí là $\text{dist}(s,k) + \text{dist}(k,a) + \text{dist}(k,b)$, nên cần all-pairs shortest paths hoặc ba lần Dijkstra.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: S, A, B ; chia sẻ tới k rồi tách, minimize $\text{dist}[s][k] + \text{dist}[k][a] + \text{dist}[k][b]$
- **Pattern**: All-pairs shortest path
- **Vì sao**: Floyd-Warshall $n \leq 200$ hoặc 3x Dijkstra

C. Tư duy lời giải từng bước

175. Floyd-Warshall $n \leq 200$ hoặc 3x Dijkstra.

176. Duy trì state/invariant bên dưới và cập nhật đáp án khi điều kiện của bài được thỏa..

Invariant / state phải giữ

Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: Floyd-Warshall $n \leq 200$ hoặc 3x Dijkstra

D. JavaScript solution() - block code để học/copy

```
function solution(n, s, a, b, fares) {
  const INF=1e15, d=Array.from( {
    length:n+1
  }, ()=>Array(n+1).fill(INF));
  for(let i=1; i<=n; i++)d[i][i]=0;
  for(const [u, v, w] of fares)d[u][v]=d[v][u]=Math.min(d[u][v], w);
  for(let k=1; k<=n; k++)for(let i=1; i<=n; i++)for(let j=1; j<=n; j++)d[i][j]=Math.min(d[i][j],
    d[i][k]+d[k][j]);
  let ans=INF;
  for(let k=1; k<=n; k++)ans=Math.min(ans, d[s][k]+d[k][a]+d[k][b]);
  return ans;
}
```

E. Đọc code theo cấu trúc

- Đọc code thành ba phần: khởi tạo state → cập nhật state theo từng phần tử/lựa chọn → trả đáp án. Đừng học thuộc từng dòng rồi rạc.
- Biến/điều kiện quan trọng nhất của bài: Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: Floyd-Warshall $n \leq 200$ hoặc 3x Dijkstra

F. Complexity + hidden-test traps

- **Độ phức tạp:** $O(n^3)$
- **Bẫy dễ sai:** Infinity init, multi-edge min
- **Recall 30 giây:** All-pairs shortest path → Floyd-Warshall $n \leq 200$ hoặc 3x Dijkstra

65. 섬 연결하기

Nối các đảo

Chủ đề	Graph/Shortest Path	Pattern	MST Kruskal
Priority	B	Complexity	$O(E \log E)$

A. Đề bài tiếng Việt - đọc để hiểu đề

Có n đảo và danh sách costs cho việc nối từng cặp đảo. Cần chọn một tập cầu sao cho tất cả đảo liên thông và tổng chi phí nhỏ nhất. Đây là Minimum Spanning Tree: sort edge theo cost và dùng Union-Find (Kruskal), hoặc Prim.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: Nối all nodes cost min
- **Pattern**: MST Kruskal
- **Vì sao**: Sort edges asc; DSU union nếu khác root; lấy $n-1$ edge

C. Tư duy lời giải từng bước

177. Sort edges asc.
178. DSU union nếu khác root.
179. lấy $n-1$ edge.

Invariant / state phải giữ

Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: Sort edges asc; DSU union nếu khác root; lấy $n-1$ edge

D. JavaScript solution() - block code để học/copy

```
function solution(n, costs) {
  costs.sort((a, b) => a[2] - b[2]);
  const p = Array.from( {
    length: n
  }, (_, i) => i);
  const f = x => p[x] === x ? x : (p[x] = f(p[x]));
  let ans = 0, cnt = 0;
  for(const [a, b, w] of costs) {
    let x = f(a), y = f(b);
    if(x === y) continue;
    p[y] = x;
    ans += w;
    if(++cnt === n - 1) break;
  }
  return ans;
}
```

E. Đọc code theo cấu trúc

- Phần sort chuẩn hóa thứ tự dữ liệu trước khi áp dụng greedy/two pointers/binary search hoặc comparator của bài.
- Biến/điều kiện quan trọng nhất của bài: Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mẫu chốt là: Sort edges asc; DSU union nếu khác root; lấy n-1 edge

F. Complexity + hidden-test traps

- **Độ phức tạp:** $O(E \log E)$
- **Bẫy dễ sai:** Path compression/rank
- **Recall 30 giây:** MST Kruskal $\rightarrow$ Sort edges asc; DSU union nếu khác root; lấy n-1 edge

66. 순위

Xếp hạng

Chủ đề	Graph/Shortest Path	Pattern	Transitive closure
Priority	B	Complexity	$O(n^3)$

A. Đề bài tiếng Việt - đọc để hiểu đề

Có kết quả so tài trực tiếp giữa người chơi. Nếu biết A thắng B và B thắng C thì suy ra A thắng C; tương tự cho quan hệ thua. Một người xác định được thứ hạng nếu biết quan hệ thắng/thua với mọi người còn lại. Hãy đếm số người có thứ hạng xác định. Có thể dùng Floyd-Warshall cho transitive closure.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: Biết quan hệ thắng thua, ai xác định được thứ hạng
- **Pattern**: Transitive closure
- **Vì sao**: Floyd boolean reachability; player known với mọi j nếu $i \rightarrow j$ hoặc $j \rightarrow i$

C. Tư duy lời giải từng bước

180. Floyd boolean reachability.

181. player known với mọi j nếu $i \rightarrow j$ hoặc $j \rightarrow i$.

Invariant / state phải giữ

Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: Floyd boolean reachability; player known với mọi j nếu $i \rightarrow j$ hoặc $j \rightarrow i$

D. JavaScript solution() - block code để học/copy

```
function solution(n, results) {
  const win=Array.from( {
    length:n
  }, ()=>Array(n).fill(false));
  for(const [a, b] of results)win[a-1][b-1]=true;
  for(let k=0; k<n; k++)for(let i=0; i<n; i++)for(let j=0; j<n; j++)win[i][j]=win[i][j] | (win[i][k]&&win[k][j]);
  let ans=0;
  for(let i=0; i<n; i++) {
    let known=0;
    for(let j=0; j<n; j++)if(i!==j&&(win[i][j] | win[j][i]))known++;
    if(known===n-1)ans++;
  }
  return ans;
}
```

E. Đọc code theo cấu trúc

- Đọc code thành ba phần: khởi tạo state → cập nhật state theo từng phần tử/lựa chọn → trả đáp án. Đừng học thuộc từng dòng rồi rạc.

- Biến/điều kiện quan trọng nhất của bài: Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mẫu chốt là: Floyd boolean reachability; player known với mọi j nếu $i \rightarrow j$ hoặc $j \rightarrow i$

F. Complexity + hidden-test traps

- **Độ phức tạp:** $O(n^3)$
- **Bẫy dễ sai:** Không tính self vào known count
- **Recall 30 giây:** Transitive closure $\rightarrow$ Floyd boolean reachability; player known với mọi j nếu $i \rightarrow j$ hoặc $j \rightarrow i$

67. 가장 큰 정사각형 찾기

Tìm hình vuông lớn nhất

Chủ đề	Dynamic Programming	Pattern	2D DP
Priority	B	Complexity	O(RC)

A. Đề bài tiếng Việt - đọc để hiểu đề

Cho ma trận nhị phân. Hãy tìm diện tích hình vuông lớn nhất chỉ gồm số 1. Với mỗi ô 1, kích thước hình vuông lớn nhất kết thúc tại ô đó bằng $1 + \min(\text{top}, \text{left}, \text{top-left})$. Lấy max kích thước rồi bình phương.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: Largest all-1 square
- **Pattern**: 2D DP
- **Vì sao**: Nếu $\text{board}[r][c] == 1$: $\text{dp} = 1 + \min(\text{top}, \text{left}, \text{diag})$; maxSide

C. Tư duy lời giải từng bước

182. Nếu $\text{board}[r][c] == 1$: $\text{dp} = 1 + \min(\text{top}, \text{left}, \text{diag})$.

183. maxSide .

Invariant / state phải giữ

$\text{dp}[r][c]$ là cạnh lớn nhất của hình vuông toàn 1 có góc dưới-phải tại (r, c) .

D. JavaScript solution() - block code để học/copy

```
function solution(board) {
  let best=0;
  for(let r=0; r<board.length; r++)for(let c=0; c<board[0].length; c++)if(board[r][c]&&board[r-1][c]&&board[r][c-1]&&board[r-1][c-1]) {
    board[r][c]=1+Math.min(board[r-1][c], board[r][c-1], board[r-1][c-1]);
    best=Math.max(best, board[r][c]);
  } else best=Math.max(best, board[r][c]);
  return best*best;
}
```

E. Đọc code theo cấu trúc

- Đọc code thành ba phần: khởi tạo state → cập nhật state theo từng phần tử/lựa chọn → trả đáp án. Đừng học thuộc từng dòng rồi rạc.
- Biến/điều kiện quan trọng nhất của bài: $\text{dp}[r][c]$ là cạnh lớn nhất của hình vuông toàn 1 có góc dưới-phải tại (r, c) .

F. Complexity + hidden-test traps

- **Độ phức tạp**: O(RC)
- **Bẫy dễ sai**: Cần xử lý first row/col
- **Recall 30 giây**: 2D DP → Nếu $\text{board}[r][c] == 1$: $\text{dp} = 1 + \min(\text{top}, \text{left}, \text{diag})$; maxSide

68. 정수 삼각형

Tam giác số nguyên

★ BẮT BUỘC theo roadmap team kia

Chủ đề	Dynamic Programming	Pattern	Path DP
Priority	A	Complexity	$O(n^2)$

A. Đề bài tiếng Việt - đọc để hiểu đề

Một tam giác số, bắt đầu ở đỉnh và mỗi bước xuống hàng dưới chỉ được chọn ô ngay dưới hoặc dưới-phải tương ứng. Hãy tìm tổng lớn nhất từ đỉnh tới đáy. DP mỗi ô = giá trị ô + max của các parent có thể tới nó.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: Max sum từ top xuống adjacent
- **Pattern**: Path DP
- **Vì sao**: $dp[r][c] = \text{value} + \max(\text{parentLeft}, \text{parentRight})$; có thể in-place

C. Tư duy lời giải từng bước

184. $dp[r][c] = \text{value} + \max(\text{parentLeft}, \text{parentRight})$.

185. có thể in-place.

Invariant / state phải giữ

$dp[r][c]$ là tổng lớn nhất có thể đạt khi đi từ đỉnh tới đúng ô (r, c) .

D. JavaScript solution() - block code để học/copy

```
function solution(triangle) {
  const dp=triangle.map(r=>[...r]);
  for(let r=1; r<dp.length; r++)for(let c=0; c<dp[r].length; c++)dp[r][c]+=Math.max(c?dp[r-1][c-1]:0,
  c<dp[r-1].length?dp[r-1][c]:0);
  return Math.max(...dp.at(-1));
}
```

E. Đọc code theo cấu trúc

- Đọc code thành ba phần: khởi tạo state → cập nhật state theo từng phần tử/lựa chọn → trả đáp án. Đừng học thuộc từng dòng rồi rạc.
- Biến/điều kiện quan trọng nhất của bài: $dp[r][c]$ là tổng lớn nhất có thể đạt khi đi từ đỉnh tới đúng ô (r, c) .

F. Complexity + hidden-test traps

- **Độ phức tạp**: $O(n^2)$
- **Bẫy dễ sai**: Boundary chỉ có 1 parent
- **Recall 30 giây**: Path DP → $dp[r][c] = \text{value} + \max(\text{parentLeft}, \text{parentRight})$; có thể in-place

69. 코딩 테스트 공부

Học coding test

★ BẮT BUỘC theo roadmap team kia

Chủ đề	Dynamic Programming	Pattern	2D state DP shortest cost
Priority	A	Complexity	$O(\text{targetA} * \text{targetC} * P)$

A. Đề bài tiếng Việt - đọc để hiểu đề

Bạn có điểm thuật toán alp và coding cop. Có các problem yêu cầu mức tối thiểu và khi giải sẽ tăng skill nhưng tốn thời gian; ngoài ra có thể học riêng từng skill, mỗi 1 điểm tốn 1 thời gian. Mục tiêu đạt đủ skill để giải được mọi problem với thời gian ít nhất. Đây là shortest path/DP trên state (alp, cop), cap state ở requirement tối đa.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: Tăng alp/cop bằng study hoặc solve problem, minimize time tới targets
- **Pattern**: 2D state DP shortest cost
- **Vì sao**: Clamp target=max required; dp[a][c]; transitions +1 study, problem if req met; cap next state

C. Tư duy lời giải từng bước

186. Clamp target=max required.
187. dp[a][c].
188. transitions +1 study, problem if req met.
189. cap next state.

Invariant / state phải giữ

Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: Clamp target=max required; dp[a][c]; transitions +1 study, problem if req met; cap next state

D. JavaScript solution() - block code để học/copy

```
function solution(alp, cop, problems) {
  const maxA=Math.max(...problems.map(p=>p[0])), maxC=Math.max(...problems.map(p=>p[1]));
  alp=Math.min(alp, maxA);
  cop=Math.min(cop, maxC);
  const INF=1e9, dp=Array.from( {
    length:maxA+1
  }, ()=>Array(maxC+1).fill(INF));
  dp[alp][cop]=0;
  for(let a=alp; a<=maxA; a++)for(let c=cop; c<=maxC; c++) {
    if(a<maxA)dp[a+1][c]=Math.min(dp[a+1][c], dp[a][c]+1);
    if(c<maxC)dp[a][c+1]=Math.min(dp[a][c+1], dp[a][c]+1);
    for(const [ra, rc, ga, gc, cost] of problems)if(a>=ra&&c>=rc) {
      const na=Math.min(maxA, a+ga), nc=Math.min(maxC, c+gc);
      dp[na][nc]=Math.min(dp[na][nc], dp[a][c]+cost);
    }
  }
}
```

```
}  
}  
return dp[maxA][maxC];  
}
```

E. Đọc code theo cấu trúc

- Đọc code thành ba phần: khởi tạo state → cập nhật state theo từng phần tử/lựa chọn → trả đáp án. Đừng học thuộc từng dòng rồi rạc.
- Biến/điều kiện quan trọng nhất của bài: Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: Clamp target=max required; dp[a][c]; transitions +1 study, problem if req met; cap next state

F. Complexity + hidden-test traps

- **Độ phức tạp:** $O(\text{targetA} * \text{targetC} * P)$
- **Bẫy dễ sai:** State space cap; đừng overshoot vô hạn
- **Recall 30 giây:** 2D state DP shortest cost → Clamp target=max required; dp[a][c]; transitions +1 study, problem if req met; cap next state

70. 등굣길

Đường đến trường

Chủ đề	Dynamic Programming	Pattern	Grid path counting DP
Priority	B	Complexity	$O(mn)$

A. Đề bài tiếng Việt - đọc để hiểu đề

Trên lưới $m \times n$, đi từ nhà tới trường chỉ bằng sang phải hoặc xuống dưới. Một số ô puddles bị chặn không đi qua được. Hãy đếm số đường đi, lấy modulo 1,000,000,007. DP tại mỗi ô = số cách từ trên + từ trái, trừ các ô bị chặn.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: Đếm đường right/down tránh puddles
- **Pattern**: Grid path counting DP
- **Vì sao**: $dp[1][1]=1$; if puddle skip else $dp=top+left \bmod$

C. Tư duy lời giải từng bước

190. $dp[1][1]=1$.

191. if puddle skip else $dp=top+left \bmod$.

Invariant / state phải giữ

$dp[r][c]$ là số đường hợp lệ tới ô đó, nên chỉ phụ thuộc hai predecessor hợp lệ: trên và trái.

D. JavaScript solution() - block code để học/copy

```
function solution(m, n, puddles) {
  const MOD=1000000007, blocked=new Set(puddles.map(([x, y])=>`${y},${x}`)), dp=Array.from( {
    length:n
  }, ()=>Array(m).fill(0));
  dp[0][0]=1;
  for(let r=0; r<n; r++)for(let c=0; c<m; c++) {
    if(blocked.has(`${r},${c}`)) {
      dp[r][c]=0;
      continue;
    }
    if(r===0&&c===0)continue;
    dp[r][c]=(r?dp[r-1][c]:0)+(c?dp[r][c-1]:0)%MOD;
  }
  return dp[n-1][m-1];
}
```

E. Đọc code theo cấu trúc

- Set dùng để kiểm tra uniqueness/visited hoặc loại trùng mà không cần vòng lặp lồng nhau.
- Biến/điều kiện quan trọng nhất của bài: $dp[r][c]$ là số đường hợp lệ tới ô đó, nên chỉ phụ thuộc hai predecessor hợp lệ: trên và trái.

F. Complexity + hidden-test traps

- **Độ phức tạp:** $O(mn)$
- **Bẫy dễ sai:** Coordinates puddle thường [col,row] dễ đảo
- **Recall 30 giây:** Grid path counting DP $\rightarrow$ $dp[1][1]=1$; if puddle skip else $dp=top+left \text{ mod}$

71. 거스름돈

Tiền thừa

Chủ đề	Dynamic Programming	Pattern	Coin change combinations
Priority	B	Complexity	$O(n \cdot \text{coins})$

A. Đề bài tiếng Việt - đọc để hiểu đề

Có các mệnh giá money và cần tạo đúng số tiền n, mỗi mệnh giá được dùng không giới hạn. Thứ tự lấy tiền không tạo cách mới; chỉ quan tâm tổ hợp số đồng. Hãy đếm số cách và lấy modulo theo đề. Đây là unbounded coin change với vòng ngoài theo coin để tránh đếm permutation.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: Đếm số cách tạo n với coin unlimited
- **Pattern**: Coin change combinations
- **Vì sao**: $dp[0]=1$; for coin outer, for sum coin..n: $dp[s] += dp[s - \text{coin}]$

C. Tư duy lời giải từng bước

192. $dp[0]=1$.

193. for coin outer, for sum coin..n: $dp[s] += dp[s - \text{coin}]$.

Invariant / state phải giữ

Sau khi xử lý coin thứ i, $dp[v]$ đếm số tổ hợp tạo v chỉ dùng các coin từ 0..i; vì vậy không đếm hoán vị.

D. JavaScript solution() - block code để học/copy

```
function solution(n, money) {
  const MOD=1000000007, dp=Array(n+1).fill(0);
  dp[0]=1;
  for(const coin of money)for(let s=coin; s<=n; s++)dp[s]=(dp[s]+dp[s-coin])%MOD;
  return dp[n];
}
```

E. Đọc code theo cấu trúc

- Đọc code thành ba phần: khởi tạo state → cập nhật state theo từng phần tử/lựa chọn → trả đáp án. Đừng học thuộc từng dòng rồi rạc.
- Biến/điều kiện quan trọng nhất của bài: Sau khi xử lý coin thứ i, $dp[v]$ đếm số tổ hợp tạo v chỉ dùng các coin từ 0..i; vì vậy không đếm hoán vị.

F. Complexity + hidden-test traps

- **Độ phức tạp**: $O(n \cdot \text{coins})$
- **Bẫy dễ sai**: Loop order quyết định combinations vs permutations
- **Recall 30 giây**: Coin change combinations → $dp[0]=1$; for coin outer, for sum coin..n: $dp[s] += dp[s - \text{coin}]$

72. 스티커 모으기(2)

Thu thập sticker (2)

Chủ đề	Dynamic Programming	Pattern	Circular house robber
Priority	B	Complexity	O(n)

A. Đề bài tiếng Việt - đọc để hiểu đề

Các sticker xếp thành vòng tròn; nếu lấy một sticker thì không được lấy hai sticker kề nó. Hãy tối đa tổng giá trị. Vì sticker đầu và cuối kề nhau, tách thành hai bài toán tuyến tính: lấy phạm vi $[0..n-2]$ hoặc $[1..n-1]$, rồi dùng DP house-robber.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: Chọn sticker không kề nhau trên vòng tròn
- **Pattern**: Circular house robber
- **Vì sao**: Case1 include range $0..n-2$; case2 $1..n-1$; linear DP $\max(\text{prev1}, \text{prev2}+v)$

C. Tư duy lời giải từng bước

194. Case1 include range $0..n-2$.
195. case2 $1..n-1$.
196. linear DP $\max(\text{prev1}, \text{prev2}+v)$.

Invariant / state phải giữ

DP tuyến tính giữ best khi không lấy / có thể lấy sticker hiện tại, trong một đoạn mà hai đầu không còn kề nhau.

D. JavaScript solution() - block code để học/copy

```
function solution(sticker) {
  if(sticker.length===1)return sticker[0];
  const rob=(l, r)=> {
    let prev2=0, prev1=0;
    for(let i=l; i<=r; i++) {
      const cur=Math.max(prev1, prev2+sticker[i]);
      prev2=prev1;
      prev1=cur;
    }
    return prev1;
  }
  ;
  return Math.max(rob(0, sticker.length-2), rob(1, sticker.length-1));
}
```

E. Đọc code theo cấu trúc

- Đọc code thành ba phần: khởi tạo state → cập nhật state theo từng phần tử/lựa chọn → trả đáp án. Đừng học thuộc từng dòng rồi rạc.

- Biến/điều kiện quan trọng nhất của bài: DP tuyến tính giữ best khi không lấy / có thể lấy sticker hiện tại, trong một đoạn mà hai đầu không còn kề nhau.

F. Complexity + hidden-test traps

- **Độ phức tạp:** $O(n)$
- **Bẫy dễ sai:** $n=1$ special
- **Recall 30 giây:** Circular house robber → Case1 include range $0..n-2$; case2 $1..n-1$; linear DP $\max(\text{prev1}, \text{prev2}+v)$

73. 자물쇠와 열쇠

Khóa và chìa khóa

Chủ đề	Matrix/Advanced Simulation	Pattern	Matrix rotate + overlay
Priority	C	Complexity	$O(4*N^2*M^2)$

A. Đề bài tiếng Việt - đọc để hiểu đề

Cho ma trận key và lock. Có thể xoay key 0/90/180/270 độ và tịnh tiến ở mọi vị trí; phần nhô của key (1) phải lấp đúng các lỗ của lock (0), đồng thời không được đè 1 lên 1 trong vùng lock. Hãy xác định có cách đặt nào mở được khóa. Cách dễ là tạo bảng mở rộng, đặt từng rotation tại mọi offset và kiểm tra vùng lock đều trở thành 1.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: Rotate key 4 lần, slide trên expanded board, lock region phải all 1
- **Pattern**: Matrix rotate + overlay
- **Vì sao**: Expand board 3N; for each offset add key, check center all1, rollback

C. Tư duy lời giải từng bước

197. Expand board 3N.

198. for each offset add key, check center all1, rollback.

Invariant / state phải giữ

Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: Expand board 3N; for each offset add key, check center all1, rollback

D. JavaScript solution() - block code để học/copy

```
function solution(key, lock) {
  const M=key.length, N=lock.length, size=N+2*(M-1);
  const rotate=a=>Array.from( {
    length:M
  }, (_, r)=>Array.from( {
    length:M
  }, (_, c)=>a[M-1-c][r]));
  let k=key;
  for(let rot=0; rot<4; rot++) {
    for(let sr=0; sr<=size-M; sr++)for(let sc=0; sc<=size-M; sc++) {
      let ok=true;
      for(let r=0; r<N&&ok; r++)for(let c=0; c<N; c++) {
        const kr=r+M-1-sr, kc=c+M-1-sc, kv=(kr>=0&&kr<M&&kc>=0&&kc<M)?k[kr][kc]:0;
        if(lock[r][c]+kv!==1) {
          ok=false;
          break;
        }
      }
    }
  }
}
```

```
    }  
    if(ok)return true;  
  }  
  k=rotate(k);  
}  
return false;  
}
```

E. Đọc code theo cấu trúc

- Đọc code thành ba phần: khởi tạo state → cập nhật state theo từng phần tử/lựa chọn → trả đáp án. Đừng học thuộc từng dòng rồi rạc.
- Biến/điều kiện quan trọng nhất của bài: Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: Expand board 3N; for each offset add key, check center all1, rollback

F. Complexity + hidden-test traps

- **Độ phức tạp:** $O(4*N^2*M^2)$
- **Bẫy dễ sai:** Overlap key 1 + lock 1 => 2 invalid; rollback đúng
- **Recall 30 giây:** Matrix rotate + overlay → Expand board 3N; for each offset add key, check center all1, rollback

74. 기둥과 보 설치

Lắp cột và dầm

Chủ đề	Matrix/Advanced Simulation	Pattern	Constraint simulation
Priority	C	Complexity	O(commands * structures)

A. Đề bài tiếng Việt - đọc để hiểu đề

Trên lưới tọa độ, nhận các lệnh BUILD/REMOVE cột hoặc dầm. Sau mỗi thao tác, toàn bộ cấu trúc phải thỏa quy tắc: cột đứng trên mặt đất/đầu cột/đầu dầm; dầm có ít nhất một đầu trên cột hoặc hai đầu nối với dầm khác. Chỉ thực hiện thao tác nếu trạng thái sau thao tác hợp lệ. Cuối cùng trả về các phần tử đã lắp theo thứ tự x, y, loại.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: Add/remove structure nếu toàn bộ config vẫn hợp lệ
- **Pattern**: Constraint simulation
- **Vì sao**: Store set posts/beams; tentatively apply then validate all structures by local rules; revert if invalid

C. Tư duy lời giải từng bước

199. Store set posts/beams.
200. tentatively apply then validate all structures by local rules.
201. revert if invalid.

Invariant / state phải giữ

Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: Store set posts/beams; tentatively apply then validate all structures by local rules; revert if invalid

D. JavaScript solution() - block code để học/copy

```
function solution(n, build_frame) {
  const set=new Set(), key=(x, y, a)=>`${x},${y},${a}`;
  const valid=(x, y, a)=> {
    if(a===0) return y===0 || set.has(key(x, y-1, 0)) || set.has(key(x-1, y, 1)) ||
    set.has(key(x, y, 1));
    return set.has(key(x, y-1, 0)) || set.has(key(x+1, y-1, 0)) ||
    (set.has(key(x-1, y, 1))&&set.has(key(x+1, y, 1)));
  }
  ;
  const allValid=()=> {
    for(const s of set) {
      const [x, y, a]=s.split(',').map(Number);
      if(!valid(x, y, a))return false;
    }
    return true;
  }
  ;
}
```



```

for(const [x, y, a, b] of build_frame) {
  const k=key(x, y, a);
  if(b===1) {
    set.add(k);
    if(!valid(x, y, a))set.delete(k);
  } else {
    set.delete(k);
    if(!allValid())set.add(k);
  }
}
return [...set].map(s=>s.split(',').map(Number)).sort((p, q)=>p[0]-q[0] || p[1]-q[1] || p[2]-q[2]);
}

```

E. Đọc code theo cấu trúc

- Phân sort chuẩn hóa thứ tự dữ liệu trước khi áp dụng greedy/two pointers/binary search hoặc comparator của bài.
- Set dùng để kiểm tra uniqueness/visited hoặc loại trùng mà không cần vòng lặp lồng nhau.
- Biến/điều kiện quan trọng nhất của bài: Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: Store set posts/beams; tentatively apply then validate all structures by local rules; revert if invalid

F. Complexity + hidden-test traps

- **Độ phức tạp:** $O(\text{commands} * \text{structures})$
- **Bẫy dễ sai:** Rule boundaries; remove ảnh hưởng neighbors
- **Recall 30 giây:** Constraint simulation → Store set posts/beams; tentatively apply then validate all structures by local rules; revert if invalid

75. 가사 검색

Tìm lời bài hát

Chủ đề	Trie/Advanced Search	Pattern	Trie / length buckets + binary search
Priority	C	Complexity	$O((N+Q)\log N * L)$

A. Đề bài tiếng Việt - đọc để hiểu đề

Có nhiều từ lyrics và nhiều query chứa chữ thường cùng ký tự '?'. Dấu ? chỉ xuất hiện liên tiếp ở đầu hoặc cuối query và khớp với bất kỳ ký tự đơn nào; chiều dài từ phải bằng chiều dài query. Với rất nhiều truy vấn, kiểm tra từng từ sẽ chậm. Có thể nhóm Trie theo độ dài và xây cả trie xuôi/lùi, hoặc nhóm chuỗi theo độ dài rồi binary search trên khoảng prefix/suffix.

- **Input** theo signature: xem code
- **Output**: trả về đúng giá trị/cấu trúc được mô tả ở cuối phần đề bài.

B. Nhìn đề phải nghĩ gì?

- **Dấu hiệu nhận diện**: Wildcard liên tiếp ở đầu hoặc cuối, count matching words
- **Pattern**: Trie / length buckets + binary search
- **Vì sao**: Group words by length; sort normal and reversed; query wildcard suffix via range bounds; hoặc trie prefix/suffix

C. Tư duy lời giải từng bước

202. Group words by length.
203. sort normal and reversed.
204. query wildcard suffix via range bounds.
205. hoặc trie prefix/suffix.

Invariant / state phải giữ

Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: Group words by length; sort normal and reversed; query wildcard suffix via range bounds; hoặc trie prefix/suffix

D. JavaScript solution() - block code để học/copy

```
function solution(words, queries) {
  const normal=new Map(), rev=new Map();
  for(const w of words) {
    const L=w.length;
    if(!normal.has(L)) {
      normal.set(L, []);
      rev.set(L, []);
    }
    normal.get(L).push(w);
    rev.get(L).push([...w].reverse().join(""));
  }
  for(const a of normal.values())a.sort();
  for(const a of rev.values())a.sort();
  const lower=(a, x)=> {
```

```

let l=0, r=a.length;
while(l<r) {
  const m=(l+r)>>1;
  if(a[m]>=x)r=m;
  else l=m+1;
}
return l;
}
;
const upper=(a, x)=> {
  let l=0, r=a.length;
  while(l<r) {
    const m=(l+r)>>1;
    if(a[m]>x)r=m;
    else l=m+1;
  }
  return l;
}
;
return queries.map(q=> {
  const L=q.length, front=q[0]==='?', arr=front?(rev.get(L)??):(normal.get(L)??):[], s=front?[...q].reverse().join(''):q;
  const lo=s.replace(/\?/g, 'a'),
    hi=s.replace(/\?/g, 'z'); return upper(arr, hi)-lower(arr, lo);
}
);
}

```

E. Đọc code theo cấu trúc

- Phần sort chuẩn hóa thứ tự dữ liệu trước khi áp dụng greedy/two pointers/binary search hoặc comparator của bài.
- Map lưu tần suất / ánh xạ / trạng thái để truy cập $O(1)$ trung bình thay vì quét lại dữ liệu.
- Vòng while là phần điều chỉnh state cho tới khi invariant trở lại đúng; khi học, cần nói được chính xác điều kiện while có nghĩa gì.
- Biến/điều kiện quan trọng nhất của bài: Các biến trạng thái phải luôn phản ánh đúng phần dữ liệu đã xử lý. Với bài này, mấu chốt là: Group words by length; sort normal and reversed; query wildcard suffix via range bounds; hoặc trie prefix/suffix

F. Complexity + hidden-test traps

- **Độ phức tạp:** $O((N+Q)\log N * L)$
- **Bẫy dễ sai:** Wildcard only contiguous; query all ?
- **Recall 30 giây:** Trie / length buckets + binary search → Group words by length; sort normal and reversed; query wildcard suffix via range bounds; hoặc trie prefix/suffix

Phụ lục - 75 bài nhìn một dòng

01. **Thời hạn hiệu lực thu thập thông tin cá nhân** — Date normalization / parsing | Đổi YYYY.MM.DD -> tổng số ngày theo lịch giả 28 ngày; Map term->months; expire = start + term*28 - 1
02. **Tiến hành bài tập** — Timeline + stack | Sort start time; dùng stack lưu [task, remaining]; xử lý gap tới task kế tiếp; tranh thủ hoàn thành task bị pause
03. **Xoay viền ma trận** — Matrix boundary rotation | Giữ temp một góc; đi 4 cạnh đúng thứ tự; cập nhật min
04. **Chu kỳ đường đi của tia sáng** — State graph cycle | State=(r,c,dir), mỗi state chỉ thuộc 1 cycle; mô phỏng tới state đã thăm
05. **Tic-tac-toe chơi một mình** — State validation | Đếm O/X; kiểm tra win rows/cols/diag; áp invariant lượt chơi
06. **Xe buýt đưa đón** — Schedule + greedy | Sort crew times; simulate boarding; chuyển cuối: còn chỗ -> giờ bus, đầy -> lastCrew-1
07. **Nén chuỗi** — Chunk simulation | For len=1..n/2; scan chunk; count repeats; build length
08. **Biến đổi dấu ngoặc** — Recursive parsing | Find shortest balanced u; nếu correct => u+rec(v); else "("+rec(v)+"")"+reverse(u inside)
09. **Tối đa hóa biểu thức** — Parsing + permutation | Parse nums/operators; thử mọi permutation operator; reduce list theo precedence
10. **Pokémon** — Set cardinality | answer=min(uniqueCount,n/2)
11. **Trang phục** — Frequency product | Map type->count; product(count+1)-1
12. **Album hay nhất** — Group + multi-key sort | Map genre total + song list; sort genre desc total; song desc plays, asc index
13. **Số thứ K** — Slice + sort | slice(i-1,j).sort((a,b)=>a-b)[k-1]
14. **Số lớn nhất** — Custom comparator | map(String).sort((a,b)=>(b+a).localeCompare(a+b)); join; xử lý all zero
15. **H-Index** — Sort + threshold | Sort desc; tăng h khi citations[i]>=i+1
16. **Máy gắp thú bông** — Simulation + stack | For move: scan rows; push/pop stack; count +=2 khi trùng top
17. **Dấu ngoặc hợp lệ** — Stack / counter | Counter +1/-1; nếu âm return false; cuối phải 0
18. **Loại bỏ từng cặp** — Adjacent cancellation stack | Nếu top==ch -> pop, else push
19. **Xe tải qua cầu** — Queue simulation | Queue [weight, exitTime] hoặc queue fixed length; mỗi tick pop xe rời, thử push xe mới
20. **Phát triển chức năng** — Queue/grouping | days=ceil((100-progress)/speed); group liên tiếp nếu day<=currentReleaseDay
21. **Tiến trình** — Queue + priority | Queue indices; nếu còn priority lớn hơn thì push lại, else execute
22. **Game phòng thủ** — Min-heap / greedy undo | Trừ soldiers mỗi wave, push enemy to max-heap; hoặc min-heap giữ k wave lớn nhất được skip
23. **Hàng đợi ưu tiên kép** — Dual heap / multiset | Hai heap + lazy deletion / Map counts; hoặc JS sorted array nếu constraints chịu được
24. **Thi thử** — Pattern cycling | score[i%pattern.length], lấy max, return indices
25. **Tám thăm** — Factor search | total=b+y; thử h từ 1..sqrt(total), w=total/h; (w-2)(h-2)==y
26. **Chấm điểm** — Math counting | For x=0;k;... <=d: yMax=floor(sqrt(d^2-x^2)/k)*k; add yMax/k+1
27. **Khai thác khoáng sản** — Greedy grouping | Chỉ xét số mineral đào được = picks*5; group 5; score group theo độ khó; sort desc; gán pick tốt nhất
28. **Thuyền cứu sinh** — Two pointers | Sort asc; left lightest/right heaviest; luôn lấy right, nếu left+right<=limit thì left++
29. **Camera tốc độ** — Interval greedy | Sort routes theo end asc; nếu start>camera thì đặt camera=end

- 30. Tập hợp tốt nhất** — Equal distribution | Nếu $s < n \rightarrow [-1]$; chia đều: $\text{base} = \text{floor}(s/n)$, remainder; output nondecreasing
- 31. Làm tổng hai queue bằng nhau** — Two queues / two pointers | $\text{target} = (\text{sum1} + \text{sum2})/2$; nếu $\text{sum1} < \text{target}$ lấy từ $q2$, $> \text{target}$ lấy $q1$; head indices
- 32. Tổng dãy con liên tiếp** — Sliding window positive | right mở rộng sum; while $\text{sum} > k$ shrink; khi $\text{sum} == k$ update length
- 33. Tòa nhà không bị phá hủy** — 2D difference / prefix | Diff 2D: 4 corner updates; prefix ngang rồi dọc; cộng board
- 34. Mua sắm đá quý** — Variable sliding window + Map | Need=Set size; right add; while $\text{map.size} == \text{need}$ update answer rồi remove left
- 35. Thử thách game giải đố** — Binary search on answer | feasible(level) tính total; level tăng $\Rightarrow$ time không tăng; lower_bound first true
- 36. Tìm kiếm xếp hạng** — Precompute buckets + lower_bound | Mỗi applicant sinh 16 key wildcard; mỗi key sort scores; query lower_bound $\geq$ score
- 37. Kiểm tra nhập cảnh** — Binary search on answer | feasible(t) = $\sum \text{floor}(t/\text{time}[i]) \geq n$; search $[1, \text{max} * \text{time} * n]$
- 38. Bảng qua đá** — Binary search on answer | feasible(x): có xuất hiện k stones với $\text{stone} - x < 0$ / $\text{stone} < x$? xác định convention rồi search max true
- 39. Độ mật mỗi** — Permutation DFS | visited[]; DFS(currentK, count); thử mọi dungeon đủ min; mark $\rightarrow$ recurse $\rightarrow$ unmark
- 40. N quân hậu** — Backtracking constraints | row-by-row; usedCol, $\text{diag1} = r - c$, $\text{diag2} = r + c$
- 41. Khóa ứng viên** — Combination + set | Enumerate bitmasks/combinations; uniqueness bằng Set row signatures; minimal: không chứa candidate cũ
- 42. Làm mới menu** — Combination counting | Sort chars mỗi order; generate combinations size k; Map count; lấy $\text{max} \geq 2$
- 43. Từ điển nguyên âm** — DFS enumeration / positional weights | DFS theo lexicographic và counter; hoặc weights [781,156,31,6,1]
- 44. Cuộc thi bắn cung** — Backtracking allocation | DFS score $10 \rightarrow 0$, mỗi ô chọn thẳng apeach bằng $a[i] + 1$ hoặc bỏ; leftover dồn score0; tie low-score more
- 45. Kiểm tra tường ngoài** — Circular + permutation | Duplicate weak + n; thử mỗi start; permutations dist; greedy cover bằng từng friend
- 46. Cừu và sói** — State-space DFS | DFS(sheep, wolf, availableNodes); chọn 1 available, thêm children, recurse
- 47. Lộ trình du lịch** — Euler-like backtracking lexical | Sort tickets lexical; DFS with used[]; stop khi route length = tickets + 1
- 48. Người dùng xấu** — Matching + set of sets | Precompute candidates; DFS per banned id; used users; canonical sorted set key
- 49. Khoảng cách ngắn nhất trên bản đồ game** — Grid BFS shortest path | Queue + head index; visited/dist; first reach target shortest
- 50. Robot Ricochet** — BFS with slide transition | BFS states cells; neighbor = slide(dir) until blocked; enqueue landing cell
- 51. Thoát mê cung** — Two-stage BFS | BFS $S \rightarrow L$ rồi $L \rightarrow E$, cộng dist; thiếu đoạn nào -1
- 52. Kiểm tra giãn cách** — Local BFS / Manhattan | Từ mỗi P BFS depth ≤ 2 qua non-X; thấy P khác $\rightarrow$ invalid
- 53. Du lịch đảo hoang** — Connected components | For each unvisited non-X BFS/DFS accumulate; sort sums
- 54. Biến đổi từ** — Implicit graph BFS | BFS words; edge if diffCount == 1; visited
- 55. Di chuyển khối** — BFS composite state | Canonical state = sorted 2 cells; generate 4 translations + rotations khi 2 ô bên cạnh trống

56. **Xây đường đua** — Shortest path with direction state | State=(r,c,dir); dist 3D; relax next cost; queue/Dijkstra-like
57. **Lắp mảnh ghép puzzle** — Components + normalization + rotation | Extract components; normalize coords to origin; generate 4 rotations normalized; multiset match by shape
58. **Network** — Connected components graph | For i unvisited DFS/BFS, count++
59. **Gộp bảng** — Union-find + value semantics | DSU parent for groups + value on root; MERGE roots; UNMERGE tách mọi member giữ value ở selected cell
60. **Game tìm đường** — Build binary tree | Sort y desc,x asc; BST insert by x; traverse
61. **Bán bàn chải đa cấp** — Parent chain propagation | Map name->index,parent; mỗi sale while amount>0: give=floor(amount*0.1), keep=amount-give
62. **Giao hàng** — Dijkstra | Adj list weighted undirected; min-heap Dijkstra
63. **Chia lưới điện thành hai** — Tree cut + DFS | For each edge removed, DFS count one side; diff=abs(n-2*count)
64. **Phí taxi đi chung** — All-pairs shortest path | Floyd-Warshall n<=200 hoặc 3x Dijkstra
65. **Nối các đảo** — MST Kruskal | Sort edges asc; DSU union nếu khác root; lấy n-1 edge
66. **Xếp hạng** — Transitive closure | Floyd boolean reachability; player known với mọi j nếu i->j hoặc j->i
67. **Tìm hình vuông lớn nhất** — 2D DP | Nếu board[r][c]==1: dp=1+min(top,left,diag); maxSide
68. **Tam giác số nguyên** — Path DP | dp[r][c]=value+max(parentLeft,parentRight); có thể in-place
69. **Học coding test** — 2D state DP shortest cost | Clamp target=max required; dp[a][c]; transitions +1 study, problem if req met; cap next state
70. **Đường đến trường** — Grid path counting DP | dp[1][1]=1; if puddle skip else dp=top+left mod
71. **Tiền thừa** — Coin change combinations | dp[0]=1; for coin outer, for sum coin..n: dp[s]+=dp[s-coin]
72. **Thu thập sticker (2)** — Circular house robber | Case1 include range 0..n-2; case2 1..n-1; linear DP max(prev1,prev2+v)
73. **Khóa và chìa khóa** — Matrix rotate + overlay | Expand board 3N; for each offset add key, check center all1, rollback
74. **Lắp cột và dầm** — Constraint simulation | Store set posts/beams; tentatively apply then validate all structures by local rules; revert if invalid
75. **Tìm lời bài hát** — Trie / length buckets + binary search | Group words by length; sort normal and reversed; query wildcard suffix via range bounds; hoặc trie prefix/suffix